Time-Space Trade-Offs for Sumcheck

Anubhav Baweja
abaweja@upenn.edu
UPenn

Alessandro Chiesa
alessandro.chiesa@epfl.ch
EPFL

Elisabetta Fedele
efedele@ethz.ch
ETH Zürich

Giacomo Fenzi
giacomo.fenzi@epfl.ch
EPFL

Pratyush Mishra
prat@upenn.edu
UPenn

Tushar Mopuri
tmopuri@upenn.edu
UPenn

Andrew Zitek-Estrada
andrew.zitek@epfl.ch
EPFL

August 13, 2025

Abstract

The sumcheck protocol is a fundamental building block in the design of probabilistic proof systems, and has become a key component of recent work on efficient succinct arguments.

We study time-space tradeoffs for the prover of the sumcheck protocol in the streaming model, and provide upper and lower bounds that tightly characterize the efficiency achievable by the prover.

- For sumcheck claims about a single multilinear polynomial we demonstrate an algorithm that runs in time $O(kN)$ and uses space $O(N^{1/k})$ for any $k \geq 1$. For non-adaptive provers (a class which contains all known sumcheck prover algorithms) we show this tradeoff is optimal.
- For sumcheck claims about products of multilinear polynomials, we describe a prover algorithm that runs in time $O(N(\log\log N + k))$ and uses space $O(N^{1/k})$ for any $k \geq 1$. We show that, conditioned on the hardness of a natural problem about multiplication of multilinear polynomials, any "natural" prover algorithm that uses space $O(N^{1/2-\varepsilon})$ for some $\varepsilon > 0$ must run in time $\Omega(N(\log\log N + \log\varepsilon))$.

We implement and evaluate the prover algorithm for products of multilinear polynomials. We show that our algorithm consumes up to $120\times$ less memory compare to the linear-time prover algorithm, while incurring a time overhead of less than $2\times$.

The foregoing algorithms and lowerbounds apply in the interactive proof model. We show that in the polynomial interactive oracle proof model one can in fact design a new protocol that achieves a better time-space tradeoff of $O(N^{1/k})$ space and $O(N(\log^* N + k))$ time for any $k \geq 1$.

Contents

1 Introduction

The sumcheck protocol [LFKN92] occupies a central role in the study of probabilistic proof systems. Its applications span a wide range of areas, from complexity theory [BFL91; BFLS91; Sha92; GKR15] to the construction of concretely efficient succinct argument systems [Set20; CBBZ23; STW24]. Hence, it is of fundamental importance to understand the time and space efficiency of the prover in the sumcheck protocol, especially when considering the application to succinct arguments. Indeed, much effort has been devoted recently to construct succinct arguments [BHRRS20; BHRRS21; BCHO22; ZCLKZ24; PP24; NTZ25] that reduce the prover's memory usage, and a key bottleneck in a number of these works [ZCLKZ24; NTZ25] has been the prover's time and space complexity in the sumcheck protocol.

Existing state-of-the-art. We focus on prover algorithms for sumcheck of products of n-variate multilinear polynomials where the prover has streaming access to evaluations of the multilinear polynomials over the boolean hypercube, as this is the setting that sumcheck-based succinct arguments typically operate in.In this setting, prior algorithms achieve either $O(N)$-time and $O(N)$-space complexity [CTY11], or $O(N \log N)$-time and $O(\log N)$-space complexity [CMT12].

1.1 Our contributions

In this work, we study time-space tradeoffs for the prover of the sumcheck protocol, and provide upper and lower bounds that tightly characterize the efficiency achievable by the prover in a wide range of settings. We prove the following theorems (over any field).

Theorem 1 (informal, see Theorem 4.1)**.** *For any $k \geq 1$, we construct a sumcheck prover algorithm which performs $O(N(\log\log N + k))$ field operations and uses $O(N^{1/k})$ space.*

Note that for space $O(\log N)$, the algorithm runs in time $O\left(\frac{N \log N}{\log\log N}\right)$, which improves upon the $O(N \log N)$ time complexity of prior work [CMT12].

We show that the algorithm in Theorem 2 achieves an essentially optimal time-space tradeoff.

Theorem 2 (informal, see Theorem 6.5)**.** *Assuming the worst-case hardness of a certain natural problem regarding polynomial multiplication, every "natural" prover algorithm for the sumcheck protocol that maintains an internal state of size $N^{1/2-\varepsilon}$ for some $\varepsilon \in (0, 1/2)$, must use $\Omega(N(\log\log N + \log \varepsilon))$ field operations.*

Unlike our upper bound, our lower bound applies to algorithms that have *random access* to the input polynomials. We note that our lower bounds apply to *non-adaptive* algorithms whose accesses to the input do not depend on the verifier challenges nor the input polynomials. All known provers for the sumcheck protocol satisfy this property [CTY11; CMT12; Tha13], including the ones we study in this work.

Tight bounds for multilinear polynomials. When the sumcheck claim is about a single multilinear polynomial, we improve these results and provide a tight characterization of the time-space tradeoff for the prover of the sumcheck protocol: there exists a prover algorithm that uses $O(N^{1/k})$ space and runs in time $O(kN)$, and this is optimal. These results are of particular interest when the characteristic of the field is 2, where the techniques of [JKRS09] do not apply.

Concrete efficiency. We show that our algorithm exhibit a time-space tradeoff that is of practical interest. In particular, the algorithm underlying Theorem 1, when parameterized with $k = 2$ (i.e., to use $O(N^{1/2})$ space), is only 1.8-2$\times$ slower than the linear-space prover algorithm of [CTY11], and is 5-6$\times$ faster than the

log-space streaming prover of [CMT12]. Our algorithm requires approximately $0.008\times$ the memory required by the linear-time prover.

Sumcheck protocols in the PIOP model. Our lower bounds apply to interactive *proofs*. To do better we design a family of protocols with better efficiency in the polynomial interactive oracle proof (PIOP) model [BFS20; CHMMVW20] that bypass our lower bound and exhibit a better time-space tradeoff:

Theorem 3 (informal, see Theorem 7.2)**.** *For any $k \geq 1$ and FFT-friendly field $\mathbb{F}$, we construct a $O(\log^* N + k)$-round PIOP for sumcheck claims of product of polynomials over $\mathbb{F}$ such that the prover algorithm performs $O(N(\log^* N + k))$ field operations and uses $O(N^{1/k})$ space.*

Our protocol improves upon Sparrow [PP24], a recent work which introduced a new protocol for sumcheck claims about products of multilinear polynomials, and described a prover algorithm for this protocol that requires $O(N \log \log N)$ field operations and $O(N^{1/2})$ space .

In the linear space regime, we present a protocol with a linear time prover and $O(\log^* N)$ round complexity, proving a generalization of the sumcheck protocol of [ARR25] to the large field setting[1].

Theorem 4 (informal, see Theorem 7.3)**.** *For any FFT-friendly field $\mathbb{F}$, we construct a $O(\log^* N)$-round PIOP for sumcheck claims about sums of products of polynomials over $\mathbb{F}$ where the prover performs $O(N)$ field operations and uses $O(N)$ space.*

The foregoing algorithm achieves *linear prover time* with a round complexity of $O(\log^* N)$, as opposed to the $O(\log \log N)$ round complexity achieved by the linear-time sumcheck provers of [LMN25; SP25].

1.2 Related work

Existing algorithms for the sumcheck protocol. As mentioned in Section 1, existing prover algorithms for the sumcheck protocol achieve various points in the time-space tradeoff spectrum. Cormode, Mitzenmacher, and Thaler [CMT12] give a prover algorithm that performs $O(N \log N)$ field operations and uses $O(\log N)$ space, while Cormode, Thaler, and Yi [CTY11] give a prover algorithm that perform $O(N)$ field operations and uses $O(N)$ space. In Fig. 1, we compare the points in the trade-off space achieved by these algorithms to the trade-offs achieved by ours, and additionally juxtapose these with our lower bounds.

A recent note by a subset of the authors [CFFZ24] introduced the prover algorithm for multilinear polynomials described in Appendix A; this work subsumes that note and complements it with a lower bound that shows that the time-space tradeoff achieved by the algorithm is optimal.

Setty, Thaler, and Wahby [STW24, Appendix G] propose a prover for the classical sumcheck protocol for products of multilinear polynomials where one one of the polynomials is *sparse*. The techniques in their work are similar to those employed in Appendix A (when we set the dense polynomial to be identically one), but are not identical. In particular, space-efficiency was not a focus of their work, and an adaptation of the protocol must be performed to capture this feature of our algorithms. In concurrent work, Nair, Thaler, and Zhu [NTZ25, Appendix A] show how to perform this adaptation, and their technique generalizes the algorithm in Appendix A.

Low-memory succinct arguments from space-efficient sumcheck. Recent works [ZCLKZ24; NTZ25] have constructed succinct arguments with reduced memory requirements by using the space-efficient sumcheck prover of [CMT12]. Using the prover algorithm of Theorem 1 in these arguments would directly reduce the time complexity of their prover algorithms, while maintaining the same space complexity.

[1] [ARR25] constructed a linear time sumcheck protocol with $O(\log^* N)$ round complexity over binary fields, and stated but did not prove a generalization to the large field setting.

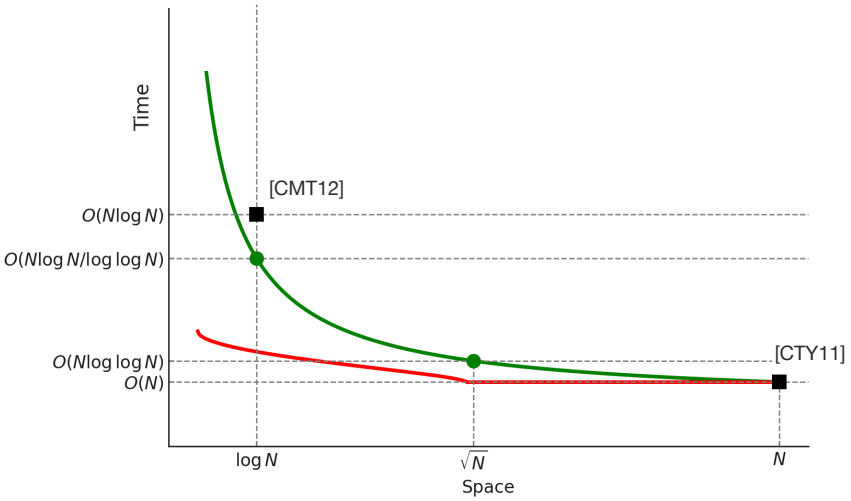


Figure 1: Our upper (green) and lower (red) bounds for the sumcheck protocol for products of multilinear polynomials. While Theorem 1 as stated achieves $O(N \log\log N)$ time when $k = 1$ (i.e. with $O(N)$ space), we can modify the algorithm to switch to the linear-time algorithm [CTY11] after $\log N - \log N/k$ rounds. This leads to a smooth tradeoff that achieves $O(N)$ time as k approaches 1. See Section 4 for details.
The lower bound curve above only reflects the product-sumcheck lower bound of Theorem 2. However, if the space is at most $N^{1/\log\log N}$, then the multilinear-sumcheck lower bound (see Theorem 2.5) dominates, and in fact *matches* the upper bound curve above (up to constants).

Alternate sumcheck protocols and their provers. We survey some alternate approaches to proving sumcheck claims.

Aurora [BCRSVW19] proposes a *univariate* sumcheck protocol for checking the sum of a univariate polynomial over a multiplicative subgroup that run in $O(N \log N)$ field operations. Gemini [BCHO22] proposes a different sumcheck protocol for univariate polynomials with two different prover algorithms: a space-intensive algorithm that requires $O(N)$ time and $O(N)$ space, and a space-efficient algorithm that requires $O(N \log N)$ time and $O(\log N)$ space. It would be interesting to see if our techniques can be adapted to achieve a similar time-space tradeoff for the Gemini protocol.

As noted previously, Sparrow [PP24] proposes a new PIOP for sumcheck of products of multilinear polynomials, and describes a prover algorithm for it that performs $O(N \log\log N)$ field operations and uses space $O(N^{1/2})$.

Two recent works [LMN25; SP25] design PIOPs for sumcheck of products of multilinear polynomials that achieve lower round complexity of $O(\log\log N)$. They provide provers that perform $O(N)$ field operations and use space $O(N)$. Hobbit [PP25] also designs a PIOP for sumcheck of products of multilinear polynomial, achieving provers that performs $O(N)$ field operations and use space $O(N^{1/2})$, albeit at the cost of an increased round and verifier complexity of $O(N^{1/2})$.

Finally, Athamnah et al. [ARR25] design a linear time sumcheck prover for sumcheck claims of products of multilinear polynomials with $O(\log^* N)$ round complexity over binary fields. We prove that their algorithm generalizes to the large field setting in Theorem 4 (as stated but not proven therein). We further provide in Theorem 3 a space-time tradeoff for a similar family of protocols.

Alternate memory models. Scribe [BMMNW24] proposes a *read-write* streaming computation model

where, the prover has random-access to only a limited amount of memory, but additionally has *read-write* streaming access to a large amount of external memory. In this model, Scribe achieves a prover algorithm for the sumcheck protocol that performs $O(N)$ field operations while using $O(\log N)$ random-access space and $O(N)$ external memory. This reliance on $O(N)$ external memory means that Scribe's algorithm does not violate the lower bound of Theorem 2.

2 Technical overview

We provide a brief overview of our techniques and results in this section. We begin by introducing the necessary background and notation in Section 2.1. We then present a space-efficient multilinear-sumcheck prover in Section 2.2, which runs in $O(N)$ time and uses $O(\sqrt{N})$ space. This acts a warm-up for our main upper bound result, which is a space-efficient product-sumcheck prover in Section 2.3 that runs in $O(N \log\log N)$ time and uses $O(\sqrt{N})$ space. We then present our lower bound results in Sections 2.4 and 2.5, which show that the time-space tradeoffs achieved by our provers are optimal for non-adaptive provers. We conclude with a discussion of how to bypass the product-sumcheck lower bound by allowing the prover to send polynomial oracles in Section 2.6.

2.1 Background and notation

We use $\langle i \rangle_n$ to represent the n-bit binary representation of $i \in \mathbb{N}$ in the most-significant-bit order. Given a multilinear polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$, we use $\boldsymbol{p} := [p(\langle i \rangle)]_{i=0}^{2^n-1}$ to denote its evaluations over the n-dimensional boolean hypercube $\{0,1\}^n \subset \mathbb{F}^n$. The *multilinear extension* of a vector $\boldsymbol{v} \in \mathbb{F}^{2^n}$ is the unique multilinear polynomial $\hat{v}(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$ such that $\hat{v}(\boldsymbol{i}) = \boldsymbol{v}[\mathsf{int}(\boldsymbol{i})]$ for all $\boldsymbol{i} \in \{0,1\}^n$. We always use N to denote 2^n. Given a finite set S, we use $x \xleftarrow{\$} S$ to denote that x is chosen uniformly at random from S.

Lagrange basis polynomials. The *Lagrange basis polynomial* with respect to $\boldsymbol{y} \in \{0,1\}^n$ is defined as $\chi_{\boldsymbol{y}}(\boldsymbol{X}) := \prod_{i=1}^{n}(y_i X_i + (1-y_i)(1-X_i))$. This is a multilinear polynomial which evaluates to 1 at $\boldsymbol{y}$ and 0 at all other points in $\{0,1\}^n$. Given a multilinear polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$ and an evaluation point $\boldsymbol{z} \in \mathbb{F}^n$, one can write

$$p(\boldsymbol{z}) = \sum_{\boldsymbol{x} \in \{0,1\}^n} \chi_{\boldsymbol{x}}(\boldsymbol{z}) \cdot p(\boldsymbol{x}) \ .$$

Streaming oracles. Given a vector $\boldsymbol{v} \in \mathbb{F}^N$, $\mathcal{S}(\boldsymbol{v})$ is a streaming oracle to $\boldsymbol{v}$ that allows an algorithm $\mathcal{A}$ to access the elements of $\boldsymbol{v}$ in a sequential streaming manner. That is, $\mathcal{A}$ can only read elements of $\boldsymbol{v}$ from left to right, and reset to the beginning of the stream at any time. We use $\mathcal{S}(\boldsymbol{v}).\mathsf{next}$ to denote the operation of returning the element of $\boldsymbol{v}$ at the current position and increments the pointer, and $\mathcal{S}(\boldsymbol{v}).\mathsf{reset}$ to denote the operation of resetting the pointer to the beginning of the stream.

Access to multilinear polynomials. We consider three types of access that an algorithm $\mathcal{A}$ can have to a multilinear polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$:

- Oracle access: $\mathcal{A}$ has *oracle access* to $p(\boldsymbol{X})$ if it can query the value of $p(\boldsymbol{x})$ at any $\boldsymbol{x} \in \mathbb{F}^n$ in constant time. $[\![p]\!]$ denotes an oracle to the polynomial $p(\boldsymbol{X})$.
- Random access: $\mathcal{A}$ has *random access* to $p(\boldsymbol{X})$ if it can query the value of $p(\boldsymbol{x})$ at any $\boldsymbol{x} \in \{0,1\}^n$ in constant time.
- Streaming access: $\mathcal{A}$ has *streaming access* to $p(\boldsymbol{X})$ if it has access to a streaming oracle $\mathcal{S}(\boldsymbol{p})$ to the evaluations of p over the hypercube $\{0,1\}^n$.

Clearly, streaming access is a restriction of random access, which is a restriction of oracle access.

The sumcheck protocol. The sumcheck protocol for n-variate polynomials [LFKN92] is an n-round interactive protocol that allows a prover to convince a verifier that the sum of the evaluations of an n-variate polynomial $p(\boldsymbol{X})$ over the hypercube $\{0,1\}^n$ is equal to some claimed value σ.

Definition 2.1 (Sumcheck language)**.** *Let $p_1(\boldsymbol{X}), p_2(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X})$ be n-variate multilinear polynomials over a finite field $\mathbb{F}$, and let $\sigma \in \mathbb{F}$. The* d-product-sumcheck language $\mathsf{SC}^{(\mathbb{F},n,d)}$ *consists of tuples of the form*

$(p_1(\boldsymbol{X}), p_2(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X}), \sigma)$ *where* $\sigma = \sum_{\boldsymbol{x}\in\{0,1\}^n} \prod_{j=1}^{d} p_j(\boldsymbol{x})$.

Definition 2.2 (Classical sumcheck protocol [LFKN92])**.** *Let* $p_1(\boldsymbol{X}), p_2(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$ *be* n*-variate multilinear polynomials, and let* $\sigma \in \mathbb{F}$ *be a claimed sum. The* classical sumcheck protocol *is an* n*-round interactive protocol for the* d*-product-sumcheck language between a prover* $\mathcal{P}$ *and a verifier* $\mathcal{V}$ *such that* $\mathcal{P}$ *has random/streaming access to the* d *polynomials,* $\mathcal{V}$ *has oracle access to the* d *polynomials, and both know* σ*. In each round* $i \in [n]$*,* $\mathcal{P}$ *sends to* $\mathcal{V}$ *a univariate "round" polynomial* $t_i(X)$ *of degree* d*, and* $\mathcal{V}$ *sends a random challenge* $r_i \xleftarrow{\$} \mathbb{F}$ *to* $\mathcal{P}$*.* $\mathcal{V}$ *checks that* $t_i(0) + t_i(1) = \sigma_i$*, where* $\sigma_1 = \sigma$*, and* $\sigma_i = t_{i-1}(r_{i-1})$*.*

If $\mathcal{P}$ *is honest, then* $t_i(X)$ *is defined as follows:*

$$t_i(X) := \sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} \prod_{j=1}^{d} p_j(r_1, \ldots, r_{i-1}, X, \boldsymbol{b}) \ .$$

Finally, $\mathcal{V}$ *accepts if and only if the checks in all* n *rounds pass and* $t_n(r_n) = \prod_{j=1}^{d} p_j(\boldsymbol{r})$*.*

Lemma 2.3 (Completeness and soundness of sumcheck [LFKN92])**.** *The classical sumcheck protocol satisfies the following properties:*

- *If* $(p_1(\boldsymbol{X}), p_2(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X}), \sigma) \in \mathsf{SC}^{(\mathbb{F},n,d)}$*, then* $\mathcal{V}$ *accepts with probability 1.*
- *If* $(p_1(\boldsymbol{X}), p_2(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X}), \sigma) \notin \mathsf{SC}^{(\mathbb{F},n,d)}$*, then* $\mathcal{V}$ *accepts with probability at most* $\frac{n\cdot d}{|\mathbb{F}|}$*.*

We will use "classical prover for $\mathsf{SC}^{(\mathbb{F},n,d)}$" to refer to an honest prover that follows the classical sumcheck protocol and sends the correct round polynomial $t_i(X)$ in each round $i \in [n]$.

Remark 2.4. The provers presented in Sections 2.2 and 2.3 will only require streaming access to their input polynomials, and we call such provers 'streaming provers'. The lower bounds in Sections 2.4 and 2.5, however, apply to the more general case of provers with random access to their inputs.

2.2 Space-efficient multilinear-sumcheck prover

We present our space-efficient multilinear-sumcheck prover $\mathcal{P}$ that only requires streaming access to the input polynomial. Given an input polynomial $p(X_1, \ldots, X_n) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$, $\mathcal{P}$ makes only 2 passes over $\mathcal{S}(\boldsymbol{p})$, and runs in $O(N)$ time while using $O(\sqrt{N})$ space.[2]

Existing ideas. For each $i \in [n]$, in the i-th round of the multilinear-sumcheck protocol, $\mathcal{P}$ must send the following polynomial to the verifier $\mathcal{V}$ (in order to satisfy completeness):

$$t_i(X) := \sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} p(r_1, \ldots, r_{i-1}, X, \boldsymbol{b}) = \sum_{\boldsymbol{x}\in\{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(r_1, \ldots, r_{i-1}) \cdot \left(\sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} p(\boldsymbol{x}, X, \boldsymbol{b}) \right) \ ,$$

where for each $j \in [i-1]$, $r_j \in \mathbb{F}$ is the random challenge chosen by $\mathcal{V}$ in the j-th round. It suffices for $\mathcal{P}$ to compute and send $t_i(0)$ and $t_i(1)$, since for all $i \in [n]$, $t_i(X)$ is a degree-1 polynomial in X and can thus be uniquely described by its evaluations at 0 and 1.

If $\mathcal{P}$ was allowed to make a pass over the input stream in every round, then it could compute every t_i in $O(N)$ time and $O(\log N)$ space [CMT12] since each value of $[\chi_{\boldsymbol{x}}(r_1, \ldots, r_{i-1})]_{\boldsymbol{x}\in\{0,1\}^{i-1}}$ can be computed

[2] In Appendix A, we show how to generalize this to perform k passes over $\mathcal{S}(\boldsymbol{p})$ and run in $O(kN)$ time, but only use $O(N^{1/k})$ space, for any $1 \leq k \leq \log N$.

in amortized constant time in a streaming manner using only $O(\log N)$ space (see Section 3.1 for details). Thus, $t_i(0)$ and $t_i(1)$ can be computed in $O(N)$ time using $O(\log N)$ space, yielding a streaming prover that runs in $O(N \log N)$ time and uses $O(\log N)$ space.

Using more space. We now describe a $\mathcal{P}$ that uses $O(\sqrt{N})$ working space, but only needs to make a pass over $\mathcal{S}(\boldsymbol{p})$ in round 1 and another in round $m := \lceil n/2 \rceil$.

Round 1. In the first round, $\mathcal{P}$ stores the following table M, which is indexed by $\boldsymbol{x} \in \{0,1\}^m$:

$$M[\boldsymbol{x}] := \sum_{\boldsymbol{b} \in \{0,1\}^{n-m}} p(\boldsymbol{x}, \boldsymbol{b}) \ .$$

Clearly, this table of size $O(2^m) = O(\sqrt{N})$ can be computed in $O(N)$ time with a single pass over $\mathcal{S}(\boldsymbol{p})$.

Using M, for any $i \in [m-1]$, $\mathcal{P}$ can compute the round polynomial $t_i(X)$ in $O(\sqrt{N})$ time without accessing $\mathcal{S}(\boldsymbol{p})$. This follows from the following expression:

$$\begin{aligned} t_i(y) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(r_1, \ldots, r_{i-1}, y, \boldsymbol{b}) &= \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(r_1, \ldots, r_{i-1}) \cdot \left(\sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(\boldsymbol{x}, y, \boldsymbol{b}) \right) \\ &= \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(r_1, \ldots, r_{i-1}) \cdot \left(\sum_{\boldsymbol{b} \in \{0,1\}^{n-m-i}} M[\boldsymbol{x} \parallel y \parallel \boldsymbol{b}] \right) \ , \end{aligned}$$

for any $i \in [m-1]$ and $y \in \{0,1\}$.

Round m. In round $m = \lceil n/2 \rceil$, once $\mathcal{P}$ knows the values of $r_1, \ldots, r_{m-1}$, it computes the following table M' indexed by $\boldsymbol{x} \in \{0,1\}^{n-m+1}$:

$$M'[\boldsymbol{x}] := p(r_1, \ldots, r_{m-1}, \boldsymbol{x}) = \sum_{\boldsymbol{b} \in \{0,1\}^{m-1}} \chi_{\boldsymbol{b}}(r_1, \ldots, r_{m-1}) \cdot p(\boldsymbol{b}, \boldsymbol{x}) \ .$$

This table of size $O(2^{n-m+1}) = O(\sqrt{N})$ can also be computed in $O(N)$ time with a single pass over $\mathcal{S}(\boldsymbol{p})$, since that the Lagrange polynomial evaluations can be computed in constant amortized time in a streaming manner (see Section 3.1 for details). Using M', for each $m \le i \le n$, $\mathcal{P}$ can compute the round polynomial $t_i(X)$ in time $O(\sqrt{N})$ with no further accesses to $\mathcal{S}(\boldsymbol{p})$. This step is akin to the "folding-style" linear-time, linear-space sumcheck of [CTY11; Tha13], but invoked on a multilinear polynomial of size $O(\sqrt{N})$, and follows from the following expression:

$$\begin{aligned} t_i(y) &= \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(r_1, \ldots, r_{m-1}, \ldots, r_{i-1}, y, \boldsymbol{b}) \\ &= \sum_{\boldsymbol{x} \in \{0,1\}^{i-m}} \chi_{\boldsymbol{x}}(r_m, \ldots, r_{i-1}) \cdot \left(\sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(r_1, \ldots, r_{m-1}, \boldsymbol{x}, y, \boldsymbol{b}) \right) \\ &= \sum_{\boldsymbol{x} \in \{0,1\}^{i-m}} \chi_{\boldsymbol{x}}(r_m, \ldots, r_{i-1}) \cdot \left(\sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} M'[\boldsymbol{x} \parallel y \parallel \boldsymbol{b}] \right) \ , \end{aligned}$$

for any $m \le i \le n$ and $y \in \{0,1\}$.

Putting it all together. Clearly, $\mathcal{P}$ only uses $O(\sqrt{N})$ space, and makes two passes over $\mathcal{S}(\boldsymbol{p})$ in addition to taking $O(\sqrt{N})$ time in each of the $\log N$ rounds, thus requiring $O(N)$ time in total.

2.3 Space-efficient product-sumcheck prover

We now present the space-efficient product-sumcheck prover from Theorem 1. We present the case for $k = 2$, i.e. an algorithm that runs in $O(N \log \log N)$ time while using $O(\sqrt{N})$ space. $\mathcal{P}$ must send the following degree-2 round polynomial $t_i(X)$ to $\mathcal{V}$ in each round $i \in [n]$ of the product-sumcheck protocol:

$$t_i(X) := \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(r_1, \ldots, r_{i-1}, X, \boldsymbol{b}) \cdot q(r_1, \ldots, r_{i-1}, X, \boldsymbol{b}) \ .$$

Each t_i is uniquely defined by 3 evaluation points. For clarity of notation, we will abuse notation and say $\mathcal{P}$ computes $t_i(X)$, when we actually mean that $\mathcal{P}$ computes $t_i(0)$, $t_i(1)$ and $t_i(z)$, for some fixed $z \in \mathbb{F} \setminus \{0, 1\}$.

Furthermore, since $g(z) = (1 - z) \cdot g(0) + z \cdot g(1)$ for any degree-1 polynomial $g(X)$, it actually suffices for $\mathcal{P}$ to compute $p_i(0)$ and $p_i(1)$, since it can obtain $p_i(z)$ from these. Hence, in the rest of this discussion, we will focus on computing $p_i(0), p_i(1)$, and $q_i(0), q_i(1)$.

We begin by discussing the "easy" case of handling the last $\lceil n/2 \rceil$ rounds, and then discuss the more challenging case of handling the first $\lfloor n/2 \rfloor$ rounds.

The last $\lceil n/2 \rceil$ rounds. Define $m := \lceil n/2 \rceil$. For rounds $i \geq m$, $\mathcal{P}$ can compute each round polynomial $t_i(X)$ in $O(\sqrt{N})$ time as follows: $\mathcal{P}$ makes a single pass over $\mathcal{S}(\boldsymbol{p})$ and $\mathcal{S}(\boldsymbol{q})$ in round m and stores the following two tables:

$$M'_p[\boldsymbol{x}] := p(r_1, \ldots, r_{m-1}, \boldsymbol{x}) \ \text{ and } \ M'_q[\boldsymbol{x}] := q(r_1, \ldots, r_{m-1}, \boldsymbol{x}), \quad \forall \boldsymbol{x} \in \{0,1\}^{n-m+1} \ .$$

Using these tables, $\mathcal{P}$ can compute $t_i(X)$ for any given $i \geq m$ in $O(\sqrt{N})$ time using similar ideas to those in Section 2.2. In more detail, $\mathcal{P}$ uses M'_p and M'_q to compute

$$[p(r_1, \ldots, r_{i-1}, y, \boldsymbol{b})]_{y \in \{0,1\}, \boldsymbol{b} \in \{0,1\}^{n-i}} \ \text{ and } \ [q(r_1, \ldots, r_{i-1}, y, \boldsymbol{b})]_{y \in \{0,1\}, \boldsymbol{b} \in \{0,1\}^{n-i}} \ .$$

This step clearly takes $O(\sqrt{N})$ time, and using the trick above, it can then compute

$$[p(r_1, \ldots, r_{i-1}, z, \boldsymbol{b})]_{\boldsymbol{b} \in \{0,1\}^{n-i}} \ \text{ and } \ [q(r_1, \ldots, r_{i-1}, z, \boldsymbol{b})]_{\boldsymbol{b} \in \{0,1\}^{n-i}} \ ,$$

and hence also $t_i(z)$, as required.

Bottleneck: computing cross products in the first m rounds. Computing $t_i(X)$ for rounds $i \in [m-1]$ is more challenging. Say $\mathcal{P}$ makes a pass over $\mathcal{S}(\boldsymbol{p})$ and $\mathcal{S}(\boldsymbol{q})$ in the first round, and wants to precompute some tables M_p and M_q that allow it to compute $t_i(X)$ for all rounds $i \in [m-1]$. In every round $i \in [m-1]$, given the challenges $\boldsymbol{r} = (r_1, \ldots, r_{i-1})$, it must be able to compute

$$\begin{aligned}
t_i(X) &= \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(\boldsymbol{r}, X, \boldsymbol{b}) \cdot q(\boldsymbol{r}, X, \boldsymbol{b}) \\
&= \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} \left(\sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r}) \cdot p(\boldsymbol{x}, X, \boldsymbol{b}) \right) \cdot \left(\sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r}) \cdot q(\boldsymbol{x}, X, \boldsymbol{b}) \right) \\
&= \sum_{\boldsymbol{x}, \boldsymbol{x}' \in \{0,1\}^{i-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r}) \cdot \chi_{\boldsymbol{x}'}(\boldsymbol{r}) \cdot \sum_{\boldsymbol{b} \in \{0,1\}^{n-i}} p(\boldsymbol{x}, X, \boldsymbol{b}) \cdot q(\boldsymbol{x}', X, \boldsymbol{b}) \ .
\end{aligned}$$

In round 1, $\mathcal{P}$ does not know the values of $r_1, \ldots, r_{m-1}$, and therefore needs to store values such that the above expression can be computed regardless of the challenges picked by $\mathcal{V}$. However, it is not clear how $\mathcal{P}$

can compute the following "cross products" efficiently for all $i \in [m-1]$:

$$\left[\sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} p(\boldsymbol{x},\boldsymbol{b})\cdot q(\boldsymbol{x}',\boldsymbol{b})\right]_{\boldsymbol{x},\boldsymbol{x}'\in\{0,1\}^{i-1}} . \tag{1}$$

Solution: reduce the number of rounds between passes. We take a different approach: instead of allowing $\mathcal{P}$ to make only a single pass before round m, we now allow it to make passes over $\mathcal{S}(\boldsymbol{p})$ and $\mathcal{S}(\boldsymbol{q})$ in multiple intermediate rounds $1 \le j < m$. Intuitively, this helps because when $\mathcal{P}$ no longer needs to precompute tables that suffice for all rounds $i \in [m-1]$, and instead only needs to store enough information for the next few rounds, until it makes its next pass. We show that it suffices for the prover to make $\log\log N = \log n$ passes.

In more detail, we have the prover perform a pass in every round $j \in [2^s]_{s=0}^{\log n-2}$, and we show that the tables stored in round $j = 2^s$ suffices to compute the round polynomial $t_i(X)$ for all rounds $i \in [2^s, 2^{s+1})$. To do this, we leverage the fact that when $\mathcal{P}$ makes a pass in some 'intermediate' round $j = 2^s$, it already knows knows the challenges $\boldsymbol{r}_L = (r_1, \ldots, r_{j-1})$. Thus, for a round $i \in [2^s, 2^{s+1}]$, given the 'new' challenges $\boldsymbol{r}_R = (r_j, r_{j_1}, \ldots, r_{i-1})$, $\mathcal{P}$ now only has to compute

$$t_i(X) = \sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} p(\boldsymbol{r}_L,\boldsymbol{r}_R,X,\boldsymbol{b})\cdot q(\boldsymbol{r}_L,\boldsymbol{r}_R,X,\boldsymbol{b}) \tag{2}$$

$$= \sum_{\boldsymbol{x},\boldsymbol{x}'\in\{0,1\}^{i-j}} \chi_{\boldsymbol{x}}(\boldsymbol{r}_R)\cdot\chi_{\boldsymbol{x}'}(\boldsymbol{r}_R)\cdot\left(\sum_{\boldsymbol{b}\in\{0,1\}^{n-i}} p(\boldsymbol{r}_L,\boldsymbol{x},X,\boldsymbol{b})\cdot q(\boldsymbol{r}_L,\boldsymbol{x}',X,\boldsymbol{b})\right) . \tag{3}$$

Accordingly, in each intermediate round $j = 2^s$, we have $\mathcal{P}$ compute the following table M_s consisting of 'cross products', which we index by $\boldsymbol{x}, \boldsymbol{x}' \in \{0,1\}^j$:

$$M_s[\boldsymbol{x},\boldsymbol{x}'] := \sum_{\boldsymbol{b}\in\{0,1\}^{n-2j+1}} p(\boldsymbol{r}_L,\boldsymbol{x},\boldsymbol{b})\cdot q(\boldsymbol{r}_L,\boldsymbol{x}',\boldsymbol{b}) .$$

This is sufficient due to the following reasons:

1. This table is of size $2^j \cdot 2^j = 2^{(2^s)} \cdot 2^{(2^s)}$, which is $O(\sqrt{N})$ for all $s \in [0, 1, \ldots, \log n - 2]$, so $\mathcal{P}$ has sufficient space to store it.
2. Given M_s, $\mathcal{P}$ can compute $t_i(X)$ for any $i \in [2^s, 2^{s+1})$ in $O(\sqrt{N})$ time. This is because each $t_i(X)$ is a linear combination of some at-most-$\sqrt{N}$-sized subset of M_s with respect to the coefficients in Eq. (3). We refer the reader to Section 4.1 for the details.
3. Each M_s can be computed in $O(N)$ time with a single pass over each of $\mathcal{S}(\boldsymbol{p})$ and $\mathcal{S}(\boldsymbol{q})$. This can be seen as follows: if $\mathcal{P}$ had access to $[p(\boldsymbol{r}_L,\boldsymbol{y})]_{\boldsymbol{y}\in\{0,1\}^{n-j+1}}$ and $[q(\boldsymbol{r}_L,\boldsymbol{y})]_{\boldsymbol{y}\in\{0,1\}^{n-j+1}}$, then it could compute each $M_s[\boldsymbol{x},\boldsymbol{x}']$ in $O(2^{n-2j+1})$ time, thus taking $O(2^{2j}\cdot 2^{n-2j+1}) = O(N)$ time to compute all 2^{2j} elements of M_s. We show in Section 4.1 that $\mathcal{P}$ can 'simulate' streaming access to $[p(\boldsymbol{r}_L,\boldsymbol{y})]_{\boldsymbol{y}\in\{0,1\}^{n-j+1}}$ and $[q(\boldsymbol{r}_L,\boldsymbol{y})]_{\boldsymbol{y}\in\{0,1\}^{n-j+1}}$, using its knowledge of $\boldsymbol{r}_L$ and streaming access to $\mathcal{S}(\boldsymbol{p})$ and $\mathcal{S}(\boldsymbol{q})$, with essentially no overhead.

We provide more details, and show how to extend these techniques polynomials that are the product of d multilinear polynomials, where d is a constant, in Section 4.

2.4 Time-space trade-off lower bound for multilinear-sumcheck provers

We now provide a brief overview of the time-space trade-off lower bound for the multilinear-sumcheck protocol, for an n-variate multilinear polynomial $p(\boldsymbol{X})$. We informally state the theorem below, and note that

when we say "non-adaptive prover", we mean a prover that is non-adaptive with respect to the polynomial it receives as input and the random challenges it receives from the verifier.

Theorem 2.5 (informal, see Theorem 5.2)**.** *Let $\mathcal{P}$ be a non-adaptive prover for the n-variate multilinear-sumcheck protocol that has random-access to its input $\boldsymbol{p} \in \mathbb{F}^N$, and which runs in time T and uses S units of random-access space. Then $T \log S = \Omega(N \log N)$.*

As before, for all $i \in [n]$, we require that $\mathcal{P}$ must compute and send the evaluations of the round polynomial $t_i(X)$ at 0 and 1.[3]

First round. Since $\mathcal{P}$ is complete, it must be compute the correct round polynomials for all input polynomials p. We start by showing that $\mathcal{P}$ must read every element of $\boldsymbol{p}$ to compute $t_1(X)$. Assume not, i.e. that there exists a point $\boldsymbol{x} \in \{0,1\}^n$ such that $\mathcal{P}$ does not read $p(\boldsymbol{x})$. Then there exists a different polynomial q whose evaluation at $\boldsymbol{x}$ differs from that of p. However, because $\mathcal{P}$ is non-adaptive, and because the locations it reads are the same for both p and q, it will produce the same round polynomial for both, which will be incorrect for at least one of the two.

Further rounds. An 'optimal' $\mathcal{P}$ would use these N reads to store a state st of size S, which it can use to compute $t_j(X)$ for further rounds $j > 1$. Say $\mathcal{P}$ makes no further accesses to $\boldsymbol{p}$ for the next few rounds after round 1. Then, in round $j = \log S + 1$, $\mathcal{P}$ must be able to use this state to compute $t_j(0) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}, 0, \boldsymbol{b})$ and $t_j(1) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}, 1, \boldsymbol{b})$, for all $\boldsymbol{r}$ in the restricted set of verifier challenges $\{0,1\}^{j-1} \subset \mathbb{F}^{j-1}$.[4] For every choice of $\boldsymbol{r}$ and $y \in \{0,1\}$, $t_j(y)$ corresponds to a sum of evaluations of $p(\boldsymbol{X})$ over distinct sub-cubes of $\{0,1\}^n$, each of which is an arbitrary element in $\mathbb{F}$ (independent of the other sums). There are 2^j such disjoint sub-cubes, and since $\mathcal{P}$ makes no further accesses to $\boldsymbol{p}$ and must be able to correctly compute all of these values, it must either:

- store a state of size at least $2^j = 2S$, which is a contradiction, or
- "compress" the $2S$ arbitrary field elements into fewer than S field elements, which can be shown to be impossible using a compression argument.

Thus, regardless of the st it stored, $\mathcal{P}$ must make further accesses to $\boldsymbol{p}$ in order to be able to send the correct polynomial in round $\log S + 1$. Since there are at least $2S - S = S$ sums whose (arbitrary) values $\mathcal{P}$ has not stored in st, and each of these is a sum over 2^{n-j} elements of $\boldsymbol{p}$, $\mathcal{P}$ must make at least $S \cdot 2^{n-j} \geq S \cdot 2^{n-\log S - 1} = N/2$ further accesses to $\boldsymbol{p}$ in order to compute the these sums correctly.

Other access strategies. The argument so far agrees with the construction in Section 2.2 with $S = \sqrt{N}$, where $\mathcal{P}$ makes a pass over $\mathcal{S}(\boldsymbol{p})$ in the first round, and then further accesses in round $m = \log S + 1 = \lceil n/2 \rceil$. In fact, the full proof shows that the prover cannot do better, and gains no advantage by making accesses spread over multiple rounds between rounds 1 and $\log S + 1$.

We complete the proof of Theorem 2.5 by repeating (an analog of) the above argument to show that in every round of the form $1, 1 + \log S, 1 + 2\log S, \ldots$, $\mathcal{P}$ must make $O(N)$ accesses to $\boldsymbol{p}$. There are $\lfloor \log N / \log S \rfloor$ such rounds, and therefore $\mathcal{P}$ must make $\Omega\left(N \log N / \log S\right)$ accesses to $\boldsymbol{p}$ in total. We provide more details in Section 5.

2.5 Time-space trade-off lower bound for product-sumcheck provers

The lower bound for multilinear-sumcheck provers also applies to product-sumcheck provers. In addition, we present a stronger lower bound for the product-sumcheck protocol which shows that the prover described in

[3] This is without loss of generality as $\mathcal{P}$ can compute $t_i(0)$ and $t_i(1)$ in constant time given any unique representation of t_i.

[4] Restricting the verifier to pick random challenges in $\{0,1\}^j$ as opposed to $\mathbb{F}^j$ makes the $\mathcal{P}$'s job easier, but as we show, it suffices for obtaining the lower bound.

Section 2.3 is optimal within a class of "natural" classical sumcheck provers for $\mathsf{SC}^{(\mathbb{F},n,2)}$.

Need for conjecture. If there exists a fast matrix multiplication algorithm for $A, B \in \mathbb{F}^{m\times m}$ that outputs $A \cdot B$ while only using $O(m^2)$ field operations, then the prover in Section 2.3 can be improved to require only $O(kN)$ field operations when using $O(N^{1/k})$ space, thus matching the multilinear-sumcheck prover in Section 2.2. At a high level, this would remove the bottleneck in the algorithm in Section 2.3 for computing the cross products, since any table M_s that will fit in memory can be computed in linear time, regardless of the value of s (see Section 4.4 for details). Consequently, a stronger lower bound cannot be unconditional and requires us to introduce the following conjecture that is related to matrix multiplication.

Definition 2.6 (Batch polynomial multiplication problem)**.** *$\mathcal{A}$ solves the* batch polynomial multiplication *problem with parameters n, ℓ (denoted by $\mathsf{BPM}(n,\ell)$) if for all $p_1, \ldots, p_\ell, q_1, \ldots, q_\ell \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$, $\mathcal{A}$ outputs $f(\boldsymbol{X}) := \sum_{i=1}^{\ell} p_i(\boldsymbol{X}) \cdot q_i(\boldsymbol{X})$ assuming that $\mathcal{A}$ has random-access to the evaluations of the inputs polynomials over $\{0,1\}^n$.*

Conjecture 2.7. *There exists a constant $\omega > 0$ such that any algorithm $\mathcal{A}$ that solves $\mathsf{BPM}(n,\ell)$ requires $\Omega(2^{(\omega+1)n} \cdot \ell)$ field operations for all $n, \ell \in \mathbb{N}$.*

The output of $\mathsf{BPM}(n,\ell)$ consists of 3^n field elements, and therefore the conjecture is trivially true if $\ell = O((3/2-\kappa)^n)$ for any constant $\kappa > 0$. However, the instances of interest for our proof are when $\ell = \Omega(2^n)$, where it is not evident if the conjecture holds. We note that the conjecture is false if $\ell = \Omega(2^n)$ and there exists a fast matrix multiplication algorithm, but as mentioned earlier, if there exists such an algorithm then our product-sumcheck prover can be improved to break this lower bound. See Section 4.4 for details.

Theorem 2.8 (informal, see Theorem 6.5)**.** *If Conjecture 2.7 holds, then an honest, non-adaptive, field-agnostic[5] prover $\mathcal{P}$ for the product-sumcheck protocol that has $O(N^{1/2-\varepsilon})$ space and random-access to p and q requires $\Omega(N(\log\log N + \log\varepsilon)/\log(1/\omega))$ time, where $\varepsilon > 0$ is a constant, assuming that $\mathcal{P}$ reads $O(N^{1-\varepsilon})$ values of $\boldsymbol{p}$ and $\boldsymbol{q}$ in any round where it performs $\leq N/4$ field operations.*

Remark 2.9. The last assumption regarding the number of reads may seem contrived, but all known product-sumcheck provers satisfy it [CTY11; CMT12; Tha13], including our own algorithm from Section 2.3. This assumption can be removed if we assume a slightly stronger conjecture, see the full statement (Theorem 6.5) for details.

Proof intuition. For simplicity, we assume that in any round where $\mathcal{P}$ accesses its input, it must access all N evaluations of p and q. This assumption is removed in the full proof in Section 6, but it is not crucial to the intuition we present here. The proof is structured in a similar manner to the multilinear-sumcheck lower bound, where we pick two rounds of the sumcheck protocol, $\mathsf{m}_{\mathcal{A}}$ and $\mathsf{m}_{\mathcal{B}}$ that are far apart, and we show that a time-optimal prover which made a read the input in round $\mathsf{m}_{\mathcal{A}}$, must read the input in or before round $\mathsf{m}_{\mathcal{B}}$ as well. Proving this statement for $O(\log\log N + \log\varepsilon)$ pairs of rounds $\mathsf{m}_{\mathcal{A}}$ and $\mathsf{m}_{\mathcal{B}}$ is sufficient to complete the proof.

We are motivated by the bottleneck in the product-sumcheck prover from Section 2.3 for computing the cross products. Recall that if $\mathcal{P}$ reads the entire input in a round $\mathsf{m}_{\mathcal{A}}$, then it can compute a state st in linear time that allows it to compute the round polynomial $t_i(X)$ for every round $i \in [\mathsf{m}_{\mathcal{A}}, 2\mathsf{m}_{\mathcal{A}}]$, without reading the input again. The restriction of $i \leq 2\mathsf{m}_{\mathcal{A}}$ ensures that the table computation requires linear time. Therefore, if we consider a round $\mathsf{m}_{\mathcal{B}}$ such that $\mathsf{m}_{\mathcal{B}} \geq c \cdot \mathsf{m}_{\mathcal{A}}$ for some constant $c > 2$, then there are two possible cases:

[5]The specific technical condition we require is only being *extension-agnostic*, which is weaker than being field-agnostic.

- $\mathcal{P}$ reads the entire input in or before round $\mathsf{m}_{\mathcal{B}}$ to compute $t_{\mathsf{m}_{\mathcal{B}}}(X)$, or
- $\mathcal{P}$ uses st alone to compute $t_{\mathsf{m}_{\mathcal{B}}}(X)$, without reading the input again after round $\mathsf{m}_{\mathcal{A}}$.

We show that if c is large enough and we are in the second case, $\mathcal{P}$ must have performed $\Omega(N \log^{\omega} N)$ field operations in round $\mathsf{m}_{\mathcal{A}}$ to compute st, where ω is the constant from Conjecture 2.7. We refer to this statement as the "main technical lemma", and we provide a proof sketch of it below. This lemma is the crux of the proof, and implies that if we are not in the first case above, we obtain the desired lower bound.

If we are in the first case, then we have shown that $\mathcal{P}$ reads the entire input again in or before round $\mathsf{m}_{\mathcal{B}}$, despite having read the input in round $\mathsf{m}_{\mathcal{A}}$. The main technical lemma only holds if $\mathsf{m}_{\mathcal{B}}$ is at most $\varepsilon \log N$, but there must still be $\Omega(\log_c(\varepsilon \log N)) = \Omega(\log \log N + \log \varepsilon)$ pairs of such rounds which adhere to this constraint, completing the proof.

Main technical lemma. In round $\mathsf{m}_{\mathcal{B}}$, $\mathcal{P}$ outputs $t_{\mathsf{m}_{\mathcal{B}}}(X)$ using the following two components: (i) st, the state computed by $\mathcal{P}$ in round $\mathsf{m}_{\mathcal{A}}$, and (ii) $\boldsymbol{r}$, the random challenges received by $\mathcal{P}$ between rounds $\mathsf{m}_{\mathcal{A}}$ and $\mathsf{m}_{\mathcal{B}}$. Define $\mathsf{m} := \mathsf{m}_{\mathcal{B}} - \mathsf{m}_{\mathcal{A}} + 1$. Note that because $\mathsf{m}_{\mathcal{B}} \leq \varepsilon \log N$, $\mathsf{m} < \log N/2$. The goal of this lemma is to show that if $\mathcal{P}$ can compute $t_{\mathsf{m}_{\mathcal{B}}}(X)$ efficiently using these two components, then we can use $\mathcal{P}$ to construct an algorithm $\mathcal{A}^*$ that solves an instance of $\mathsf{BPM}(\mathsf{m}, 2^{n-\mathsf{m}_{\mathcal{B}}})$ in time $O(T + N)$, where T is the total time taken by $\mathcal{P}$ to compute st. Because $2^{n-\mathsf{m}_{\mathcal{B}}} = \Omega(\sqrt{N})$, and because $2^{\mathsf{m}} = O(\sqrt{N})$, we can apply Conjecture 2.7 to $\mathcal{A}^*$ to obtain $T = \Omega(N \log^{\omega} N)$, where ω is the constant from the conjecture.

Before describing the construction of $\mathcal{A}^*$, we first note that computing $t_{\mathsf{m}_{\mathcal{B}}}(X)$ for 3^{m} different values of $\boldsymbol{r}$ is equivalent to computing the following polynomial $f : \mathbb{F}^{\mathsf{m}} \to \mathbb{F}$:

$$f(\boldsymbol{X}) := \sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p(\boldsymbol{t}, \boldsymbol{X}, \boldsymbol{b}) \cdot q(\boldsymbol{t}, \boldsymbol{X}, \boldsymbol{b}) \ , \tag{4}$$

where $\boldsymbol{t} \in \mathbb{F}^{\mathsf{m}_{\mathcal{A}}-1}$ is the vector of random challenges $\boldsymbol{r}$ received by $\mathcal{P}$ before round $\mathsf{m}_{\mathcal{A}}$. Clearly, computing this polynomial is equivalent to solving $\mathsf{BPM}(\mathsf{m}, 2^{n-\mathsf{m}_{\mathcal{B}}})$. This motivates the construction of $\mathcal{A}^*$ as follows: using the fact that $\mathcal{P}$ is field-agnostic and non-adaptive, we can model $\mathcal{P}$ as an arithmetic circuit $\mathcal{C}$ such that each gate of $\mathcal{C}$ outputs a *bilinear* polynomial in the inputs of the circuit: $\boldsymbol{p}$ and $\boldsymbol{q}$. Therefore, the output of $\mathcal{C}$ can be expressed as follows:

$$f(\boldsymbol{r}) = \sum_{i,j=0}^{|\mathsf{st}^*|} c_{i,j}(\boldsymbol{r}) \cdot \mathsf{st}^*_i \cdot \mathsf{st}^*_j \ , \tag{5}$$

If $|\mathsf{st}^*| = O(N^{1/2-\varepsilon})$, and $3^{\mathsf{m}} = O(N^{2\varepsilon})$ (which holds because we restricted $\mathsf{m}_{\mathcal{B}} \leq \varepsilon \log N$), then the number of coefficients $c_{i,j}(\cdot)$ is $O(N)$, and therefore all $f(\boldsymbol{r})$ can be computed in $O(N)$ time by solving the system of linear equations defined by Eq. (5).

Therefore, using st alone, $\mathcal{A}^*$ can compute $f(\boldsymbol{r})$ in $O(N)$ time for 3^{m} values of $\boldsymbol{r}$, which uniquely defines the polynomial $f(\boldsymbol{X})$, and therefore $\mathcal{A}^*$ can solve $\mathsf{BPM}(\mathsf{m}, 2^{n-\mathsf{m}_{\mathcal{B}}})$ in $O(T + N)$ time. Proving the fact that $\mathcal{P}$'s output must satisfy Eq. (5) requires a lot of care, and we refer the reader to Section 6 for the details.

2.6 New product-sumcheck PIOP for FFT-friendly fields

We provide a brief overview of a proof of Theorem 3 and Theorem 4. For Theorem 3, we describe the protocol informally for the case when $k = 1$, and refer the reader to Section 7 for the full details.

Protocol overview. Let $s = \log^*(n/2)$, and let $\mathbb{G}_1, \mathbb{G}_2, \ldots, \mathbb{G}_{s+1}$ be multiplicative subgroups of $\mathbb{F}$ such that

- For $j \in [s-1]$: $|\mathbb{G}_j| = T(j)$, where $T(j)$ is the j-th tower power of 2 ($T(1) = 2$, $T(2) = 2^2 = 4$, $T(3) = 2^4 = 16$, $T(4) = 2^{16}$ and so on).

- For $j = s$: $|\mathbb{G}_s| = \sqrt{N}/\left(\prod_{j=1}^{s-1} |\mathbb{G}_j|\right)$.
- For $j = s+1$: $|\mathbb{G}_{s+1}| = \sqrt{N}$.

These subgroups exist if $\mathbb{F}$ is FFT-friendly, i.e., $|\mathbb{F}| - 1$ is product of several small primes (in this case, it divides a large power of 2). We define the *mixed polynomial* $\tilde{p} : \mathbb{F}^{s+1}$ as the unique polynomial such that for all $(g_1, \ldots, g_{s+1}) \in (\mathbb{G}_1 \times \ldots \times \mathbb{G}_{s+1})$, we have

$$\tilde{p}(g_1, \ldots, g_{s+1}) = p(\phi_1(g_1), \ldots, \phi_{s+1}(g_{s+1})) \ ,$$

where $\phi_j(g)$ is the binary representation of i such that $g = g_j^i$ where g_j is a generator of $\mathbb{G}_j$, and $\tilde{p}$ has degree at most $|\mathbb{G}_j| - 1$ in the j-th variable. $\tilde{q}$ is defined analogously. Instead of the verifier $\mathcal{V}$ having oracle access to the multilinear polynomials $p, q : \mathbb{F}^n \to \mathbb{F}$, we assume that $\mathcal{V}$ has oracle access to the mixed polynomials $\tilde{p}, \tilde{q} : \mathbb{F}^{s+1} \to \mathbb{F}$.

In the classical sumcheck protocol, $\mathcal{P}$ sends a degree-2 polynomial t_j to $\mathcal{V}$ in round j, and $\mathcal{V}$ send back a random challenge $r_j \in \mathbb{F}$. Instead, in the j-th round of our protocol, $\mathcal{P}$ sends a *polynomial oracle* to a $(2|\mathbb{G}_j| - 2)$-degree univariate polynomial t_j to $\mathcal{V}$, and $\mathcal{V}$ sends back a challenge $r_j \xleftarrow{\$} \mathbb{F}$. This round polynomial t_j is defined as follows:

$$t_j(X) = \sum_{\substack{g_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ g_{s+1} \in \mathbb{G}_{s+1}}} \tilde{p}(r_1, \ldots, r_{j-1}, X, g_{j+1}, \ldots, g_{s+1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, X, g_{j+1}, \ldots, g_{s+1})$$

$\mathcal{V}$ then performs the following check:

$$\sum_{g \in \mathbb{G}_j} t_j(g) = t_{j-1}(r_{j-1}) \ . \tag{6}$$

If the check passes, $\mathcal{V}$ continues to the next round, and otherwise it rejects. $\mathcal{V}$ has polynomial oracle access to both t_j and t_{j-1}, and therefore can obtain $t_{j-1}(r_{j-1})$ in one oracle query. The check in Eq. (6) is therefore simply a *univariate sumcheck* over the subgroup $\mathbb{G}_j$. The univariate sumcheck protocol of Aurora [BCRSVW19] can be used to implement this check, and requires $O(|\mathbb{G}_j| \cdot \log |\mathbb{G}_j|)$ field operations for the prover with only 1 round of communication.

$\mathcal{P}$'s time bottleneck is computing the round polynomials t_i for all $i \in [s]$. This involves multiplying $N/(|\mathbb{G}_1| \cdots |\mathbb{G}_{i-1}|)$ many pairs of $(|\mathbb{G}_i|-1)$-degree univariate polynomials, which requires $O(|\mathbb{G}_i| \cdot \log |\mathbb{G}_i| \cdot N/(|\mathbb{G}_1| \cdots |\mathbb{G}_i|))$ field operations using FFTs. By our construction, $\log |\mathbb{G}_i| \leq |\mathbb{G}_{i-1}|$, and therefore the above complexity is simply $O(N)$. Since $s = O(\log^* N)$, we obtain the desired time and round complexity.

$\mathcal{P}$'s space bottleneck is simply storing the round polynomials t_j. The degree of t_j is $2|\mathbb{G}_j| - 2$, and therefore $\mathcal{P}$ requires $O(|\mathbb{G}_j|)$ space to it. The largest value of $|\mathbb{G}_j|$ for any $j \in [s+1]$ is $\sqrt{N}$, and therefore $\mathcal{P}$ requires $O(\sqrt{N})$ space.

We emphasize that unlike the algorithm described in Section 2.3, this is not an interactive proof for the sumcheck language. Rather, it is a PIOP similar to Sparrow's protocol [PP24]. However, the polynomial oracles for t_j can be realized via a univariate polynomial commitment scheme [KZG10; BBHR18; ACFY24] to obtain an *interactive argument* for the sumcheck language. This does not affect the time complexity of the prover since each univariate polynomial has degree at most $\sqrt{N}$, and these commitment schemes require $O(d \log d)$ prover time to commit and open a polynomial of degree d.

Linear-time prover. If one is willing to use a linear-space prover, then a similar round complexity can be achieved with a linear-time prover by adapting the folding-style sumcheck of [CTY11; Tha13] to the mixed

polynomials. In particular, the algorithm described above requires $O(N)$ in each round to simply compute the round polynomial t_j. However, if we allow $\mathcal{P}$ to store the following *folded polynomials* $\tilde{p}_j, \tilde{q}_j$ in round j where

$$\forall \boldsymbol{g} \in \mathbb{G}_{j+1} \times \ldots \times \mathbb{G}_{s+1} : \tilde{p}_j(\boldsymbol{g}) = \sum_{g_j \in \mathbb{G}_j} \tilde{p}_{j-1}(g_j, \boldsymbol{g}) \cdot \mathsf{L}_{j,g_j}(r_j) \ ,$$

where $\mathsf{L}_{j,g_j}(r_j)$ is the univariate Lagrange polynomial evaluated at r_j for subgroup $\mathbb{G}_j$ that evaluates to 1 at g_j and 0 at all other points in $\mathbb{G}_j$. Using $\tilde{p}_{j-1}$ and $\tilde{q}_j$ as opposed to $\tilde{p}$ and $\tilde{q}$ to compute t_j allows $\mathcal{P}$ to do so in $O\Big(\prod_{i=j-1}^{s+1} |\mathbb{G}_i|\Big)$ time, as opposed to $O(N)$ time. Adding the cost over all rounds, the total number of field operations required by $\mathcal{P}$ is $O(N)$. We refer the reader to Section 4.4 for the full details of this linear-time prover.

3 Preliminaries

3.1 Computation of Lagrange basis polynomial in a streaming manner

Given $\boldsymbol{r} \in \mathbb{F}^n$, we can compute $\{\chi_{\langle 0\rangle}(\boldsymbol{r}), \chi_{\langle 1\rangle}(\boldsymbol{r}), \ldots, \chi_{\langle 2^n-1\rangle}(\boldsymbol{r})\}$ in a streaming manner as follows. We first compute the Lagrange basis polynomial $\chi_{\langle 0\rangle}(\boldsymbol{r})$, which is equal to $\prod_{i=1}^{n}(1-r_i)$. For each $v \in [2^n-1]$, we compute $\chi_{\langle v\rangle}(\boldsymbol{r})$ using $\chi_{\langle v-1\rangle}(\boldsymbol{r})$ as follows:

$$\chi_{\langle v\rangle}(\boldsymbol{r}) = \chi_{\langle v-1\rangle}(\boldsymbol{r}) \cdot \frac{r_j}{1-r_j} \cdot \prod_{i=1}^{j-1} \frac{1-r_i}{r_i} \quad ,$$

where j is the index of the least significant bit of $\langle v\rangle$ that is equal to 0. This computation clearly requires only $O(n)$ space. Furthermore, when computing $\chi_{\langle 0\rangle}(\boldsymbol{r}), \chi_{\langle 1\rangle}(\boldsymbol{r}), \ldots, \chi_{\langle 2^n-1\rangle}(\boldsymbol{r})$, the above algorithm multiplies by $(1-r_i)/r_i$ or $r_i/(1-r_i)$ only 2^i times. Therefore, the total number of multiplications performed is $\sum_{i=1}^{n} 2^i = O(2^n)$, and the amortized cost of computing one Lagrange basis polynomial is $O(1)$. This algorithm is used in previous work [VSBW13; BMMNW24], and follows a similar approach to an unpublished observation by Vu. Furthermore, a note by Rothblum [Rot24] demonstrates how to compute these evaluations in the *gray code ordering* in a streaming manner with only $O(n)$ space.

3.2 Model of computation

Throughout this paper, we assume the arithmetic model of computation for a given finite field $\mathbb{F}$. That is, addition and multiplication of field elements takes constant time, and one field element can be stored in a single unit of space. Furthermore, specifically for the purposes of our lower bounds in Sections 5 and 6, we model algorithms as *arithmetic circuits*.

Definition 3.1 (Arithmetic circuit)**.** *An* arithmetic circuit $\mathcal{C}$ *is a directed acyclic graph* G *such that*
- *All vertices are labelled as constant gates, input gates, addition gates, or multiplication gates.*
- *The in-degree of input gates and constant gates is 0, and the in-degree of addition gates and multiplication gates is 2.*
- *Additionally, some vertices are labeled as output gates.*

$\mathcal{C}$ *is* equipped *with field* $\mathbb{F}$ *if the constant gates are labelled with elements of* $\mathbb{F}$*, and the addition and multiplication gates output the sum and product of their input wires in* $\mathbb{F}$ *respectively. A* $\mathcal{C}$ *equipped with* $\mathbb{F}$ computes functions $f_1, f_2, \ldots, f_k : \mathbb{F}^n \to \mathbb{F}$ *if the* i*-th output gate of* $\mathcal{C}$ *outputs* $f_i(\boldsymbol{x})$ *for all* $\boldsymbol{x} \in \mathbb{F}^n$ *where* $\boldsymbol{x}$ *is the vector of field elements corresponding to the input gates of* $\mathcal{C}$*.*

Classical prover as set of arithmetic circuits. The classical prover for $\mathsf{SC}^{(\mathbb{F},n,d)}$ can be modelled as n-many arithmetic circuits $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_n$, one for each round of the protocol. Since we prove our lower bounds in the random-access model, each $\mathcal{C}_i$ receives as input $\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_d \in \mathbb{F}^N$ as $(d \cdot N)$-many input gates. We refer to this set of values as $\mathcal{J} \in \mathbb{F}^{d \cdot N}$. $\mathcal{C}_1$ receives as input $\mathcal{J}$ and outputs $\mathcal{O}_1$, which consists of the *state* that is passed to $\mathcal{C}_2$. Subsequent circuits $\mathcal{C}_i$ receive as input $\mathcal{J}$, $\mathcal{O}_{i-1}$ and random challenges $(r_1, \ldots, r_{i-1})$, and output $\mathcal{O}_i$.

Non-adaptive algorithms. Arithmetic circuits capture the *non-adaptivity* assumption we make about the prover for our lower bounds. That is, the field operations performed by the prover do not depend on the input values $\mathcal{J}$ and the random challenges $(r_1, \ldots, r_{i-1})$. In particular, the total number of field operations performed by $\mathcal{C}_i$ is the same regardless of the input. To the best of our knowledge, all known prover algorithms

for $\mathsf{SC}^{(\mathbb{F},n,d)}$ are non-adaptive [CTY11; CMT12; Tha13], including ours. Section 5.1 discusses non-adaptivity in more detail.

Time and space complexity. This model implies natural lower bounds on the time and space complexity of the prover:

- The time complexity of the prover is at least the total number of wires leaving the input gates $\mathcal{J}$ in $\mathcal{C}_1, \ldots, \mathcal{C}_n$. In other words, the total number of reads made by the prover across n rounds.
- The space complexity of the prover is at least $\max_{i\in[n]}\{|\mathcal{O}_i|\}$.

When discussing the algorithms in Sections 4 and 7 and Appendix A, we will argue the time and space complexity in terms of number of field operations performed and number of field elements stored by the prover respectively.

Definition 3.2 (S-space bounded classical prover for $\mathsf{SC}^{(\mathbb{F},n,d)}$)**.** *A classical prover for $\mathsf{SC}^{(\mathbb{F},n,d)}$ is S-space bounded if $\max_{i\in[n]}\{|\mathcal{O}_i|\} \le S$, where $\mathcal{O}_i$ is the output of the i-th round of the protocol. That is, the prover can pass on a state of size at most S to the next round.*

3.3 Compression lemma

Lemma 3.3 ([DTT10], Fact 8.1)**.** *Suppose these is a randomized encoding procedure $\mathsf{Enc} : \{0,1\}^N \times \{0,1\}^r \to \{0,1\}^M$ and a decoding procedure $\mathsf{Dec} : \{0,1\}^M \times \{0,1\}^r \to \{0,1\}^N$ such that*

$$\Pr_{\boldsymbol{r}\xleftarrow{\$}\{0,1\}^r}[\mathsf{Dec}(\mathsf{Enc}(\boldsymbol{v},\boldsymbol{r}),\boldsymbol{r}) = \boldsymbol{v}] \ge \delta \ ,$$

for all $\boldsymbol{v} \in \{0,1\}^N$. Then $M \ge N - \log 1/\delta$.

On a high level, this lemma is useful in our lower bound because our proof partitions the prover of the sumcheck protocol into multiple parties. That is, the prover in round i is a party that needs to encode the information that it reads (and therefore invokes the Enc function from the above lemma), and the prover in round $j > i$ is a party that needs to decode the information that it receives (and therefore invokes the Dec function from the above lemma). Since we work in the arithmetic model of computation, we use the following corollary of the above lemma.

Corollary 3.4. *Suppose these is a randomized encoding procedure $\mathsf{Enc} : \mathbb{F}^N \times \{0,1\}^r \to \mathbb{F}^M$ and a decoding procedure $\mathsf{Dec} : \mathbb{F}^M \times \{0,1\}^r \to \mathbb{F}^N$ such that*

$$\Pr_{\boldsymbol{r}\xleftarrow{\$}\{0,1\}^r}[\mathsf{Dec}(\mathsf{Enc}(\boldsymbol{v},\boldsymbol{r}),\boldsymbol{r}) = \boldsymbol{v}] \ge \delta \ ,$$

for all $\boldsymbol{v} \in \mathbb{F}^N$. Then $M \ge N - \log_{2|\mathbb{F}|} 1/\delta$.

Proof. Let $t = \lceil \log |\mathbb{F}| \rceil$, and let $\mathsf{Enc} : \{0,1\}^{tN} \times \{0,1\}^r \to \{0,1\}^{tM}$ be the encoding procedure that encodes each field element in $\mathbb{F}$ as t bits. Similarly consider the decoding procedure $\mathsf{Dec} : \{0,1\}^{tM} \times \{0,1\}^r \to \{0,1\}^{tN}$. By Lemma 3.3, we have $tM \ge tN - \log 1/\delta$, and therefore $M \ge N - (\log 1/\delta)/t \ge (\log 1/\delta)/(\log 2|\mathbb{F}|)$ as desired. □

4 Space-efficient prover for the classical sumcheck protocol

We present a space-efficient prover for the classical sumcheck protocol for products of two multilinear polynomials, and prove the following tradeoff between time and space for the prover.

Theorem 4.1. *For any $k > 0$, there exists an honest prover $\mathcal{P}$ for $\mathsf{SC}^{(\mathbb{F},n,2)}$ which has streaming access to the evaluations of two multilinear polynomials, p, q, and requires $O(N(\log\log N + k))$ field operations and $O\left(N^{1/k}\right)$ space.*

Substituting $k = \log N/\log\log N$ in the above theorem, we get the following corollary.

Corollary 4.2. *There exists an honest prover $\mathcal{P}$ for $\mathsf{SC}^{(\mathbb{F},n,2)}$ which has streaming access to the evaluations of two multilinear polynomials, p, q, and requires $O(N\log N/\log\log N)$ field operations and $O(\log N)$ space.*

4.1 Overview

In every round $j \in [n]$ of the protocol, $\mathcal{P}$ must correctly send the following values, for $v \in E := \{0, 1, z\}$, to $\mathcal{V}$:

$$f_j(v) = \sum_{\boldsymbol{b}\in\{0,1\}^{n-j}} p(\boldsymbol{r}_{:j}, v, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j}, v, \boldsymbol{b}) \ ,$$

where $\boldsymbol{r}_{:j} \in \mathbb{F}^{j-1}$ is the vector of random challenges received from $\mathcal{V}$ so far.

Fixing an arbitrary $k > 0$, the key idea of our protocol is as follows: $\mathcal{P}$ performs a single pass of the input streams in some special rounds j', and stores a "state" of size $O\left(N^{1/k}\right)$. Using this state, $\mathcal{P}$ can efficiently compute f_j for all rounds $j \in [j', j' + w(j') - 1]$ for some $w(j') \geq 1$, *without any additional passes* over the input streams. Ideally, we would like $w(j')$ to be as large as possible, but we will see that our time and space restrictions prevent us from doing so.

Choosing a state. Let $M_{j'}$ be the state stored by $\mathcal{P}$ after making the pass in round j'. We discuss what $M_{j'}$ needs to be in order to correctly compute the replies for the round $j = j' + w(j') - 1$.[6] Let the new challenges received in rounds j' to j be $\boldsymbol{r}' \in \mathbb{F}^{w(j')-1}$, such that $\boldsymbol{r}_{:j} = (\boldsymbol{r}_{:j'}, \boldsymbol{r}')$. For each $v \in E$, the prover $\mathcal{P}$ needs to compute the following

$$\begin{aligned}
f_j(v) &= \sum_{\boldsymbol{b}\in\{0,1\}^{n-j}} p(\boldsymbol{r}_{:j}, v, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j}, v, \boldsymbol{b}) \\
&= \sum_{\boldsymbol{b}\in\{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{r}', v, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{r}', v, \boldsymbol{b}) \\
&= \sum_{\boldsymbol{b}\in\{0,1\}^{n-j}} \left(\sum_{\boldsymbol{b}_1\in\{0,1\}^{w(j')-1}} \chi_{\boldsymbol{b}_1}\left(\boldsymbol{r}'\right) p(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, v, \boldsymbol{b}) \right) \cdot \left(\sum_{\boldsymbol{b}_2\in\{0,1\}^{w(j')-1}} \chi_{\boldsymbol{b}_2}\left(\boldsymbol{r}'\right) q(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, v, \boldsymbol{b}) \right) \\
&= \sum_{\boldsymbol{b}_1,\boldsymbol{b}_2\in\{0,1\}^{w(j')-1}} \chi_{\boldsymbol{b}_1}\left(\boldsymbol{r}'\right) \chi_{\boldsymbol{b}_2}\left(\boldsymbol{r}'\right) \left(\sum_{\boldsymbol{b}\in\{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, v, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, v, \boldsymbol{b}) \right) \\
&:= \sum_{\boldsymbol{b}_1,\boldsymbol{b}_2\in\{0,1\}^{w(j')-1}} \chi_{\boldsymbol{b}_1}\left(\boldsymbol{r}'\right) \chi_{\boldsymbol{b}_2}\left(\boldsymbol{r}'\right) \cdot g(\boldsymbol{b}_1, \boldsymbol{b}_2, v)
\end{aligned}$$

[6] Computing the replies for rounds j' such that $j \leq j' < j + t_j - 1$ follows easily from this analysis.

It suffices for $\mathcal{P}$ to compute $g(\boldsymbol{b}_1, \boldsymbol{b}_2, v)$ for all $\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{w(j')-1}$ and $v \in E$ in order to correctly compute $f_j(v)$. In order to do this, $\mathcal{P}$ in round j' computes and stores as its "state" the matrix $M_{j'} \in \mathbb{F}^{2^{w(j')}} \times \mathbb{F}^{2^{w(j')}}$ such that for each $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')}$:

$$M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \coloneqq \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$$

Using this state, for all $\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{w(j')-1}$, $\mathcal{P}$ can compute $g(\boldsymbol{b}_1, \boldsymbol{b}_2, v)$ for all $v \in E$.

- If $v \in \{0,1\}$, then

$$g(\boldsymbol{b}_1, \boldsymbol{b}_2, v) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, v, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, v, \boldsymbol{b}) = M_{j'}[(\boldsymbol{b}_1, v), (\boldsymbol{b}_2, v)]$$

- If $v = z$, then

$$\begin{aligned} g(\boldsymbol{b}_1, \boldsymbol{b}_2, z) &= \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p\big(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, z, \boldsymbol{b}\big) \cdot q\big(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, z, \boldsymbol{b}\big) \\ &= \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} \big((1-z) \cdot p(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, 0, \boldsymbol{b}) + z \cdot p(\boldsymbol{r}_{:j'}, \boldsymbol{b}_1, 1, \boldsymbol{b})\big) \cdot \\ &\qquad \big((1-z) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, 0, \boldsymbol{b}) + z \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{b}_2, 1, \boldsymbol{b})\big) \\ &= (1-z)^2 \cdot M_{j'}[(\boldsymbol{b}_1, 0), (\boldsymbol{b}_2, 0)] + z(1-z) \cdot M_{j'}[(\boldsymbol{b}_1, 0), (\boldsymbol{b}_2, 1)] + \\ &\qquad z(1-z) \cdot M_{j'}[(\boldsymbol{b}_1, 1), (\boldsymbol{b}_2, 0)] + z^2 \cdot M_{j'}[(\boldsymbol{b}_1, 1), (\boldsymbol{b}_2, 1)] \end{aligned}$$

Clearly, $|M_{j'}| = 2^{2w(j')} = O(N^{1/k})$, which enforces our first restriction on $w(j')$. That is, $w(j') = O(\log N/k)$, which is identical to the restriction in the multilinear case (see Appendix A). However, another factor to consider while deciding the value of $w(j')$ is the ability to compute $M_{j'}$ efficiently.

Computing the state. The elements of $M_{j'} \in \mathbb{F}^{2^{w(j')}} \times \mathbb{F}^{2^{w(j')}}$ are:

$$M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \coloneqq \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$$

Given streaming access to p and q, $\mathcal{P}$ can compute any given $p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b})$ and $q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$ in $O(2^{j'})$ time using just $O(\log N)$ space. This is because $p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b})$ can be written as

$$p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b}) = \sum_{\boldsymbol{x} \in \{0,1\}^{j'-1}} \chi_{\boldsymbol{x}}\left(\boldsymbol{r}_{:j'}\right) p(\boldsymbol{x}, \boldsymbol{\beta}_1, \boldsymbol{b}) \ ,$$

and the first $\chi_{\boldsymbol{x}}\left(\boldsymbol{r}_{:j'}\right)$ takes $O(j')$ time to compute, but every subsequent value can be computed in amortized $O(1)$ time (see Section 3.1). Therefore each $p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b})$ can be computed in $O(2^{j'})$ time. Same holds for $q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$. We claim that $\mathcal{P}$ can compute and store $M_{j'}$ in $O\Big(2^{n-j'} \cdot (2^{j'} + 2^{w(j')})\Big)$ time as follows: $\mathcal{P}$ initializes $M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \leftarrow 0$ for all $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')+1}$. Then, for each $\boldsymbol{b} \in \{0,1\}^{n-j}$:

- $\mathcal{P}$ computes and stores $p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}, \boldsymbol{b})$ and $q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}, \boldsymbol{b})$ for all $\boldsymbol{\beta} \in \{0,1\}^{w(j')+1}$ in $O(2^{j'} \cdot 2^{w(j')})$ time.
- $\mathcal{P}$ updates $M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \leftarrow M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] + p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$ for all $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')+1}$, which requires $O(2^{2w(j')})$ time.

Therefore, this algorithm takes $O(2^{n-j} \cdot 2^{w(j')} \cdot (2^{j'} + 2^{w(j')})) = O(2^{n-j'} \cdot (2^{j'} + 2^{w(j')}))$ time. In particular, if $w(j') \leq j'$, the time complexity simplifies to $O(2^{n-j'} \cdot 2^{j'}) = O(N)$. This enforces our second restriction on $w(j')$.

In summary, we have two restrictions on $w(j')$: $w(j') = O(\log N/k)$ because of our space restrictions, and $w(j') = O(j')$ because of our time restrictions. This motivates us to split our protocol into *two phases*.

Two phases. We define $\ell := \lfloor n/2k \rfloor$, and split our protocol into two phases, a *time-constrained phase* with comprising of the first $\ell - 1$ rounds, and a *space-constrained phase* comprising of the remaining rounds.

1. **Time-constrained phase** ($1 \leq j \leq \ell - 1$)**.** In the first few rounds of the protocol, the bottleneck is the time taken to compute $M_{j'}$, as we require that $w(j') \leq j'$. In order to be able to reply correctly for all rounds $j \in [\ell - 1]$, $\mathcal{P}$ computes $M_{j'}$ for each round $j \in \{1, 2, 4, \ldots, 2^s\}$, where s is the largest integer such that $2^s \leq \ell - 1$, with $w(j') := j'$.
2. **Space-constrained phase** ($\ell \leq j \leq n$)**.** In the later rounds of the protocol, the bottleneck is the space required to store $M_{j'}$. To ensure that $\mathcal{P}$ only requires $O(N^{1/k})$ space, we require that $w(j') \leq n/2k$. In order to be able to reply correctly for all rounds $\ell \leq j \leq n$, $\mathcal{P}$ computes $M_{j'}$ for each round $j' \in \{\ell, 2\ell, \ldots, \lfloor n/\ell \rfloor \cdot \ell\}$, with $w(j') := \ell$.

$\mathcal{P}$ makes $\lfloor \log_2 \ell \rfloor = O(\log \log N)$ passes of the input streams in phase 1 and $\lfloor n/\ell \rfloor = O(k)$ passes in phase 2. The total number of passes made by $\mathcal{P}$ is thus $O(\log \log N + k)$, with each passing taking $O(N)$ time. We now describe the prover's implementation in detail.

4.2 Product-sumcheck prover implementation

$\mathsf{ProductSC}_k^{p,q}$:

1. Define $\ell := \lfloor n/2k \rfloor$.
2. For every round $j \in [n]$:
   (a) Initialize $P \leftarrow 0$. ▷ Indicates whether state computation is needed this round
   (b) Define $s := \lfloor \log_2 j \rfloor$.
   (c) If $j \leq \ell - 1$: ▷ time-constrained phase
      i. If $2^s = j$: Set $P \leftarrow 1$.
      ii. Define $j' := 2^s$. ▷ Last round for which $M_{j'}$ was computed
      iii. Define $w(j') := \min(2^s, n - j' + 1)$. ▷ Number of rounds that can be handled by $M_{j'}$
   (d) Else: ▷ space-constrained phase
      i. If $\ell \mid j$: Set $P \leftarrow 1$.
      ii. Define $j' := \ell \cdot \lfloor j/\ell \rfloor$. ▷ Last round for which $M_{j'}$ was computed
      iii. Define $w(j') := \min(\ell, n - j' + 1)$. ▷ Number of rounds that can be handled by $M_{j'}$
   (e) If $P = 1$ (we must have $j = j'$): Let the challenges received so far be $\boldsymbol{r} \in \mathbb{F}^{j'-1}$. For all $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')}$, compute and store

$$M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] := \sum_{\boldsymbol{b} \in \{0,1\}^{n-j'-w(j')+1}} p(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_1, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{\beta}_2, \boldsymbol{b})$$

   This table is computed as follows:

i. For each $\boldsymbol{b} \in \{0,1\}^{n-j'-w(j')+1}$:

ii. For each $\boldsymbol{\beta} \in \{0,1\}^{w(j')}$:

iii. Compute and store $X[\boldsymbol{\beta}] \coloneqq p(\boldsymbol{r}, \boldsymbol{\beta}, \boldsymbol{b}) = \sum_{\boldsymbol{x} \in \{0,1\}^{j'-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r})\, p(\boldsymbol{x}, \boldsymbol{\beta}, \boldsymbol{b})$

iv. For each $\boldsymbol{\beta} \in \{0,1\}^{w(j')}$:

v. Compute and store $Y[\boldsymbol{\beta}] \coloneqq q(\boldsymbol{r}, \boldsymbol{\beta}, \boldsymbol{b}) = \sum_{\boldsymbol{x} \in \{0,1\}^{j'-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r})\, q(\boldsymbol{x}, \boldsymbol{\beta}, \boldsymbol{b})$

vi. For each $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')}$:

vii. Update $M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \leftarrow M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] + X[\boldsymbol{\beta}_1] \cdot Y[\boldsymbol{\beta}_2]$

(f) Let the received challenges so far be $\boldsymbol{r}_{:j} = (\boldsymbol{r}_{:j'}, \boldsymbol{r}') \in \mathbb{F}^{j'-1} \times \mathbb{F}^{j-j'}$.

(g) Compute round polynomial evaluations:

$$f_j(0) = \sum_{\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}')\, \chi_{\boldsymbol{b}_2}(\boldsymbol{r}') \sum_{\boldsymbol{b} \in \{0,1\}^{w(j')-j+j'-1}} M_{j'}[\boldsymbol{b}_{1,0}, \boldsymbol{b}_{2,0}] \tag{7}$$

$$f_j(1) = \sum_{\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}')\, \chi_{\boldsymbol{b}_2}(\boldsymbol{r}') \sum_{\boldsymbol{b} \in \{0,1\}^{w(j')-j+j'-1}} M_{j'}[\boldsymbol{b}_{1,1}, \boldsymbol{b}_{2,1}] \tag{8}$$

$$f_j(z) = \sum_{\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}')\, \chi_{\boldsymbol{b}_2}(\boldsymbol{r}') \sum_{\boldsymbol{b} \in \{0,1\}^{w(j')-j+j'-1}} \begin{pmatrix} (1-z)^2 M_{j'}[\boldsymbol{b}_{1,0}, \boldsymbol{b}_{2,0}] \\ + z(1-z) M_{j'}[\boldsymbol{b}_{1,0}, \boldsymbol{b}_{2,1}] \\ + z(1-z) M_{j'}[\boldsymbol{b}_{1,1}, \boldsymbol{b}_{2,0}] \\ + z^2 M_{j'}[\boldsymbol{b}_{1,1}, \boldsymbol{b}_{2,1}] \end{pmatrix} \tag{9}$$

where $\boldsymbol{b}_{1,0} \coloneqq (\boldsymbol{b}_1, 0, \boldsymbol{b})$, $\boldsymbol{b}_{1,1} \coloneqq (\boldsymbol{b}_1, 1, \boldsymbol{b})$, $\boldsymbol{b}_{2,0} \coloneqq (\boldsymbol{b}_2, 0, \boldsymbol{b})$ and $\boldsymbol{b}_{2,1} \coloneqq (\boldsymbol{b}_2, 1, \boldsymbol{b})$.

(h) Send $f_j(0), f_j(1), f_j(z)$ to $\mathcal{V}$.

(i) Receive r_j from $\mathcal{V}$.

Figure 2: Prover algorithm using $O(N(\log\log N + k))$ time and $O(N^{1/k})$ space for the classical sumcheck protocol for products of multilinear polynomials.

Completeness. Completeness for rounds $j \in [n]$ such that $j = j' + w(j') - 1$ follows from the definition of $M_{j'}$ and the exposition in Section 4.1. We now show completeness for the remaining rounds. In round j, $\mathcal{P}$ outputs

$$\begin{aligned} f_j(0) &= \sum_{\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}')\, \chi_{\boldsymbol{b}_2}(\boldsymbol{r}') \sum_{\boldsymbol{v} \in \{0,1\}^{w(j')-j+j'-1}} M_{j'}[(\boldsymbol{b}_1, 0, \boldsymbol{v}), (\boldsymbol{b}_2, 0, \boldsymbol{v})] \\ &= \sum_{\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}')\, \chi_{\boldsymbol{b}_2}(\boldsymbol{r}') \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}_1, \boldsymbol{b}_1, 0, \boldsymbol{b}) \cdot q(\boldsymbol{r}_1, \boldsymbol{b}_2, 0, \boldsymbol{b}) \\ &= \sum_{\boldsymbol{b} \in \{0,1\}^{n-j}} p(\boldsymbol{r}_{:j'}, \boldsymbol{r}', 0, \boldsymbol{b}) \cdot q(\boldsymbol{r}_{:j'}, \boldsymbol{r}', 0, \boldsymbol{b}) \ , \end{aligned}$$

which is the correct value for $f_j(0)$. Similarly, it can be seen that $\mathcal{P}$ outputs the correct values for $f_j(1)$ and $f_j(z)$ as well.

Efficiency. We will now provide a detailed analysis of the efficiency of $\mathsf{ProductSC}_k^{p,q}$. $\mathcal{P}$'s work can be divided into two parts: computing the tables $M_{j'}$ and computing the round polynomial evaluations in every round j (given the corresponding $M_{j'}$). We will analyze the time and space complexity of each of these parts separately.

Computing the tables. The computation of each table $M_{j'}$, requires $O(N)$ time. Consider an execution of Item 2e : for some fixed values of $\boldsymbol{r} \in \mathbb{F}^{j'-1}, \boldsymbol{\beta}_1 \in \{0,1\}^{w(j')}, \boldsymbol{b} \in \{0,1\}^{n-j'-w(j')+1}$, $\mathcal{P}$ computes

$$p(\boldsymbol{r}, \boldsymbol{\beta}_1, \boldsymbol{b}) = \sum_{\boldsymbol{x} \in \{0,1\}^{j'-1}} \chi_{\boldsymbol{x}}(\boldsymbol{r})\, p(\boldsymbol{x}, \boldsymbol{\beta}_1, \boldsymbol{b})$$

The first $\chi_{\boldsymbol{x}}(\boldsymbol{r})$ takes $O(j')$ time to compute, but every subsequent value can be computed in amortized $O(1)$ time. Therefore each $p(\boldsymbol{r}, \boldsymbol{\beta}_1, \boldsymbol{b})$ can be computed in $O(2^j)$ time and $O(1)$ space. The same can be said of each $q(\boldsymbol{r}, \boldsymbol{\beta}_2, \boldsymbol{b})$. Thus computing and storing $p(\boldsymbol{r}, \boldsymbol{\beta}_1, \boldsymbol{b})$ and $q(\boldsymbol{r}, \boldsymbol{\beta}_2, \boldsymbol{b})$ for all $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^{w(j')}$ would take $O(2^{w(j')} \cdot 2^{j'})$ time and $O(2^{w(j')})$ space.

Given these values, for fixed $\boldsymbol{r}$ and $\boldsymbol{b}$, the algorithm can update and store $M_{j'}[\boldsymbol{\beta}_1, \boldsymbol{\beta}_2] \leftarrow p(\boldsymbol{r}, \boldsymbol{\beta}_1, \boldsymbol{b}) \cdot q(\boldsymbol{r}, \boldsymbol{\beta}_2, \boldsymbol{b})$ for all $\boldsymbol{\beta}_1, \boldsymbol{\beta}_2 \in \{0,1\}^t$ in $O(2^{2t})$ time while using $O(2^{2w(j')})$ space.

Iterating over all possible values of $\boldsymbol{b} \in \{0,1\}^{n-j'-w(j')+1}$, the computation of the table $M_{j'}$ takes

$$O\left(2^{n-j'-w(j')+1} \cdot (2^{w(j')} \cdot 2^{j'} + 2^{2w(j')})\right) = O(2^{n-j'} \cdot (2^{j'} + 2^{w(j')}))$$

time and $O(2^{2w(j')})$ space. Since we ensure that $w(j') \leq j'$ for each j', computing $M_{j'}$ takes $O(2^n) = O(N)$ time.

Since $\mathcal{P}$ makes $\lceil \log_2 \ell \rceil = O(\log \log N)$ passes in phase 1 and $\lceil n/\ell \rceil = O(k)$ passes in phase 2, the total number of rounds j' where $\mathcal{P}$ computes $M_{j'}$ is $O(\log \log N + k)$. Thus the total time taken to compute the tables is $O(N(\log \log N + k))$. The space required to compute and store the tables can be reused between rounds, and is $O(2^{2w(j')}) = O(N^{1/k})$ since we ensure that $w(j') \leq n/2k$ for all j'.

Computing the round polynomial evaluations. In each round $j \in [n]$, $\mathcal{P}$ needs to compute $f_j(0)$, $f_j(1)$ and $f_j(z)$. We will examine the efficiency of computing $f_j(0)$ (the same asymptotic upper bound applies for $f_j(1)$ and $f_j(z)$) as can be seen from Eqs. (8) and (9). Firstly, computing each value of $\chi_{\boldsymbol{b}_1}(\boldsymbol{r}')$ and $\chi_{\boldsymbol{b}_2}(\boldsymbol{r}')$ takes amortized $O(1)$ field operations (see Section 3.1). For each value of $\boldsymbol{b}_1, \boldsymbol{b}_2 \in \{0,1\}^{j-j'}$, we take a linear combination of $2^{w(j')-j+j'-1}$ values of $M_{j'}$, which requires $O(2^{w(j')-j+j'})$ field operations. Doing this computation for all values of $\boldsymbol{b}_1, \boldsymbol{b}_2$ therefore requires $O(2^{w(j')+j-j'}) \leq O(2^j)$ field operations (since $w(j') \leq j'$). Adding over all rounds $j \in [n]$, the total number of field operations required to compute all round polynomials is $O(\sum_{j=1}^{n} 2^j) = O(2^n) = O(N)$. Note that only $O(1)$ extra random-access space is required to output these values for each round.

Overall efficiency. $\mathcal{P}$ requires $O(N(k + \log \log N))$ field operations to compute the tables $M_{j'}$ and $O(N)$ field operations to compute the round polynomial evaluations, which gives us the desired upper bound.

Furthermore, the input streams are only accessed in Item 2(e)iii and Item 2(e)v. In the pseudocode, we omitted the details of how $\mathcal{P}$ accesses $\mathcal{S}(p)$ and $\mathcal{S}(q)$, but we can see that $\mathcal{P}$ only needs streaming access to the evaluations of p and q in the most-significant-bit lexicographic order:[7] in Item 2(e)iii and Item 2(e)v we call $\mathcal{S}(p).\mathsf{next}()$ and $\mathcal{S}(q).\mathsf{next}()$ respectively $2^{w(j')}$ times, and with one pass over the input streams, $\mathcal{P}$ can compute $M_{j'}$ with only $O(N^{1/k})$ space as desired.

Optimization via switching to folding-based sumcheck. We can design another classical prover for $\mathsf{SC}^{(\mathbb{F},n,2)}$ that behaves as follows:

[7] For $n = 3$, this order of evaluations is $(0,0,0), (1,0,0), (0,1,0), (1,1,0), (0,0,1), (1,0,1), (0,1,1), (1,1,1)$.

1. Use $\mathsf{ProductSC}_k$ until round $\lceil n - n/k \rceil$.
2. Use the folding-based prover of [CTY11; Tha13] for the remaining rounds.

Clearly, this prover only requires $O(N^{1/k})$ space, as the folding-based prover is invoked on polynomials of size $O(N/2^{n-n/k}) = O(N^{1/k})$. Also, this prover makes only $O(\log(n-n/k)) = O(\log((1-1/k)\log N) + k)$ passes over the input streams. As k approaches 1 from above (in particular, when $k \leq \log N/(\log N - 1)$), the time complexity of this prover becomes $O(N)$, which matches the folding-based prover of [CTY11; Tha13].

4.3 Extending to products of d multilinear polynomials

We briefly list the changes that need to be made to Fig. 2 in order to extend it to handle products of d multilinear polynomials $p_1, p_2, \ldots, p_d$.

- We define $\ell := \lfloor n/dk \rfloor$, as opposed to $\lfloor n/2k \rfloor$ in the original protocol.
- The space-constrained phase begins in the round $j \geq (d-1)\cdot\ell$. In this phase, we make a pass over the input streams every ℓ iterations of the sumcheck protocol. When a pass is made in round j', $\mathcal{P}$ computes and stores the following values for each $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_d \in \{0,1\}^\ell$:

$$M_{j'}[\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_d] := \sum_{\boldsymbol{b} \in \{0,1\}^{n-j'-\ell+1}} p_1(\boldsymbol{r}, \boldsymbol{x}_1, \boldsymbol{b}) \cdot p_2(\boldsymbol{r}, \boldsymbol{x}_2, \boldsymbol{b}) \cdot \ldots \cdot p_d(\boldsymbol{r}, \boldsymbol{x}_d, \boldsymbol{b}) \ .$$

  - Space: The amount of space needed to store M is $O(2^{d\ell}) = O(N^{1/k})$ as desired.
  - Time: Since $j' \geq (d-1)\cdot\ell$, each entry of M is a linear combination of $O(2^{n-d\ell})$ evaluation products (each of these is a product of d evaluations), and there are $O(2^{d\ell})$ entries in M, the entire table can be computed in $O(d \cdot 2^n) = O(d \cdot N)$ time as desired.
- The time-constrained phase ends in the round $j < (d-1)\cdot\ell$. In Fig. 2, it was sufficient to make passes over the input streams in rounds $1, 2, 4, 8, \ldots$ (powers of 2) in order to not exceed $O(N)$ time in every round. Now, we instead make passes in the rounds $1, \lceil\eta\rceil, \lceil\eta^2\rceil, \ldots$, where $\eta := \frac{d}{d-1}$. When a pass is made over the input streams in round $j' = \lceil\eta^t\rceil$ for some $t \in \mathbb{N}$, $\mathcal{P}$ computes and stores the following values for each $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_d \in \{0,1\}^{\lceil\eta^{t+1}\rceil - \lceil\eta^t\rceil}$:

$$M_{j'}[\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_d] := \sum_{\boldsymbol{b} \in \{0,1\}^{n-\lceil\eta^{t+1}\rceil+1}} p_1(\boldsymbol{r}, \boldsymbol{x}_1, \boldsymbol{b}) \cdot p_2(\boldsymbol{r}, \boldsymbol{x}_2, \boldsymbol{b}) \cdot \ldots \cdot p_d(\boldsymbol{r}, \boldsymbol{x}_d, \boldsymbol{b}) \ .$$

  - Space: clearly $M_{j'}$ requires $O(2^{d\cdot(\eta^{t+1}-\eta^t)}) = O(2^{d\cdot\eta^t/(d-1)}) = O(2^{\eta^{t+1}})$ space. In the time-constrained phase, we must have $\eta^{t+1} \leq n/k$, therefore this is upper bounded by $O(N^{1/k})$ space as desired.
  - Time: there are $O(2^{\eta^{t+1}})$ entries in $M_{j'}$ and each is a linear combination of $O(2^{n-\eta^{t+1}})$ evaluation products. Therefore, the entire table can be computed in $O(d \cdot 2^n) = O(d \cdot N)$ time as desired.
- The computation of the round polynomials also only requires $O(1)$ additional space and $O(N^{1/k})$. In round $j \in [j', j'+\ell-1]$ where $M_{j'}$ was computed, the round polynomial computation is similar to Eq. (7):

$$f_j(0) = \sum_{\substack{\boldsymbol{b}_1 \in \{0,1\}^{j-j'} \\ \vdots \\ \boldsymbol{b}_d \in \{0,1\}^{j-j'}}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}') \cdot \ldots \cdot \chi_{\boldsymbol{b}_d}(\boldsymbol{r}') \sum_{\boldsymbol{b} \in \{0,1\}^{\ell-j+j'-1}} M_{j'}[(\boldsymbol{x}_1 \parallel 0 \parallel \boldsymbol{b}), \ldots, (\boldsymbol{x}_d \parallel 0 \parallel \boldsymbol{b})] \ ,$$

  where $\boldsymbol{r}'$ is the vector of random challenges received between rounds j' and j. Clearly, this requires $O(1)$ extra space and $O(|M_{j'}|) = O(N^{1/k})$ time. These lower order terms can be ignored in the analysis of the total time and space complexity of the algorithm.

- Finally, the total number of passes made is $\log_\eta(n/k) + dk$, and in each pass $\mathcal{P}$ performs $O(N \cdot d)$ field operations. Therefore, the total time complexity of this algorithm is

$$O\left(N \cdot d \cdot \left(\frac{\log\log N}{k \log(d/(d-1))} + dk\right)\right) .$$

4.4 Linear-time prover using fast matrix multiplication

The time-constrained phase of Fig. 2 exists because computing the table $M_{j'}$ requires $O(2^{n-j'+w(j')})$ time, which restricts the value of $w(j')$ to be at most j' so that the $M_{j'}$ can be computed in linear time. This leads to the number of passes being $O(\log\log N)$ in the time-constrained phase. However, if there exists a *fast matrix multiplication algorithm* that can compute these cross products efficiently, then we can remove the time-constrained phase and get away with only $O(k)$ passes over the input streams. We formalize this in the following theorem.

Theorem 4.3. *Say there exists an algorithm $\mathcal{A}$ that takes as input two matrices $A, B \in \mathbb{F}^{m \times m}$ and outputs the product $C = A \cdot B$ in $O(m^2)$ field operations. Then, for any $k \geq 1$, there exists a prover $\mathcal{P}$ for the classical sumcheck protocol for $\mathsf{SC}^{(\mathbb{F},n,2)}$ which requires $O(k \cdot N)$ field operations and $O(N^{1/k})$ space.*[8]

Before proving Theorem 4.3, we first show that the batch polynomial multiplication problem can be solved efficiently using fast matrix multiplication. For simplicity, we prove the theorem for fields of characteristic greater than 2 (so we can assume that $1/2$ exists), but the proof can be easily extended to fields of characteristic 2 by using a different evaluation point.

Definition 4.4 (Adjacent evaluations)**.** *Consider an n-variate multilinear polynomial $p(\boldsymbol{X}) \in \mathbb{F}[\boldsymbol{X}]$. Given an evaluation point $\boldsymbol{x} \in \{0, 1, 1/2\}^n$, we define the set of adjacent evaluations of $\boldsymbol{r}$ as follows:*

$$\mathsf{adj}(\boldsymbol{x}) := \{\boldsymbol{b} \in \{0,1\}^n : \textit{for all } i \in [n] : |x_i - b_i| \leq 1/2\} .$$

That is, b_i can be 0 or 1 if $x_i = \frac{1}{2}$, but $b_i = x_i$ otherwise. For all $\boldsymbol{x} \in \{0, 1, 1/2\}^n$, we have

$$\chi_{\boldsymbol{b}}(\boldsymbol{x}) = \begin{cases} 0 & \textit{if } \boldsymbol{b} \notin \mathsf{adj}(\boldsymbol{x}) \\ \frac{1}{2^w} & \textit{if } \boldsymbol{b} \in \mathsf{adj}(\boldsymbol{x}) \textit{ where } w \textit{ is the number of } 1/2\textit{s in } \boldsymbol{x} \end{cases}$$

This gives us $p_k(\boldsymbol{x}) = \sum_{\boldsymbol{b} \in \{0,1\}^n} \chi_{\boldsymbol{b}}(\boldsymbol{x})\, p_k(\boldsymbol{b}) = \frac{1}{2^w} \sum_{\boldsymbol{b} \in \mathsf{adj}(\boldsymbol{x})} p_k(\boldsymbol{b})$.

Lemma 4.5. *$\mathsf{BPM}(n, \ell)$ can be solved in $O(2^{(1+\omega)n} \cdot \ell)$ time if $\ell \geq 2^n$, and there exists an algorithm $\mathcal{M}$ for multiplying two $m \times m$ matrices in $O(m^{2+\omega})$ time.*

Proof. Consider the $N := 2^n$ evaluations of $p_1(\boldsymbol{X}), \ldots, p_S(\boldsymbol{X})$ over $\{0,1\}^n$. These can be arranged in a matrix P of dimensions $N \times S$ by arranging them in a row-major order. Similarly, the evaluations of $q_1(\boldsymbol{X}), \ldots, q_S(\boldsymbol{X})$ can be arranged in a matrix Q of dimensions $S \times N$ by arranging them in column-major order. Now, consider the values of the product matrix $T := PQ$. Clearly $T[i][j] = \sum_{k=1}^{S} p_k(\langle i \rangle) \cdot q_k(\langle j \rangle)$. P can be broken into S/N many $N \times N$ matrices $P_1, \ldots, P_{S/N}$, and Q can be broken into S/N many $N \times N$ matrices $Q_1, \ldots, Q_{N/S}$. Clearly $T = PQ = \sum_{i=1}^{S/N} P_i Q_i$, and therefore T can be computed in $O(N^{2+\omega} \cdot S/N) = O(N^{1+\omega} \cdot S)$ time.

[8] For ease of exposition, we assume that the field has characteristic at least 3 so that the $1/2 \in \mathbb{F}$. This can be easily extended to fields of characteristic 2 by using a different evaluation point.

Now consider the desired output polynomial $f(\boldsymbol{X}) := \sum_{k=1}^{S} p_k(\boldsymbol{X}) \cdot q_k(\boldsymbol{X})$. This polynomial has at most degree 2 in each variable, and therefore requires 3^n unique evaluations to define it. We can compute these evaluations from T, since for all $\boldsymbol{x} \in \{0, 1, 1/2\}^n$, we have:

$$f(\boldsymbol{x}) = \sum_{k=1}^{S} p_k(\boldsymbol{x}) q_k(\boldsymbol{x}) = \frac{1}{2^{2w}} \sum_{k=1}^{S} \left(\sum_{\boldsymbol{b} \in \mathsf{adj}(\boldsymbol{x})} p_k(\boldsymbol{b}) \right) \left(\sum_{\boldsymbol{b} \in \mathsf{adj}(\boldsymbol{x})} q_k(\boldsymbol{b}) \right)$$
$$= \frac{1}{2^{2w}} \sum_{\boldsymbol{b}, \boldsymbol{b}' \in \mathsf{adj}(\boldsymbol{x})} \sum_{k=1}^{S} p_k(\boldsymbol{b}) q_k(\boldsymbol{b}') = \frac{1}{2^{2w}} \sum_{\boldsymbol{b}, \boldsymbol{b}' \in \mathsf{adj}(\boldsymbol{x})} T[\mathsf{int}(\boldsymbol{b})][\mathsf{int}(\boldsymbol{b}')] \ .$$

There are $2^{n-i} \cdot \binom{n}{i}$ evaluations points that have i many $1/2$'s, and each of them is a sum of 2^i values in T. Therefore, the total time needed to compute all of these evaluations is $\sum_{i=0}^{n} 2^{n-i} \cdot \binom{n}{i} \cdot 2^i = \sum_{i=0}^{n} 2^n \cdot \binom{n}{i} = 2^{2n} = O(N^2)$. Therefore $\mathcal{A}$ is an algorithm that solves $\mathsf{BPM}(n, S)$ in $O(2^{(1+\omega)n} \cdot S)$ time. In particular if $\omega = 0$, this contradicts Conjecture 2.7 as desired. □

Proof of Theorem 4.3. Consider the i-th round of the interaction, such that $\mathcal{P}$ has received the random challenges $\boldsymbol{r} = (r_1, \ldots, r_{i-1})$ from $\mathcal{V}$. Using the algorithm from Lemma 4.5, $\mathcal{P}$ can compute

$$f(\boldsymbol{X}) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-t-i}} p(\boldsymbol{r}, \boldsymbol{X}, \boldsymbol{b}) \cdot q(\boldsymbol{r}, \boldsymbol{X}, \boldsymbol{b}) \ ,$$

This requires $O(2^t \cdot 2^{n-t-i}) = O(2^{n-i})$ time if $\omega = 0$, in addition to the $O(N)$ time required to initialize polynomials $p(\boldsymbol{r}, \boldsymbol{X}, \boldsymbol{b})$ and $q(\boldsymbol{r}, \boldsymbol{X}, \boldsymbol{b})$. Therefore, regardless of the value of t or i, it takes $O(N)$ time to initialize the polynomial f. We do not have a time-constrained phase like we did in Fig. 2, so the value of t can be set as large as the space constraints would allow. Given $O(N^{1/k})$ space, we can set $t = O(n/k)$ to match the space constraints, and therefore only make a pass of the input polynomials $O(k)$ times, giving a total time complexity of $O(kN)$ as desired. This matches the time-space tradeoff of the multilinear-sumcheck protocol (see Appendix A), and the multilinear-sumcheck lower bound of Section 5. □

5 Time-space tradeoff lower bound for multilinear-sumcheck

In Section 5.1, we describe the general behavior of an honest prover $\mathcal{P}$ of the classical sumcheck protocol for $\mathsf{SC}^{(\mathbb{F},n,1)}$, and introduce the relevant notation. In Section 5.2, we give a formal statement of the lower bound theorem, and then prove it using the description of the honest prover in Section 5.1.

5.1 Behavior of space-bounded sumcheck provers

Consider $(p(\boldsymbol{X}), \sigma)$, an instance of $\mathsf{SC}^{(\mathbb{F},n,1)}$, and consider an honest sumcheck prover $\mathcal{P}$ with space bound $S = 2^s$. We assume that in round $i \in [n]$, $\mathcal{P}$ sends a degree-1 polynomial $t_i(X)$ in the form of $a_i := t_i(0)$ and $b_i := t_i(1)$ to $\mathcal{V}$. This is without loss of generality for the purposes of this lower bound, since both $\mathcal{P}$ and $\mathcal{V}$ can compute $t_i(0)$ and $t_i(1)$ in $O(1)$ time and space using any two values that uniquely determine t_i.

We discuss the computation that $\mathcal{P}$ performs in the rounds $\mathcal{I} := \{1, 1+s, 1+2s, \ldots, 1+(\lfloor n/s \rfloor - 1)\, s\}$. In this section we assume that $\mathcal{P}$ can correctly compute $t_i(X)$ for each $i \notin \mathcal{I}$ for free, and focus on its behavior in the remaining rounds $i \in \mathcal{I}$. Any operations that $\mathcal{P}$ would have performed in any round $j \notin \mathcal{I}$ are therefore delegated to a round $i \in \mathcal{I}$ that follows it. Since we are focussed on only the rounds in $\mathcal{I}$, we can re-interpret the classical sumcheck protocol as a $|\mathcal{I}| = \lfloor n/s \rfloor$ round interaction, such that in round $i \in \mathcal{I}$, $\mathcal{V}$ sends s random field elements $(r_i, r_{i+1}, \ldots, r_{i+s-1})$ to $\mathcal{P}$. We define $\mathsf{chall}_i := (r_1, \ldots, r_{i-1})$ as all the challenges received by $\mathcal{P}$ before the start of round i, with $\mathsf{chall}_1 := \perp$.

In a particular round $i \in \mathcal{I}$, $\mathcal{P}$ begins with the state $\mathsf{state}_i \in \mathbb{F}^S$ (we assume that $\mathsf{state}_1 = 0^S$) and performs the following steps:

1. *Perform reads.* It outputs an ordered sequence of indices of $\boldsymbol{p}$ to read, $\mathsf{indices}_i$, given random access to $p(\boldsymbol{X})$. This is captured by the $\mathsf{ComputeIndices}_i$ function:

$$\mathsf{indices}_i \leftarrow \mathsf{ComputeIndices}_i^{\boldsymbol{p}}(\mathsf{state}_i, \mathsf{chall}_i) \ .$$

   This specifies $\mathsf{values}_i = \boldsymbol{p}|_{\mathsf{indices}_i}$, the values of $\boldsymbol{p}$ corresponding to the indices that are read by $\mathcal{P}$. For convenience, we define the set of indices read in the i-th round along with the corresponding values as $\mathsf{reads}_i := \{((\mathsf{indices}_i)_j, (\mathsf{values}_i)_j) \ : \ j \in [|\mathsf{indices}_i|]\}$.
2. *Compute responses.* Using state_i, reads_i, chall_{i-s}, $\mathcal{P}$ computes the two field elements it must send to $\mathcal{V}$. This is captured by the $\mathsf{ComputeResponse}_i$ function:

$$(a_i, b_i) \leftarrow \mathsf{ComputeResponse}_i(\mathsf{state}_i, \mathsf{reads}_i, \mathsf{chall}_i) \ .$$

3. *Compute new state.* Finally, $\mathcal{P}$ uses state_i, reads_i, chall_{i+s} (this step takes place after the challenges for round i are received) to compute state_{i+s}, the state to be passed on for the next round. This is captured by the $\mathsf{ComputeState}_i$ function:

$$\mathsf{state}_{i+s} \leftarrow \mathsf{ComputeState}_i(\mathsf{state}_i, \mathsf{reads}_i, \mathsf{chall}_{i+s}) \ .$$

   Our lower bound proof does not rely on the implementation of $\mathsf{ComputeState}_i$, and applies for any $\mathsf{state}_{i+s} \in \mathbb{F}^S$.

We note that we are not concerned with the efficiency of these steps performed by $\mathcal{P}$.

Definition 5.1 (Non-adaptive provers)**.** *An honest sumcheck prover $\mathcal{P}$ for $\mathsf{SC}^{(\mathbb{F},n,1)}$ is said to be* non-adaptive *if for all $i \in \mathcal{I}$, $\mathsf{ComputeIndices}_i^{\boldsymbol{p}}$ is independent of $\boldsymbol{p}$, state_i and chall_i.*

Ideal provers. For the purposes of this lower bound, we are concerned with honest provers for $\mathsf{SC}^{(\mathbb{F},n,1)}$ which are deterministic, non-adaptive, and have a space bound of $S := 2^s$. We denote this set of provers as $\mathfrak{I}^{(\mathbb{F},n,s)}$, and refer to them as *ideal provers*.

5.2 Lower bound theorem and proof

Theorem 5.2. *Consider a prover $\mathcal{P} \in \mathfrak{I}^{(\mathbb{F},n,s)}$, and define $T := \sum_{i\in\mathcal{I}} |\mathsf{reads}_i(\mathcal{P})|$, $N := 2^n$ and $S := 2^s$. Then, we have*

$$T \geq N + \frac{N}{2} \cdot \left(\left\lfloor \frac{n}{s} \right\rfloor - 1 \right) \tag{10}$$

In particular, $T \log S = \Omega(N \log N)$.

We fix a prover $\mathcal{P} \in \mathfrak{I}^{(\mathbb{F},n,s)}$ and a multilinear polynomial $p(\boldsymbol{X})$ for the purposes of this proof. We first show that $\mathcal{P}$ must read N indices of $\boldsymbol{p}$ in round 1 (Lemma 5.3), and then show that $\mathcal{P}$ read at least $N/2$ indices of $\boldsymbol{p}$ in each round $i \in \mathcal{I} \setminus \{1\}$. Proving these two lemmas clearly implies Theorem 5.2.

Lemma 5.3. $|\mathsf{reads}_1(\mathcal{P})| \geq N$.

Proof. Assume for the sake of contradiction that $|\mathsf{reads}_1(\mathcal{P})| \leq N - 1$. Without loss of generality, say $\mathcal{P}$ did not read $p(\langle 0\rangle)$. There exists another n-variate multilinear polynomial q such that $p(\langle 0\rangle) \neq q(\langle 0\rangle)$ but $p(\langle j\rangle) = q(\langle j\rangle)$ for all $j \in \{1,\ldots,N-1\}$. $\mathcal{P}$ must fail to return the correct value of either $p_1(0) = \sum_{\boldsymbol{x}\in\{0,1\}^{n-1}} p(0, x_1, \ldots, x_{n-1})$ or $q_1(0) = \sum_{\boldsymbol{x}\in\{0,1\}^{n-1}} q(0, x_1, \ldots, x_{n-1})$. This is because $\mathcal{P}$ would return the same value of a_1 on both inputs, as it has read the exact same values in both cases, and $\mathsf{state}_1 = 0^S$ and $\mathsf{chall}_1 = \perp$. However, since $p_1(0) \neq q_1(0)$, this contradicts the fact that $\mathcal{P}$ is complete. □

Lemma 5.4. *For all $t \in \{1,\ldots,\lfloor n/s\rfloor - 1\}$, we have $|\mathsf{reads}_{ts+1}(\mathcal{P})| \geq N/2$.*

Proof. Assume for the sake of contradiction that $|\mathsf{reads}_{ts+1}(\mathcal{P})| < N/2$ for some $t \in \{1,\ldots,\lfloor n/s\rfloor - 1\}$. Since $\mathcal{P}$ is non-adaptive, $\mathsf{indices}_{ts+1}$ is independent of $\boldsymbol{p}$. Since $\mathcal{P}$ is also complete, it must be the case that for every sequence of challenges $\mathsf{chall}_{ts+1} = \boldsymbol{r} \in \mathbb{F}^{ts}$ that $\mathsf{ComputeResponse}_{ts+1}(\mathsf{state}_{ts+1}, \mathsf{reads}_{ts+1}, \mathsf{chall}_{ts+1})$ outputs $(p^{\boldsymbol{r}}_{ts+1}(0), p^{\boldsymbol{r}}_{ts+1}(1))$. We partition $\mathsf{chall}_{ts+1} = (\boldsymbol{r}_L, \boldsymbol{r}_R)$, such that $\boldsymbol{r}_R := (r_{(t-1)s+1}, \ldots, r_{ts})$ are the random challenges received after the round $(t-1)s+1$, and $\boldsymbol{r}_L := (r_1, \ldots, r_{(t-1)s})$ are the random challenges received in previous rounds. For each $v \in \{0,1\}$, we can rewrite the evaluations $p^{\boldsymbol{r}}_{ts+1}(v)$ as follows:

$$\begin{aligned}
p^{\boldsymbol{r}}_{ts+1}(v) &= \sum_{\boldsymbol{x}\in\{0,1\}^{n-ts-1}} p(\boldsymbol{r}_L, \boldsymbol{r}_R, v, \boldsymbol{x}) \\
&= \sum_{\boldsymbol{x}\in\{0,1\}^{n-ts-1}} \sum_{\boldsymbol{b}\in\{0,1\}^s} \chi_{\boldsymbol{b}}(\boldsymbol{r}_R) \cdot p(\boldsymbol{r}_L, \boldsymbol{b}, v, \boldsymbol{x}) \qquad \text{[Lagrange interpolation of } p\text{]} \\
&= \sum_{\boldsymbol{b}\in\{0,1\}^s} \chi_{\boldsymbol{b}}(\boldsymbol{r}_R) \cdot \sum_{\boldsymbol{x}\in\{0,1\}^{n-ts-1}} p(\boldsymbol{r}_L, \boldsymbol{b}, v, \boldsymbol{x}) \\
&= \sum_{\boldsymbol{b}\in\{0,1\}^s} \chi_{\boldsymbol{b}}(\boldsymbol{r}_R) \cdot \mathsf{S}_{t-1}(\boldsymbol{b}, v) \ ,
\end{aligned}$$

where we define $\mathsf{S}_{t-1}(\boldsymbol{b}, v) := \sum_{\boldsymbol{x}\in\{0,1\}^{n-ts-1}} p(\boldsymbol{r}_L, \boldsymbol{b}, v, \boldsymbol{x})$. Therefore, if $\boldsymbol{r}_R \in \{0,1\}^s$, then we have

$$p^{\boldsymbol{r}}_{s+1}(v) = \sum_{\boldsymbol{b}\in\{0,1\}^s} \chi_{\boldsymbol{b}}(\boldsymbol{r}_R) \cdot \mathsf{S}_{t-1}(\boldsymbol{b}, v) = \mathsf{S}_{t-1}(\boldsymbol{r}_R, v) \ ,$$

where the last equality holds because $\chi_{\boldsymbol{b}}(\boldsymbol{r}_R) = 1$ if $\boldsymbol{b} = \boldsymbol{r}_R$ and 0 otherwise. Therefore, we have $\mathsf{ComputeResponse}_{ts+1}(\mathsf{state}_{ts+1}, \mathsf{reads}_{ts+1}, (\boldsymbol{r}_L, \boldsymbol{r}_R)) = (\mathsf{S}_{t-1}(\boldsymbol{r}_R, 0), \mathsf{S}_{t-1}(\boldsymbol{r}_R, 1))$, for any $\boldsymbol{r}_R \in \{0,1\}^s$.

Sub-cubes of evaluations. For each $\boldsymbol{\beta} \in \{0,1\}^{s+1}$, we define a *sub-cube* of the hypercube $\{0,1\}^n$ as follows:

$$C(\boldsymbol{\beta}) := \left\{(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \ : \ \boldsymbol{x}_L \in \{0,1\}^{ts-s}, \boldsymbol{x}_R \in \{0,1\}^{n-ts-1}\right\}$$

For each $\boldsymbol{r}_R \in \{0,1\}^s$ and $v \in \{0,1\}$, $\mathsf{S}_{t-1}(\boldsymbol{r}_R, v)$ is a linear combination of the evaluations of p over a sub-cube $C(\boldsymbol{\beta})$, where $\boldsymbol{\beta} = (\boldsymbol{r}_R, v)$:

$$\begin{aligned}
\mathsf{S}_{t-1}(\boldsymbol{r}_R, v) &= \sum_{\boldsymbol{x}_R \in \{0,1\}^{n-ts-1}} p(\boldsymbol{r}_L, \boldsymbol{r}_R, v, \boldsymbol{x}_R) \\
&= \sum_{\substack{\boldsymbol{x}_L \in \{0,1\}^{ts-s} \\ \boldsymbol{x}_R \in \{0,1\}^{n-ts-1}}} \chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \cdot p(\boldsymbol{x}_L, \boldsymbol{r}_R, v, \boldsymbol{x}_R) \\
&= \sum_{(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \in C(\boldsymbol{\beta})} \chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \cdot p(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \ .
\end{aligned}$$

For each $\boldsymbol{\beta} \in \{0,1\}^{s+1}$, we partition the sub-cube $C(\boldsymbol{\beta})$ into two sub-cubes: the read sub-cube $R(\boldsymbol{\beta}) \subseteq C(\boldsymbol{\beta})$, containing indices that $\mathcal{P}$ has read in round $ts+1$, and the unread sub-cube $U(\boldsymbol{\beta}) \subseteq C(\boldsymbol{\beta})$, containing indices that $\mathcal{P}$ has not read in round $ts+1$. More precisely, $C(\boldsymbol{\beta}) = U(\boldsymbol{\beta}) \cup R(\boldsymbol{\beta})$, and $U(\boldsymbol{\beta}) \cap \mathsf{indices}_{ts+1} = \emptyset$, and $R(\boldsymbol{\beta}) \cap \mathsf{indices}_{ts+1} = R(\boldsymbol{\beta})$.

Additionally, for each $\boldsymbol{\beta} \in \{0,1\}^{s+1}$, we define weighted sums of the unread and read parts of $C(\boldsymbol{\beta})$ as follows:

$$\begin{aligned}
\mathsf{US}_{t-1}(\boldsymbol{\beta}) &:= \sum_{(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \in U(\boldsymbol{\beta})} \chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \cdot p(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \\
\mathsf{RS}_{t-1}(\boldsymbol{\beta}) &:= \sum_{(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R) \in R(\boldsymbol{\beta})} \chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \cdot p(\boldsymbol{x}_L, \boldsymbol{\beta}, \boldsymbol{x}_R)
\end{aligned}$$

Clearly, $\mathsf{S}_{t-1}(\boldsymbol{\beta}) = \mathsf{US}_{t-1}(\boldsymbol{\beta}) + \mathsf{RS}_{t-1}(\boldsymbol{\beta})$ for all $\boldsymbol{\beta} \in \{0,1\}^{s+1}$.

Several partially unread sub-cubes. Since we assume that $|\mathsf{reads}_{ts+1}(\mathcal{P})| < N/2$, there are at least $S+1$ sub-cubes that are partially unread, where a sub-cube $C(\boldsymbol{\beta})$ is said to be partially unread if $U(\boldsymbol{\beta}) \neq \emptyset$.

To see this, assume for the sake of contradiction that there are at most S sub-cubes that are partially unread by $\mathcal{P}$ in round $ts+1$. Therefore, there are at least $2^{s+1} - S = S$ sub-cubes that are completely read by $\mathcal{P}$ in round $ts+1$, let these sub-cubes be $\boldsymbol{\beta}_1, \ldots, \boldsymbol{\beta}_M$. That is, $R(\boldsymbol{\beta}_i) = C(\boldsymbol{\beta}_i)$ for all $i \in [S]$. By construction of the sub-cubes, they are all disjoint, and each of them has size $N/2S$. Therefore, $|\mathsf{reads}_{ts+1}(\mathcal{P})| \geq \sum_{i=1}^{S} |R(\boldsymbol{\beta}_i)| = S \cdot (N/2S) = N/2$, contradicting our assumption.

Rewinding algorithm. Let $\mathcal{J}$ be the set of all $\boldsymbol{\beta} \in \{0,1\}^{s+1}$ such that the corresponding sub-cube $C(\boldsymbol{\beta})$ is partially unread by $\mathcal{P}$ in round $ts+1$. Consider the following "rewinding" algorithm $\mathcal{R}$ in Fig. 3: it receives as input state_{ts+1} and reads_{ts+1}, and invokes the $\mathsf{ComputeResponse}_{ts+1}$ method to output the weighted sum $\mathsf{US}_{t-1}(\boldsymbol{\beta})$ for each $\boldsymbol{\beta} \in \mathcal{J}$:

$\mathcal{R}(\mathsf{state}_{ts+1}, \mathsf{reads}_{ts+1}) \to [\mathsf{US}_{t-1}(\boldsymbol{\beta})]_{\boldsymbol{\beta} \in \mathcal{J}}$:
1. For each $\boldsymbol{b} \in \{0,1\}^s$:

2. $(\mathsf{S}_{t-1}(\boldsymbol{b},0),\mathsf{S}_{t-1}(\boldsymbol{b},1)) \leftarrow \mathsf{ComputeResponse}_{ts+1}(\mathsf{state}_{ts+1},\mathsf{reads}_{ts+1},\boldsymbol{b})$.
3. For each $\boldsymbol{\beta} \in \mathcal{J}$:
4. Compute $\mathsf{RS}_{t-1}(\boldsymbol{\beta})$ using reads_{ts+1}.
5. Obtain $\mathsf{S}_{t-1}(\boldsymbol{\beta})$ by picking the appropriate value from $[\mathsf{S}_{t-1}(\boldsymbol{b})]_{\boldsymbol{b}\in\{0,1\}^{s+1}}$.
6. Compute $\mathsf{US}_{t-1}(\boldsymbol{\beta}) \leftarrow \mathsf{S}_{t-1}(\boldsymbol{\beta}) - \mathsf{RS}_{t-1}(\boldsymbol{\beta})$.
7. Output $[\mathsf{US}_{t-1}(\boldsymbol{\beta})]_{\boldsymbol{\beta}\in\mathcal{J}}$.

Figure 3: Rewinding algorithm to compute unread sums.

We will now show that this ability to output the weighted sums of all the unread sub-cubes corresponding to $\mathcal{J}$ allows us to solve the problem in Corollary 3.4.

Reduction from Corollary 3.4. Let $S' := |\mathcal{J}| > S$. We show how $\mathcal{A}$, upon receiving a vector $\boldsymbol{u} \xleftarrow{\$} \mathbb{F}^{S'}$ as input, can send a message Π with $|\Pi| = S$ to $\mathcal{B}$, using which $\mathcal{B}$ outputs $\boldsymbol{u}$ with probability 1.

Since $\mathcal{P} \in \mathfrak{I}^{(\mathbb{F},n,s)}$ is non-adaptive, $\mathcal{A}$ and $\mathcal{B}$ both have access to $\mathsf{indices}_{ts+1}$, and therefore know $R(\boldsymbol{b})$ and $U(\boldsymbol{b})$ for all $\boldsymbol{b} \in \{0,1\}^{s+1}$. Additionally, $\mathcal{A}$ and $\mathcal{B}$ also know the set $\mathcal{J}$. Thus they can fix an indexing function $I : \mathcal{J} \to [S']$ (consider the natural bijection obtained by lexicographically ordering $\mathcal{J}$ and mapping the i-th element to i).

Before receiving $\boldsymbol{u}$, $\mathcal{A}$ and $\mathcal{B}$ can both fix a value for the challenges $\boldsymbol{r}_L$ from the space $\mathbb{F}^{ts-s} \setminus \{0,1\}^{ts-s}$, and partially initialize the polynomial p such that $p(\boldsymbol{j}) = 0$, for every $\boldsymbol{j} \in \cup_{\boldsymbol{b}\in\{0,1\}^{s+1}} R(\boldsymbol{b})$.

Upon receiving $\boldsymbol{u}$, $\mathcal{A}$ initializes the rest of the polynomial p such that for each $\boldsymbol{b} \in \mathcal{J}$ we have

$$\begin{aligned} u_{I(\boldsymbol{b})} &= \mathsf{US}_{t-1}(\boldsymbol{b}) \\ &= \sum_{(\boldsymbol{x}_L,\boldsymbol{b},\boldsymbol{x}_R)\in U(\boldsymbol{b})} \chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \cdot p(\boldsymbol{x}_L,\boldsymbol{b},\boldsymbol{x}_R) \ . \end{aligned}$$

This is possible because $\boldsymbol{r}_L \in \mathbb{F}^{ts-s} \setminus \{0,1\}^{ts-s}$ implies that $\chi_{\boldsymbol{x}_L}(\boldsymbol{r}_L) \neq 0$ for all $\boldsymbol{x}_L \in \{0,1\}^{ts-s}$, and the fact that $U(\boldsymbol{b})$ are all disjoint, over all $\boldsymbol{b} \in \mathcal{J}$, means that the evaluations of p that $\mathcal{A}$ sets to satisfy one equation do not affect the the validity of the other equations.

Finally, $\mathcal{A}$ and $\mathcal{B}$ engage in the following protocol:

1. $\mathcal{A}$ simulates the sumcheck prover $\mathcal{P}$ on the polynomial p until round $ts+1$ using the random challenges $\boldsymbol{r}_L$ and obtains state_{ts+1}.
2. $\mathcal{A}$ sends $\Pi := \mathsf{state}_{ts+1}$ to $\mathcal{B}$.
3. $\mathcal{B}$ parses Π as state_{ts+1}, obtains reads_{ts+1} from the initialized values : $p(\boldsymbol{j}) = 0$, for every $\boldsymbol{j} \in \cup_{\boldsymbol{b}\in\{0,1\}^{s+1}} R(\boldsymbol{b})$.
4. $\mathcal{B}$ runs the rewinding algorithm $\mathcal{R}(\mathsf{state}_{ts+1},\mathsf{reads}_{ts+1})$ to compute $[\mathsf{US}_{t-1}(\boldsymbol{b})]_{\boldsymbol{b}\in\mathcal{J}}$.
5. $\mathcal{B}$ defines $u_{I(\boldsymbol{b})} := \mathsf{US}_{t-1}(\boldsymbol{b})$ for all $\boldsymbol{b} \in \mathcal{J}$ and outputs $\boldsymbol{u}$.

Clearly, the amount of communication in the above protocol is $|\mathsf{state}_{ts+1}| \leq S$, and $\mathcal{B}$ is able to output $\boldsymbol{u} \in \mathbb{F}^{S'}$ with probability 1, which contradicts Corollary 3.4. Therefore, $|\mathsf{reads}_{ts+1}(\mathcal{P})| \geq N/2$ as desired. □

6 Time-space tradeoff lower bound for product-sumcheck

In this section we show that the space-efficient product-sumcheck prover presented in Section 4 achieves the optimal time-space tradeoff within a class of natural provers. In particular, we make the following assumptions about the prover algorithm $\mathcal{P}$:

- Similar to Section 5, $\mathcal{P}$ is *non-adaptive* with respect to the input polynomials and the random challenges received in the interactive phase.
- $\mathcal{P}$ is extension-agnostic: for a fixed field characteristic χ, $\mathcal{P}$ implements the same arithmetic circuits $\mathcal{C}_1, \ldots, \mathcal{C}_n$ for all finite fields with characteristic χ.
- We assume that in any round of the sumcheck protocol where $\mathcal{P}$ makes at most $N/4$ field operations, it reads at most $N^{1-\varepsilon}$ values of p and q for some constant $\varepsilon > 0$. This assumption is not needed for the reduction to Conjecture 6.2.

The prover algorithm in Section 4 adheres to these restrictions, and so do the prover algorithms of [CTY11; CMT12; Tha13].

6.1 Background

Conjectures. Consider n-variate multilinear polynomials that evaluate to 0 over a 2^{n-1}-sized subset of the hypercube $\{0,1\}^n$. In addition to Conjecture 2.7, we also propose a slightly stronger conjecture about the complexity of computing the sum of products of such multilinear polynomials. Assuming this stronger conjecture helps us remove the third assumption above.

Definition 6.1. *$\mathcal{A}$ solves the* batch half-polynomial multiplication *problem with parameters n, ℓ (denoted by $\mathsf{BHPM}(n,\ell)$) if there exist sets $B_1, \ldots, B_\ell \subseteq \{0,1\}^n$ such that each $|B_i| = 2^{n-1}$ and for all $p_1, \ldots, p_\ell, q_1, \ldots, q_\ell \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_n]$, $\mathcal{A}$ outputs $f(\boldsymbol{X}) := \sum_{i=1}^{\ell} p'_i(\boldsymbol{X}) \cdot q'_i(\boldsymbol{X})$ where $p'_i(\boldsymbol{x}) = p_i(\boldsymbol{x})$ if $\boldsymbol{x} \in B_i$, and $p'_i(\boldsymbol{x}) = 0$ if $\boldsymbol{x} \notin B_i$, assuming that $\mathcal{A}$ has random-access to the input polynomials.*

Conjecture 6.2. *There exists a constant $\omega > 0$ such that any algorithm $\mathcal{A}$ that solves $\mathsf{BHPM}(n,\ell)$ requires $\Omega(2^{(\omega+1)n} \cdot \ell)$ field operations for all $n, \ell \in \mathbb{N}$.*

Extension-agnostic prover. The output of the prover of a product-sumcheck protocol is bilinear in the two input polynomials p and q. Previous works that study lower bounds for arithmetic circuits computing bilinear functions [Raz02; SY10] use the following high-level idea. Given an arithmetic circuit $\mathcal{C}$ over a field $\mathbb{F}$ of characteristic 0 that outputs bilinear functions $f_1(\boldsymbol{X}, \boldsymbol{Y}), \ldots, f_k(\boldsymbol{X}, \boldsymbol{Y})$, there exists a slightly modified circuit $\mathcal{C}'$ of similar size and structure that also outputs $f_1(\boldsymbol{X}, \boldsymbol{Y}), \ldots, f_k(\boldsymbol{X}, \boldsymbol{Y})$, but additionally *every gate is bilinear* in $\boldsymbol{X}$ and $\boldsymbol{Y}$. That is, the inputs of multiplication gates are always of the form $L_i(\boldsymbol{X})$ and $L_j(\boldsymbol{Y})$ where L_i and L_j are linear polynomials in $\boldsymbol{X}$ and $\boldsymbol{Y}$ respectively. We cannot directly apply this result to our setting since we are working with finite fields, but we get a similar result by assuming that the algorithm is *extension-agnostic*, i.e., its structure is independent of the field's extension degree.

Definition 6.3 (Extension-agnostic product-sumcheck prover)**.** *Fix an input size $n \in \mathbb{N}$. An extension-agnostic prover for $\mathsf{SC}^{(\mathbb{F},n,2)}$ is a function that takes as input a field characteristic $\chi \geq 2$, and outputs a set of arithmetic circuits $\mathcal{C} := (\mathcal{C}_1, \ldots, \mathcal{C}_n)$ such that $\mathcal{C}$ is a prover for the classical sumcheck protocol (see Definition 2.2) for all finite fields $\mathbb{F}$ of characteristic χ. That is, the prover algorithm can be non-uniform with respect to different input sizes and characteristic of the field, but must be uniform with respect to the extension degree of the field.*

Since we restrict the structure of the arithmetic circuits to be the same for all fields of characteristic χ, we make the following observation:

Lemma 6.4. *Consider an extension-agnostic product-sumcheck prover $\mathcal{P}$. For a characteristic $\chi \geq 2$, let $\mathcal{C}_i$ be the arithmetic circuit that computes the i-th round polynomial $t_i(\boldsymbol{X}, \boldsymbol{Y})$ over all finite fields of characteristic χ. Let T_i and S_i be the time and space required to evaluate $\mathcal{C}_i$ respectively.[9] Then, there exists an arithmetic circuit $\mathcal{C}_i^*$ which output $t_i(\boldsymbol{X}, \boldsymbol{Y})$, such that every gate of $\mathcal{C}_i^*$ outputs a bilinear polynomial in $\boldsymbol{X}$ and $\boldsymbol{Y}$, and the outputs of $\mathcal{C}_i^*$ can be computed in $O(T_i)$ time and $O(S_i)$ space.*

Proof. The argument proceeds in two steps. Firstly, we show that for *every output gate* of $\mathcal{C}_i$, the polynomial output by the gate is bilinear in $\boldsymbol{X}$ and $\boldsymbol{Y}$. This is the step where we use the extension-agnostic property of the prover. Secondly, we show how transform $\mathcal{C}_i$ into a circuit $\mathcal{C}_i^*$ such that *every internal gate* of $\mathcal{C}_i^*$ outputs a bilinear polynomial in $\boldsymbol{X}$ and $\boldsymbol{Y}$, and the time and space complexity of computing $\mathcal{C}_i^*$ is only a constant factor larger than that of $\mathcal{C}_i$.

Handling output gates. Let $\mathbb{F}_\alpha$ and $\mathbb{F}_\beta$ be two distinct finite fields of characteristic χ. Consider an output gate of $\mathcal{C}_i$ that outputs $f_\alpha(\boldsymbol{X}, \boldsymbol{Y}) \in \mathbb{F}_\alpha[\boldsymbol{X}, \boldsymbol{Y}]$ when equipped with field $\mathbb{F}_\alpha$, and outputs $f_\beta(\boldsymbol{X}, \boldsymbol{Y}) \in \mathbb{F}_\beta[\boldsymbol{X}, \boldsymbol{Y}]$ when equipped with field $\mathbb{F}_\beta$. The degree of f_α in $\boldsymbol{X}$ must be at most 1. To see this, suppose the degree is $d > 1$. Then d must be equivalent to 1 modulo $|\mathbb{F}_\alpha^\times| = |\mathbb{F}_\alpha| - 1$ and also modulo $|\mathbb{F}_\beta^\times| = |\mathbb{F}_\beta| - 1$. This implies $d - 1$ is a multiple of both $|\mathbb{F}_\alpha| - 1$ and $|\mathbb{F}_\beta| - 1$. Since we can choose arbitrarily large extension degrees, this is only possible if $d = 1$. Similarly, we can show that f_α and f_β are also linear in $\boldsymbol{Y}$.

Handling internal gates. Given that the output polynomials of $\mathcal{C}_i$ must be bilinear in $\boldsymbol{X}$ and $\boldsymbol{Y}$, we obtain a "homogenized" $\mathcal{C}_i^*$ from $\mathcal{C}_i$ by splitting every gate into 4 gates: the first gate only outputs constant polynomials, the second gate outputs polynomials with only degree-1 terms in $\boldsymbol{X}$, the third gate outputs polynomials with only degree-1 terms in $\boldsymbol{Y}$, and the fourth gate outputs polynomials with terms that have degree-1 in both $\boldsymbol{X}$ and $\boldsymbol{Y}$.

- Every constant gate with value $c \in \mathbb{F}$ is replaced with $(c, 0, 0, 0)$.
- Every input gate with value X_i is replaced with $(0, X_i, 0, 0)$ and every input gate with value Y_i is replaced with $(0, 0, Y_i, 0)$.
- Every addition gate with inputs (a_1, b_1, c_1, d_1) and (a_2, b_2, c_2, d_2) outputs $(a_1 + a_2, b_1 + b_2, c_1 + c_2, d_1 + d_2)$.
- Every multiplication gate with inputs (a_1, b_1, c_1, d_1) and (a_2, b_2, c_2, d_2) outputs $(a_1 \cdot a_2, a_1 \cdot b_2 + a_2 \cdot b_1, a_1 \cdot c_2 + a_2 \cdot c_1, b_1 \cdot c_2 + b_2 \cdot c_1 + a_1 \cdot d_2 + a_2 \cdot d_1)$. The other terms such as $b_1 \cdot b_2$ produced by the multiplication gate can be ignored since they do not contribute to any of the output polynomials of $\mathcal{C}_i$.

Clearly, every gate of $\mathcal{C}_i^*$ outputs a bilinear polynomial in $\boldsymbol{X}$ and $\boldsymbol{Y}$. Furthermore, given that the transformation blows up the size of the circuit by a constant factor with only local changes, the time and space complexity of computing $\mathcal{C}_i^*$ increases by a constant factor as well. □

Using this background we can now formally state the main lower bound theorem.

Theorem 6.5. *Consider an honest, non-adaptive, extension-agnostic prover $\mathcal{P}$ of the classical sumcheck protocol for $\mathsf{SC}^{(\mathbb{F},n,2)}$, which has access to at most $O(N^{1/2-\varepsilon})$ space for some constant $\varepsilon > 0$. If either*

- *Conjecture 6.2 holds, or*
- *Conjecture 2.7 holds and $\mathcal{P}$ reads $O(N^{1-\varepsilon})$ values of p and q in any round where it performs $\leq N/4$ field operations,*

then $\mathcal{P}$ requires $\Omega(N(\log\log N + \log\varepsilon)/\log(1 + 1/\omega))$[10] field operations in total.

[9] The time and space complexity of evaluating an arithmetic circuit are formalized via pebbling games [Coo73]. We omit the details here since the transformation in this lemma only modifies the local structure on a gate-by-gate basis, and therefore the time and space complexity of computing the given arithmetic circuits is preserved.

[10] ω is the constant from Conjecture 2.7 or Conjecture 6.2.

6.2 Proving lower bound for two rounds that are far apart

Lemma 6.6. *Consider an honest, non-adaptive, extension-agnostic prover of the product-sumcheck protocol over a domain of size $N := 2^n$, which has access to at most $O(N^{1/2-\varepsilon})$ space for some constant $\varepsilon > 0$. Let Alice be the prover of the product-sumcheck protocol in round $\mathsf{m}_{\mathcal{A}}$ such that $\log\log N \leq \mathsf{m}_{\mathcal{A}} \leq \varepsilon\omega \log N/2$, and let Bob be the prover in round $\mathsf{m}_{\mathcal{B}} := \lceil 2(\omega+1)\mathsf{m}_{\mathcal{A}}/\omega + 1\rceil$. Let the message sent by Alice to Bob be $\Pi_{\mathcal{A}}(p,q)$ and the set of indices read by Bob be $R_{\mathcal{B}}(p,q)$. Given inputs $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$, and random challenges $r_1,\ldots,r_{\mathsf{m}_{\mathcal{B}}-1} \in \mathbb{F}^{\mathsf{m}_{\mathcal{B}}-1}$, Bob outputs the polynomial*

$$f(X) := \sum_{\boldsymbol{b}\in\{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p(r_1,\ldots,r_{\mathsf{m}_{\mathcal{B}}-1},X,\boldsymbol{b})\cdot q(r_1,\ldots,r_{\mathsf{m}_{\mathcal{B}}-1},X,\boldsymbol{b}) \ .$$

If Bob performs at most $N/4$ field operations, and additionally, (i) Conjecture 2.7 holds and Bob reads at most $N^{1-\varepsilon}/4$ values of p and q, or (ii) Conjecture 6.2 holds, then Alice requires $\Omega(N\log^{\omega/2} N)$ field operations to output $\Pi_{\mathcal{A}}(p,q)$.

This lemma is similar to Lemma 5.4 in the multilinear-sumcheck lower bound proof, since we consider two rounds of the protocol $\mathsf{m}_{\mathcal{A}}$ and $\mathsf{m}_{\mathcal{B}}$ that are far apart. The pairs of rounds that we consider, $(\mathsf{m}_{\mathcal{A}},\mathsf{m}_{\mathcal{B}})$, grow geometrically since $\mathsf{m}_{\mathcal{B}} \approx (2(\omega+1)/\omega)\mathsf{m}_{\mathcal{A}}$, as opposed to the linear growth in Lemma 5.4. We restrict the number of field operations that Bob performs to $N/4$ so that it cannot simply compute the round polynomial $f(X)$ without using $\Pi_{\mathcal{A}}(p,q)$ at all. This restriction allows us to prove that $\Pi_{\mathcal{A}}(p,q)$ on its own is sufficient to solve an instance of BHPM, and therefore Alice must have performed a large number of field operations to compute $\Pi_{\mathcal{A}}(p,q)$. If additionally Bob reads at most $N^{1-\varepsilon}/4$ values of p and q, then we can similarly argue that $\Pi_{\mathcal{A}}(p,q)$ is sufficient to solve an instance of BPM.

Consider a fixed random challenge vector $\boldsymbol{t} \in \{0,1,z\}^{\mathsf{m}_{\mathcal{A}}-1}$, which consists of the first $(\mathsf{m}_{\mathcal{A}}-1)$ random challenges of the sumcheck protocol. Define multilinear polynomials $p_{\boldsymbol{t}} : \{0,1\}^{n-\mathsf{m}_{\mathcal{A}}+1} \to \mathbb{F}$ and $q_{\boldsymbol{t}} : \{0,1\}^{n-\mathsf{m}_{\mathcal{A}}+1} \to \mathbb{F}$ as follows, where $\boldsymbol{t}$ fixes the first $\mathsf{m}_{\mathcal{A}}-1$ variables of p and q:

$$p_{\boldsymbol{t}}(\boldsymbol{x}) := \sum_{\boldsymbol{b}\in\{0,1\}^{\mathsf{m}_{\mathcal{A}}-1}} \chi_{\boldsymbol{b}}(\boldsymbol{t})\cdot p(\boldsymbol{b},\boldsymbol{x}) \qquad \text{and} \qquad q_{\boldsymbol{t}}(\boldsymbol{x}) := \sum_{\boldsymbol{b}\in\{0,1\}^{\mathsf{m}_{\mathcal{A}}-1}} \chi_{\boldsymbol{b}}(\boldsymbol{t})\cdot q(\boldsymbol{b},\boldsymbol{x})$$

Define $\mathsf{m} := \mathsf{m}_{\mathcal{B}} - \mathsf{m}_{\mathcal{A}}$. Consider a fixed vector $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$, which includes the m random challenges that Bob knows but Alice did not have access to, and the variable in the $\mathsf{m}_{\mathcal{B}}$-th position.

Now, depending on the restriction on $|R_{\mathcal{B}}(p,q)|$, we can initialize two different pairs of polynomials derived from p and q that $R_{\mathcal{B}}(p,q)$ does not intersect with.

- **Restricted case:** $|R_{\mathcal{B}}(p,q)| \leq N^{1-\varepsilon}/4$. There exists a set of "suffixes" $W_{\mathsf{res}} \subseteq \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}$ such that $|W_{\mathsf{res}}| \geq 2^{n-\mathsf{m}_{\mathcal{B}}-1}$, and $\left|R_{\mathcal{B}}(p,q) \cap \{(\boldsymbol{x},\boldsymbol{b})\}_{\boldsymbol{x}\in\{0,1\}^{\mathsf{m}_{\mathcal{B}}},\boldsymbol{b}\in W_{\mathsf{res}}}\right| = 0$ (there are no indices read by Bob whose suffix lies in W_{res}). This follows from $N^{1-\varepsilon}/4 = 2^{n-\mathsf{m}_{\mathcal{B}}-2}\cdot 2^{\mathsf{m}_{\mathcal{B}}}\cdot 2^{-\varepsilon\log N} \leq 2^{n-\mathsf{m}_{\mathcal{B}}-2}\cdot 2^{\varepsilon\log N+1}\cdot 2^{-\varepsilon\log N} = 2^{n-\mathsf{m}_{\mathcal{B}}-1}$.
- **Unrestricted case:** $|R_{\mathcal{B}}(p,q)| \leq N/4$. Now, there exists a set of suffixes $W_{\mathsf{unres}} \subseteq \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}$ such that $|W_{\mathsf{unres}}| \geq 2^{n-\mathsf{m}_{\mathcal{B}}-1}$, and $\left|R_{\mathcal{B}}(p,q) \cap \{(\boldsymbol{x},\boldsymbol{b})\}_{\boldsymbol{x}\in\{0,1\}^{\mathsf{m}_{\mathcal{B}}},\boldsymbol{b}\in W_{\mathsf{unres}}}\right| \leq 2^{\mathsf{m}_{\mathcal{B}}-1}$ (W_{unres} consists of suffixes for which at most half the entries are read).

Accordingly, for the restricted and unrestricted cases we define the specific BPM and BHPM problem instance that we will solve using $\mathcal{B}$.

- **Restricted case:** Compute $f_{\mathsf{res}}(\boldsymbol{r}) := \sum_{\boldsymbol{b}\in W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r},\boldsymbol{b})\cdot q_{\boldsymbol{t}}(\boldsymbol{r},\boldsymbol{b})$ for all values of $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$. This is an instance of $\mathsf{BPM}(\mathsf{m}+1, 2^{n-\mathsf{m}_{\mathcal{B}}-1})$ since $|W_{\mathsf{res}}| \geq 2^{n-\mathsf{m}_{\mathcal{B}}-1}$.

- **Unrestricted case:** Compute $f_{\mathsf{unres}}(\boldsymbol{r}) := \sum_{\boldsymbol{b} \in W_{\mathsf{unres}}} p^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b})$ for all $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$ where for all $\boldsymbol{x} \in \{0,1\}^{\mathsf{m}+1}$ and $\boldsymbol{b} \in W_{\mathsf{unres}}$,

$$p^U_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b}) := \sum_{\boldsymbol{y} \in U(\boldsymbol{x}, \boldsymbol{b})} \chi_{\boldsymbol{y}}(\boldsymbol{t}) \cdot p(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{b}) \qquad \text{and} \qquad q^U_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b}) := \sum_{\boldsymbol{y} \in U(\boldsymbol{x}, \boldsymbol{b})} \chi_{\boldsymbol{y}}(\boldsymbol{t}) \cdot q(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{b})$$

  where $U(\boldsymbol{x}, \boldsymbol{b}) \subseteq \{0,1\}^{\mathsf{m}_{\mathcal{A}}-1}$ is the set of "unread prefixes" for $(\boldsymbol{x}, \boldsymbol{r})$. Since $|R_{\mathcal{B}}(p,q)| \le N/4$, $p^U_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})$ and $q^U_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})$ are 0 for at most 2^{m} values of $\boldsymbol{x}$ for each $\boldsymbol{b} \in W_{\mathsf{unres}}$ (and the remaining values are arbitrary field elements), and therefore this is an instance of $\mathsf{BHPM}(\mathsf{m}+1, 2^{n-\mathsf{m}_{\mathcal{B}}-1})$. Analogously, we also define $p^R_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})$ and $q^R_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})$ that correspond to the read values.

Proof outline of Lemma 6.6. The proof proceeds in 3 steps:

1. We first prove that $f_{\mathsf{res}}(\boldsymbol{r})$ and $f_{\mathsf{unres}}(\boldsymbol{r})$ can be computed efficiently using $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$ and $\boldsymbol{r}$, and an additional message from Alice $\Pi'_{\mathcal{A}}(p,q) \in \mathbb{F}^{3^{\mathsf{m}+1}}$ by rewinding $3^{\mathsf{m}+1}$ times (see Lemma 6.7 and Lemma 6.8).
2. We then prove that Bob can compute the same values efficiently using a single message $\Pi^*_{\mathcal{A}}(p,q) \in \mathbb{F}^{3^{\mathsf{m}+1}}$ and $\boldsymbol{r}$ without using $R_{\mathcal{B}}(p,q)$ at all (see Lemma 6.9).
3. Finally, we show that Alice can compute $f_{\mathsf{res}}(\boldsymbol{r})$ and $f_{\mathsf{unres}}(\boldsymbol{r})$ for all $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$ efficiently (see Lemma 6.10).

Lemma 6.7. *Let $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$. If $\mathcal{B}$ can compute $\sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b})$ in at most $N/4$ field operations using $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$, then there exists an algorithm $\mathcal{B}'$ that can compute $f_{\mathsf{res}}(\boldsymbol{r})$ in at most $N/2$ field operations using $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$, and an additional message $\Pi'_{\mathcal{A}}(p,q) \in \mathbb{F}^{3^{\mathsf{m}+1}}$ from Alice.*

Proof. $\mathcal{B}$ outputs the following value:

$$\begin{aligned}\sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) &= \sum_{\boldsymbol{b} \in W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) + \sum_{\boldsymbol{b} \notin W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \\ &:= \sum_{\boldsymbol{b} \in W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) + h_{\mathsf{res}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))\end{aligned}$$

where $h_{\mathsf{res}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))$ is defined such that the equality is satisfied. Since Bob's reads are non-adaptive, Alice can compute $R_{\mathcal{B}}(p,q)$, and therefore $h_{\mathsf{res}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))$ for each $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$. We allow Alice to compute $[h_{\mathsf{res}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))]_{\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}}$ for free, and send these values to Bob in an additional message $\Pi'_{\mathcal{A}}(p,q)$. Additionally, we can assume without loss of generality that Alice sends $\Pi'_{\mathcal{A}}(p,q)$ in addition to $\Pi_{\mathcal{A}}(p,q)$ because $|\Pi'_{\mathcal{A}}(p,q)| = 3^{\mathsf{m}+1} \le N^{1/2-\varepsilon}$, and therefore the total communication is still $O(N^{1/2-\varepsilon})$. Clearly, $\mathcal{B}'$ can output $\sum_{\boldsymbol{b} \in W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}}} p(\boldsymbol{r}, \boldsymbol{b}) \cdot q(\boldsymbol{r}, \boldsymbol{b}) - h_{\mathsf{res}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))$ in $N/4 + O(1) \le N/2$ field operations using $\mathcal{B}$ and $\Pi'_{\mathcal{A}}(p,q)$. □

Lemma 6.8. *Let $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$. If $\mathcal{B}$ can compute $\sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b})$ in at most $N/4$ field operations using $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$, then there exists an algorithm $\mathcal{B}'$ that can compute $f_{\mathsf{unres}}(\boldsymbol{r})$ in at most $N/2$ field operations using $\Pi_{\mathcal{A}}(p,q)$, $R_{\mathcal{B}}(p,q)$, and an additional message $\Pi'_{\mathcal{A}}(p,q) \in \mathbb{F}^{3^{\mathsf{m}+1}}$ from Alice.*

Proof. $\mathcal{B}$ outputs the following value:

$$\begin{aligned}\sum_{\boldsymbol{b} \in \{0,1\}^{n-\mathsf{m}_{\mathcal{B}}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) &= \sum_{\boldsymbol{b} \in W_{\mathsf{unres}}} \left(p^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) + p^R_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})\right) \cdot \left(q^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) + q^R_{\boldsymbol{t}}(\boldsymbol{x}, \boldsymbol{b})\right) + \sum_{\boldsymbol{b} \notin W_{\mathsf{unres}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \\ &:= \sum_{\boldsymbol{b} \in W_{\mathsf{unres}}} p^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q^U_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) + h_{\mathsf{unres}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))\end{aligned}$$

where $h_{\mathsf{unres}}(\boldsymbol{r}, \Pi_{\mathcal{A}}(p,q), R_{\mathcal{B}}(p,q))$ is defined such that the equality is satisfied. The remaining proof is similar to Lemma 6.7. □

Using $\mathcal{B}'$ from Lemma 6.7 or Lemma 6.8, we now show that exists an efficient algorithm $\mathcal{B}^*$ that computes $f_{\mathsf{res}}(\boldsymbol{r})$ and $f_{\mathsf{unres}}(\boldsymbol{r})$ using a different message $\Pi^*_{\mathcal{A}}(p,q)$ and $\boldsymbol{r}$ (without using $R_{\mathcal{B}}(p,q)$ at all). We must prove the following two claims to show this:

- If Alice requires T time to compute $\Pi_{\mathcal{A}}(p,q)$, then there exists an algorithm $\mathcal{A}^*$ that can compute $\Pi^*_{\mathcal{A}}(p,q)$ in at most $O(T+N)$ time.
- There exists an algorithm $\mathcal{B}^*$ that can compute $f_{\mathsf{res}}(\boldsymbol{r})$ and $f_{\mathsf{unres}}(\boldsymbol{r})$ for any $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$ in at most $N/2$ field operations using $\Pi^*_{\mathcal{A}}(p,q)$.

We prove these claims by constructing algorithms $\mathcal{A}^*$ and $\mathcal{B}^*$ from $\mathcal{A}$ and $\mathcal{B}'$ respectively:

Lemma 6.9. *Suppose Alice computes $\Pi_{\mathcal{A}}(p,q)$ in T time, and $\mathcal{B}'$ computes $f_{\mathsf{res}}(\boldsymbol{r})$ for any $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$ using $\Pi_{\mathcal{A}}(p,q)$. Then there exist a message $\Pi^*_{\mathcal{A}}(p,q)$ such that $|\Pi^*_{\mathcal{A}}(p,q)| = |\Pi_{\mathcal{A}}(p,q)|$ that can be computed in T time such that there exists an algorithm $\mathcal{B}^*$ that requires at most $N/2$ field operations to output $f_{\mathsf{res}}(\boldsymbol{r})$ using $\Pi^*_{\mathcal{A}}(p,q)$.*

Proof. Consider each step (field operation) of the algorithm $\mathcal{A}$. We categorize the elements that $\mathcal{A}$ works with into two types: elements that are read by Bob (represented by $\perp$), and any intermediate field element stored by Alice (represented by $x, y \in \mathbb{F}$). We exhaustively list all possible types of operations with respect to these two types, and show what operation $\mathcal{A}^*$ must perform instead to output $\Pi^*_{\mathcal{A}}(p,q)$.

- $\mathcal{A}$ outputs $x \star y$: $\mathcal{A}^*$ outputs $x \star y$.
- $\mathcal{A}$ outputs $x \star \perp$: $\mathcal{A}^*$ outputs x.
- $\mathcal{A}$ outputs $\perp \star \perp$: $\mathcal{A}^*$ outputs $\perp$.

Clearly, if $\mathcal{A}$ requires T time, so does $\mathcal{A}^*$. We similarly define $\mathcal{B}^*$ relative to $\mathcal{B}$. The final output of $\mathcal{B}^*$ is not a function of $R_{\mathcal{B}}(p,q)$ at all, and therefore replacing any operation that $\mathcal{A}$ and $\mathcal{B}$ perform that uses an element of $R_{\mathcal{B}}(p,q)$ can be replaced by a no-op to output the same value. □

A similar statement can be proven for the unrestricted case, where $\mathcal{B}^*$ computes $f_{\mathsf{unres}}(\boldsymbol{r})$ instead of $f_{\mathsf{res}}(\boldsymbol{r})$. Lemma 6.9 showed that it requires T time to execute $\mathcal{A}^*$ and output $\Pi^*_{\mathcal{A}}(p,q)$, and therefore it can be simulated by Alice. Therefore, without loss of generality, we can assume that Alice sends $\Pi^*_{\mathcal{A}}(p,q)$ to Bob.

Finally, we proceed to the final step of the proof outline of Lemma 6.6.

Lemma 6.10. *There exists an algorithm $\mathcal{A}_{\mathsf{final}}$ that can compute*

$$\boldsymbol{h}_{\mathsf{res}} := [f_{\mathsf{res}}(\boldsymbol{r})]_{\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}}}$$

in $O(T+N)$ time, where T is the time required to compute $\Pi_{\mathcal{A}}(p,q)$.

Proof. Using Lemma 6.4, we know that $\mathcal{B}$ outputs a quadratic function of $\Pi_{\mathcal{A}}(p,q)$. By definition of $\mathcal{B}^*$, it must also be quadratic in $\Pi^*_{\mathcal{A}}(p,q)$, and therefore we can rewrite the output as

$$\sum_{\boldsymbol{b} \in W_{\mathsf{res}}} p_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) \cdot q_{\boldsymbol{t}}(\boldsymbol{r}, \boldsymbol{b}) = \sum_{i=0, j=0}^{|\Pi^*_{\mathcal{A}}(p,q)|} c_{i,j}(\boldsymbol{r}) \cdot \Pi^*_{\mathcal{A}}(p,q)_i \cdot \Pi^*_{\mathcal{A}}(p,q)_j \tag{11}$$

where $\Pi^*_{\mathcal{A}}(p,q)_0 = 1$, and $c_{i,j}(\cdot)$ are polynomials in $\boldsymbol{r}$. We now define the following algorithm $\mathcal{A}_{\mathsf{final}}$:

1. Compute $\Pi^*_{\mathcal{A}}(p,q)$ in T time using Lemma 6.9.
2. All $c_{i,j}(\boldsymbol{r})$ are only a function of $\boldsymbol{r}$, and not of p or q. Therefore, for all $\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}$, $c_{i,j}(\boldsymbol{r})$ are simply constants, and can be precomputed irrespective of the inputs, and stored in a matrix $M \in \mathbb{F}^{3^{\mathsf{m}+1} \times (|\Pi_{\mathcal{A}}(p,q)|+1)^2}$. Since $\mathsf{m}+1 \leq \left(\frac{2}{\omega}\right)\mathsf{m}_{\mathcal{A}} \leq \varepsilon \log N$, we have $3^{\mathsf{m}+1} \leq 3^{\varepsilon \log N} \leq N^{2\varepsilon}$. Therefore there are at most $O(N^{1/2-\varepsilon} \cdot N^{1/2-\varepsilon} \cdot N^{2\varepsilon}) = O(N)$ elements in M.
3. Compute $\boldsymbol{h}_{\mathsf{res}} = M \times \Pi^*_{\mathcal{A}}(p,q)$ in $O(N)$ time.

□

A similar statement can be proven for the unrestricted case, where $\mathcal{A}_{\mathsf{final}}$ computes $\boldsymbol{h}_{\mathsf{unres}} := [f_{\mathsf{unres}}(\boldsymbol{r})]_{\boldsymbol{r} \in \{0,1,z\}^{\mathsf{m}+1}}$. We can now finally prove our main technical lemma:

Proof of Lemma 6.6. Since $\mathcal{A}_{\mathsf{final}}$ only requires $O(T+N)$ time to compute $\boldsymbol{h}_{\mathsf{res}}$ (or $\boldsymbol{h}_{\mathsf{unres}}$) according to Lemma 6.10, and only requires access to p, q, we have shown that $\mathsf{BPM}(\mathsf{m}+1, 2^{n-\mathsf{m}_{\mathcal{B}}-1})$ (or $\mathsf{BHPM}(\mathsf{m}+1, 2^{n-\mathsf{m}_{\mathcal{B}}-1})$) can be solved in $O(T+N)$ time. By Conjecture 2.7 (or Conjecture 6.2), this requires $\Omega(2^{(\omega+1)\mathsf{m}} \cdot 2^{n-\mathsf{m}_{\mathcal{B}}}) = \Omega(N \cdot 2^{\omega \mathsf{m}_{\mathcal{B}} - (\omega+1)\mathsf{m}_{\mathcal{A}}}) \geq \Omega(N \cdot 2^{\omega \mathsf{m}_{\mathcal{B}}/2}) = \Omega(N \log^{\omega/2} N)$ time to compute, and therefore $T = \Omega(N \log^{\omega/2} N)$ as desired. □

6.3 Proof of Theorem 6.5

Proof. Define a function that maps integers to round numbers of the sumcheck protocol: $f(q) := \lceil 2(\omega+1)q/\omega + 1 \rceil$. Let $m_i := f^i(\log \log N)$ and k be the number of rounds for which we will apply Lemma 6.6. We use Lemma 6.6 for all values of $\mathsf{m}_{\mathcal{A}} \in \{m_0, \ldots, m_{\mathsf{k}-1}\}$. We have

$$m_{\mathsf{k}-1} = f^{\mathsf{k}-1}(\log \log N) \leq (\log \log N + 1) \cdot \left(\frac{2(\omega+1)}{\omega}\right)^{\mathsf{k}-1} .$$

We need $m_{\mathsf{k}-1} \leq \varepsilon\omega \log N/2$ as stated in Lemma 6.6. If we set $\mathsf{k} = \lfloor (\log \varepsilon + \log \log N/2)/\log(2(\omega+1)/\omega) - 1 \rfloor$, then this condition is satisfied. If $\mathcal{P}$ requires less than $N/4$ field operations in round m_i for any $i \in [\mathsf{k}]$ to output $f_i(X)$, then $\mathcal{P}$ must use $\Omega(N \log^{\omega/2} N)$ time in round m_{i-1} by Lemma 6.6, in which case we are done since $N \log^{\omega/2} N \geq N \log \log N$ for large enough values of N. Otherwise, $\mathcal{P}$ requires at least $N/4$ field operations in round m_i for all $i \in [\mathsf{k}]$, and therefore the total number of field operations performed by $\mathcal{P}$ across these rounds is at least $N/4 \cdot \mathsf{k} = \Omega(N(\log \log N + \log \varepsilon)/\log(1+1/\omega))$ as desired. □

7 Product-sumcheck over FFT-friendly fields

In this section, we describe a new sumcheck protocol for $\mathsf{SC}^{(\mathbb{F},n,2)}$ over FFT-friendly finite fields. We begin by introducing the appropriate definitions and notation used in this section.

7.1 Background and setup

Let $\mathbb{F}$ be an FFT-friendly finite field and let $\mathbb{F}^\times$ be its multiplicative subgroup. We use $\log^* n$ to denote the base 2 iterated logarithm of n, which is equal to the number of times we must take the logarithm base 2 of a given number to obtain a value less than or equal to 1 (and therefore is an integer). We continue to use the notation from Section 4 and assume that the prover uses $O(N^{1/k})$ space, where $N := 2^n$ and $\ell := n/k$.[11]

Power towers of 2. We define the power towers of 2 as follows:

$$T(i) := \begin{cases} 2 & \text{if } i = 1 \\ 2^{T(i-1)} & \text{if } i > 1 \end{cases}.$$

Therefore, $T(i)$ is a tower of i many 2s, and in particular, $T(\log^* x) \geq x$.

Subgroups of desired sizes. Let $s = \log^* \ell$. Consider subgroups $\mathbb{G}_1 < \mathbb{G}_2 < \ldots < \mathbb{G}_{s-1} < \mathbb{G}_s \leq \mathbb{G}_{s+1} = \ldots = \mathbb{G}_{s+k-1} < \mathbb{F}^\times$, where $|\mathbb{G}_i| = T(i)$ for $i \in [s-1]$, $|\mathbb{G}_s| = 2^\ell / \left(\prod_{i=1}^{s-1} T(i)\right)$, and $|\mathbb{G}_{s+i}| = 2^\ell$ for all $i \in [k-1]$. These subgroups exist if $\mathbb{F}$ is sufficiently smooth. We also define $G := \mathbb{G}_1 \times \mathbb{G}_2 \times \ldots \times \mathbb{G}_{s+k-1}$, and $G_j := \mathbb{G}_1 \times \ldots \times \mathbb{G}_j$ for $j \in [s+k-1]$.

Univariate Lagrange polynomials. Consider the subgroup $\mathbb{G}_j < \mathbb{F}^\times$. We define the g-th univariate Lagrange polynomial $\mathsf{L}_{j,g}(X)$ for group $\mathbb{G}_j$ as

$$\mathsf{L}_{j,g}(X) = \prod_{e \in \mathbb{G}_j \setminus \{g\}} \frac{X - e}{g - e} = \frac{g \cdot (X^{|\mathbb{G}_j|} - 1)}{|\mathbb{G}_j| \cdot (X - g)}$$

Clearly, the above polynomial is equal to 1 for $X = g$ and is equal to 0 for each $X \in \mathbb{G}_j \setminus \{g\}$. For any $r \in \mathbb{F} \setminus \mathbb{G}_j$, $\mathsf{L}_{j,g}(r)$ can be computed in $O(\log |\mathbb{G}_j|)$ field operations. Furthermore, for a fixed $r \in \mathbb{F} \setminus \mathbb{G}_j$, the vector of values $[\mathsf{L}_{j,g}(r)]_{g \in \mathbb{G}_j}$ can be computed with a total of $O(|\mathbb{G}_j|)$ field operations as follows:

1. First compute the value $r^{|\mathbb{G}_j|} - 1$ in $O(\log |\mathbb{G}_j|)$ field operations. Store the value $x \leftarrow \omega_j/(r - \omega_j)$ and $c \leftarrow (r^{|\mathbb{G}_j|} - 1)/|\mathbb{G}_j|$ where ω_j is the generator of $\mathbb{G}_j$.
2. For $i \in [|\mathbb{G}_j|]$: return $c \cdot x$ and update $x \leftarrow x \cdot \omega_j \cdot (r - \omega_j^{i+1})/(r - \omega_j^i)$.

Vanishing polynomials. We also define the j-th vanishing polynomial $\mathsf{V}_j \in \mathbb{F}^{\leq |\mathbb{G}_j|}[X]$ for subgroup $\mathbb{G}_j$ as the unique polynomial with degree at most $|\mathbb{G}_j|$ such that $\mathsf{V}_j(g) = 0$ for all $g \in \mathbb{G}_j$. In particular, we have $\mathsf{V}_j(X) = \prod_{g \in \mathbb{G}_j}(X - g) = X^{|\mathbb{G}_j|} - 1$, and can also be computed in $O(\log |\mathbb{G}_j|)$ field operations.

Binary representation. Let $\mathbb{G}_j$ be a subgroup with generator ω_j. We denote the corresponding binary representation of ω_j^i, the i-th element of $\mathbb{G}_j$, as $\phi_j(\omega_j^i) := \langle i \rangle$.

Mixed polynomials. Given a multilinear polynomial $p : \mathbb{F}^n \to \mathbb{F}$, we define a related multivariate polynomial $\tilde{p} : \mathbb{F}^{s+k-1} \to \mathbb{F}$ as follows.

$$\forall \boldsymbol{x} \in G : \tilde{p}(\boldsymbol{x}) = \sum_{\boldsymbol{g} \in G} \left(\prod_{j=1}^{s+k-1} \mathsf{L}_{j,g_j}(x_j) \right) \cdot p(\phi_1(g_1), \ldots, \phi_{s+k-1}(g_{s+k-1})) \tag{12}$$

[11] We assume ℓ is an integer for simplicity.

We call this a *mixed polynomial* because the variables in this polynomials have different degrees. Intuitively, we partition the input variables into sets such that the i-th set of variables has size $\log |\mathbb{G}_i|$ and therefore can be replaced with a single variable of degree $|\mathbb{G}_i| - 1$. This replacement is done for the first ℓ variables, and the remaining $n - \ell$ variables are grouped into sets of size ℓ and replaced with single variables of degree $|\mathbb{G}_i| - 1$. In particular, the mixed polynomial in Eq. (12) is the unique polynomial with the following properties:

- has degree at most $|\mathbb{G}_i| - 1$ in the i-th variable for each $i \in [s]$,
- has degree at most $|\mathbb{G}_s| - 1$ in the last $k - 1$ variables, and
- for all $\boldsymbol{x} \in G$ satisfies

$$\tilde{p}(\boldsymbol{x}) = p(\phi_1(x_1), \ldots, \phi_{s+k-1}(x_{s+k-1}))$$

It is clear that $\tilde{p}$ satisfies these properties by definition of Lagrange polynomials. We similarly define the mixed polynomial $\tilde{q}$ for q. The sumcheck claim $\sigma = \sum_{\boldsymbol{b} \in \{0,1\}^n} p(\boldsymbol{b}) \cdot q(\boldsymbol{b})$ is therefore equivalent to $\sigma = \sum_{\boldsymbol{x} \in G} \tilde{p}(\boldsymbol{x}) \cdot \tilde{q}(\boldsymbol{x})$. This new sumcheck claim motivates the following definition:

Definition 7.1 (Mixed polynomial sumcheck language)**.** *Let $\tilde{p}_1(\boldsymbol{X}), \tilde{p}_2(\boldsymbol{X}), \ldots, \tilde{p}_d(\boldsymbol{X})$ be $(s + k - 1)$-variate polynomials over a finite field $\mathbb{F}$ such that $\deg_{X_i}(\tilde{p}_j) < |\mathbb{G}_i|$ for all $i \in [s]$ and and $\deg_{X_i}(\tilde{p}_j) < |\mathbb{G}_s|$ for $i \in \{s + 1, \ldots, s + k - 1\}$, and let $\sigma \in \mathbb{F}$. The* mixed polynomial sumcheck language $\mathsf{MixedSC}^{(\mathbb{F},n,d)}$ *consists of tuples of the form $(\tilde{p}_1(\boldsymbol{X}), \tilde{p}_2(\boldsymbol{X}), \ldots, \tilde{p}_d(\boldsymbol{X}), \sigma)$ where $\sigma = \sum_{\boldsymbol{x} \in G} \prod_{j=1}^{d} \tilde{p}_j(\boldsymbol{x})$.*

We can now state the main theorems of this section.

Theorem 7.2. *Let $k \geq 1$ and let $\mathbb{F}$ be an FFT-friendly finite field. There exist PIOPs for $\mathsf{MixedSC}^{(\mathbb{F},n,2)}$ with the following properties:*

prover time	prover space	rounds	communication	verifier time	oracle size	oracles/queries
$O(N(\log^* N + k))$	$O(N^{1/k})$	$O(\log^* N + k)$	$O(\log^* N + k)$	$O(\log N + k)$	$O(kN^{1/k})$	$O(\log^* N + k)$
$O(N(\log^* N + k))$	$O(N^{1/k})$	$O(\log^* N + k)$	$O(\log N + k)$	$O(\log N + k)$	$O(kN^{1/k})$	$O(k)$

Note that we get a *family of PIOPs*: one for each integer $k \geq 1$. If we are satisfied with $O(N)$ prover space, then we can modify the folding-based sumcheck protocol of [CTY11; Tha13] to obtain linear prover time with $O(\log^* N)$ rounds to obtain the following two additional trade-offs, which we describe in detail in Section 7.5.

Theorem 7.3. *There exist PIOPs for $\mathsf{MixedSC}^{(\mathbb{F},n,2)}$ where $\mathbb{F}$ is FFT-friendly with the following properties:*

prover time	prover space	rounds	communication	verifier time	oracle size	oracles/queries
$O(N)$	$O(N)$	$O(\log^* N)$	$O(\log^* N)$	$O(\log N)$	$O(\sqrt{N})$	$O(\log^* N)$
$O(N)$	$O(N)$	$O(\log^* N)$	$O(\log N)$	$O(\log N)$	$O(\sqrt{N})$	$O(1)$

7.2 Mixed polynomial sumcheck protocol

The definition of the mixed polynomials $\tilde{p}$ and $\tilde{q}$ in Eq. (12) partially gives away the intuition behind our protocol. In the classical sumcheck protocol for multilinear polynomials, the prover $\mathcal{P}$ sends a degree-2 polynomial t_j to the verifier $\mathcal{V}$ in round j, and $\mathcal{V}$ sends back a random challenge $r_j \in \mathbb{F}$. Additionally, $\mathcal{V}$ checks if

$$t_j(0) + t_j(1) = t_{j-1}(r_{j-1}) \ . \tag{13}$$

Instead, in the j-th round of our protocol, $\mathcal{P}$ sends a degree $2|\mathbb{G}_j| - 2$ univariate polynomial t_j to $\mathcal{V}$, and $\mathcal{V}$ sends $r_j \xleftarrow{\$} \mathbb{F}$. This round polynomial t_j is defined as follows:

$$t_j(X) = \sum_{\substack{g_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ g_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, X, g_{j+1}, \ldots, g_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, X, g_{j+1}, \ldots, g_{s+k-1}) \quad (14)$$

The verifier performs the following check:

$$\sum_{g \in \mathbb{G}_j} t_j(g) = t_{j-1}(r_{j-1}) \ . \quad (15)$$

The verifier can perform this check directly by performing $O(|\mathbb{G}_j|)$ field operations if the prover sends t_j to $\mathcal{V}$. Alternatively, the $\mathcal{P}$ can send a univariate polynomial oracle $[\![t_j]\!]$ and the verifier can check the claim in Eq. (15) via a univariate sumcheck. For simplicity, we describe our protocol to perform univariate sumcheck in every round.

Modified protocol. Using these ideas, the new protocol can be succinctly described as follows:

$\langle \mathcal{P}(\mathcal{S}(\tilde{p}), \mathcal{S}(\tilde{q}), \sigma); \mathcal{V}([\![\tilde{p}]\!], [\![\tilde{q}]\!], \sigma) \rangle$:

1. For each round $j \in [s + k - 1]$:
2. $\quad \mathcal{P}$ sends $[\![t_j]\!]$ (as defined in Eq. (14)) to $\mathcal{V}$.
3. $\quad$ If $j = 1$: $\mathcal{V}$ sets $\sigma_j \leftarrow \sigma$.
4. $\quad$ Else: $\mathcal{V}$ queries $[\![t_{j-1}]\!]$ at r_{j-1} and sets $\sigma_j \leftarrow t_{j-1}(r_{j-1})$.
5. $\quad \mathcal{P}$ and $\mathcal{V}$ engage in a univariate sumcheck protocol to check $\sum_{g_j \in \mathbb{G}_j} t_j(g_j) = \sigma_j$.
   (a) $\mathcal{P}$ computes the unique polynomials $u_j \in \mathbb{F}^{<|\mathbb{G}_j|-1}[X]$, $v_j \in \mathbb{F}^{<|\mathbb{G}_j|}[X]$ such that $t_j(X) = (X \cdot u_j(X) + \frac{\sigma_j}{|\mathbb{G}_j|}) + \mathsf{V}_j(X) \cdot v_j(X)$.
   (b) $\mathcal{P}$ sends $[\![u_j]\!], [\![v_j]\!]$ to $\mathcal{V}$.
   (c) $\mathcal{V}$ samples a random challenge $z_j \xleftarrow{\$} \mathbb{F}$ and queries $[\![t_j]\!], [\![u_j]\!], [\![v_j]\!]$ at z_j.
   (d) $\mathcal{V}$ checks if $t(z_j) = (z_j \cdot u_j(z_j) + \frac{\sigma_j}{|\mathbb{G}_j|}) + \mathsf{V}_j(z_j) \cdot v_j(z_j)$.
6. $\quad \mathcal{V}$ samples a random challenge $r_j \xleftarrow{\$} \mathbb{F}$.
7. $\quad$ If $j < s + k - 1$: $\mathcal{V}$ sends r_j to $\mathcal{P}$.
8. $\quad$ Else: $\mathcal{V}$ checks if $\tilde{p}(r_1, \ldots, r_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{s+k-1}) = \sigma_{s+k-1}$.

Figure 4: The mixed polynomial sumcheck protocol for the claim "$\sum_{\boldsymbol{x} \in G} \tilde{p}(\boldsymbol{x}) \cdot \tilde{q}(\boldsymbol{x}) = \sigma$".

The specific univariate sumcheck protocol that we engage in is the one described in Aurora [BCRSVW19], which itself is a PIOP that only requires $O(1)$ polynomial oracles, queries, and rounds.

Efficiency parameters. Before describing a time and space efficient implementation of the prover $\mathcal{P}$ in Section 7.3, we first analyze other efficiency parameters of Fig. 4.

- Rounds: Clearly the number of rounds is $O(s + k) = O(\log^* N + k)$.
- Communication: The only non-oracle communication consists of the verifier random challenges, which is therefore $O(s + k) = O(\log^* N + k)$.

- Verifier time: Outside of the polynomial oracle queries, the only non-trivial work for the verifier is computing $\mathsf{V}_j(r_j)$, which can be done in $O(\log|\mathbb{G}_j|)$ field operations. The total work is therefore $O\Big(s+k+\sum_{j=1}^{s+k-1}\log|\mathbb{G}_j|\Big) = O\Big(\log^* N + k + \log\Big(\prod_{j=1}^{s+k-1}|\mathbb{G}_j|\Big)\Big) = O(\log N + k)$.
- Number of polynomial oracles: In each round $\mathcal{P}$ sends $O(1)$ polynomial oracles, and therefore the total number of polynomial oracles is $O(\log^* N + k)$.
- Number of queries: In each round, $\mathcal{V}$ makes $O(1)$ oracle queries, and therefore the total number of queries is $O(\log^* N + k)$.

Remark 7.4 (Decreasing number of polynomial oracles and queries)**.** Alternatively, in rounds $j \in [s-1]$, $\mathcal{P}$ can send the polynomials t_j in the clear to $\mathcal{V}$ instead of sending the polynomial oracles $[\![t_j]\!], [\![u_j]\!], [\![v_j]\!]$. $\mathcal{V}$ can then perform the check of Eq. (15) without invoking the univariate sumcheck protocol. Clearly, the number of polynomial oracles and the number of queries made by $\mathcal{V}$ decreases to $O(k)$. The total communication, however, increases to $O\Big(\log^* N + k + \sum_{j=1}^{s-1}|\mathbb{G}_j|\Big) = O(\log N + k)$.

7.3 Efficient implementation of the prover

We describe $\mathcal{P}$'s algorithm for the PIOP defined in Fig. 4 such that it is time and space efficient.

$\mathsf{MixedSC}_k^{\tilde{p},\tilde{q}}$:

1. Define $\ell := n/k$ and $s := \log^* \ell$ and $m := k-1$.
2. Define subgroups $\mathbb{G}_1, \mathbb{G}_2, \ldots, \mathbb{G}_{s+m} \subseteq \mathbb{F}^\times$ as defined in Section 7.1.
3. For each $j \in [s+m]$:
   (a) Define $G_{j-1} := \mathbb{G}_1 \times \ldots \times \mathbb{G}_{j-1}$ and $\boldsymbol{r}_{j-1} := (r_1, \ldots, r_{j-1}) \in \mathbb{F}^{j-1}$.
   (b) If $j-1 \le s$: For each $\boldsymbol{g} = (g_1, \ldots, g_{j-1}) \in G_j$:
   Compute and store $\mathsf{LagProd}_{j-1}[\boldsymbol{g}] \leftarrow \prod_{i=1}^{j-1} \mathsf{L}_{i,g_i}(r_i)$.
   (c) Define the $(2|\mathbb{G}_j|-2)$-degree polynomial t_j:

$$t_j(X) := \sum_{\substack{g_{j+1}\in\mathbb{G}_{j+1} \\ \vdots \\ g_{s+m}\in\mathbb{G}_{s+m}}} \tilde{p}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_{s+m}) \cdot \tilde{q}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_{s+m})$$

   (d) Compute t_j as defined above:
   i. Initialize $f_j(X) \leftarrow 0$.
   ii. For each $\boldsymbol{g} = (g_{j+1}, \ldots, g_{s+m}) \in \mathbb{G}_{j+1} \times \ldots \times \mathbb{G}_{s+m}$:
   iii. If $j-1 \le s$, compute

$$\tilde{p}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g}) = \sum_{\boldsymbol{x}\in G_{j-1}} \mathsf{LagProd}_{j-1}[\boldsymbol{x}] \cdot \tilde{p}(\boldsymbol{x}, X, \boldsymbol{g})$$

   iv. Else compute

$$\tilde{p}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g}) = \sum_{\boldsymbol{x}\in G_j} \mathsf{LagProd}_s[x_1, \ldots, x_s] \cdot \left(\prod_{i=s+1}^{j-1} \mathsf{L}_{i,g_i}(r_i)\right) \cdot \tilde{p}(\boldsymbol{x}, X, \boldsymbol{g})$$

   v. Similarly, compute $\tilde{q}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g})$.

vi. Compute $\mathsf{ProductPoly}(X) := \tilde{p}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g}) \cdot \tilde{q}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g})$ via FFT.
vii. $f_j(X) \leftarrow f_j(X) + \mathsf{ProductPoly}(X)$.
viii. $t_j(X) \leftarrow f_j(X)$.

(e) Send $[\![t_j]\!]$ to $\mathcal{V}$.

(f) Prove to $\mathcal{V}$ that $\sum_{g_j \in \mathbb{G}_j} t_j(g_j) = \sigma_j$, where $\sigma_j = \sigma$ if $j = 1$ and $\sigma_j = t_{j-1}(r_{j-1})$ otherwise, using a univariate sumcheck protocol:
i. Compute the unique polynomials $u_j \in \mathbb{F}^{<|\mathbb{G}_j|-1}$, $v_j \in \mathbb{F}^{<|\mathbb{G}_j|}[X]$ such that $t_j(X) = (X \cdot u_j(X) + \frac{\sigma_j}{|\mathbb{G}_j|}) + \mathsf{V}_j(X) \cdot v_j(X)$.
ii. Send $[\![u_j]\!], [\![v_j]\!]$ to $\mathcal{V}$.
iii. Receive random challenge z_j from $\mathcal{V}$.

(g) Receive random challenge $r_j \xleftarrow{\$} \mathbb{F}$ from $\mathcal{V}$.

Figure 5: Prover algorithm using $O(N(\log^* N + k))$ time and $O(N^{1/k})$ space for the mixed polynomial sumcheck protocol.

Completeness. The left-hand side of Eq. (15) is equal to

$$\sum_{g_j \in \mathbb{G}_j} t_j(g_j) = \sum_{\substack{g_j \in \mathbb{G}_j \\ \vdots \\ g_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(\boldsymbol{r}_j, g_j, g_{j+1}, \ldots, g_{s+k-1}) \cdot \tilde{q}(\boldsymbol{r}_j, g_j, g_{j+1}, \ldots, g_{s+k-1})$$

The above sum is equal to σ if $j = 1$, and is equal to $t_{j-1}(r_{j-1})$ otherwise as desired.

Time and space complexity. In round j, $\mathcal{P}$ initializes the polynomial t_j, and also the table $\mathsf{LagProd}_j$. We will argue the time complexity of computing and space complexity of storing these two objects.

- Space complexity:
  - $\mathsf{LagProd}$ tables: $|\mathsf{LagProd}_j| \leq |G_j|$ if $j \leq s$. Therefore $\mathsf{LagProd}_1, \ldots, \mathsf{LagProd}_s$ collectively require $O(|G_s|) = O(2^\ell) = O(N^{1/k})$ space.
  - Polynomials: t_j, f_j, u_j, v_j, and $\mathsf{ProductPoly}$ all have degree at most $(2|\mathbb{G}_j| - 2)$, and therefore can be stored in $O(|\mathbb{G}_j|) = O(2^\ell) = O(N^{1/k})$ space.
- Time complexity:
  - $\mathsf{LagProd}$ table computation: $\mathsf{LagProd}_j$ can be computed in $O(|G_j|)$ field operations if $j \leq s$ as argued in Section 7.1. Since $|G_j| \leq 2^\ell$, this can be done in $O(2^\ell) = O(N^{1/k})$ field operations. If $j > s$, then $\mathsf{LagProd}_j$ can be similarly computed in $O(|\mathbb{G}_j|) = O(2^\ell) = O(N^{1/k})$ field operations.
  - Polynomial computation: Each of the polynomials $\tilde{p}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_s)$ and $\tilde{q}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_s)$ can be computed in $O(|G_j|)$ field operations, since they are a linear combination of $O(|G_j|)$ values.[12] There are $O(|\mathbb{G}_{j+1}| \cdot |\mathbb{G}_{j+2}| \cdots |\mathbb{G}_{s+k-1}|)$ many such polynomials, and therefore all of these can be computed in $O(|G|) = O(N)$ field operations. Next, the computation of t_j requires $O(|\mathbb{G}_{j+1}| \cdot |\mathbb{G}_{j+2}| \cdots |\mathbb{G}_{s+k-1}|)$ many multiplications of univariate polynomials of degree $|\mathbb{G}_j| - 1$. This requires $O(|\mathbb{G}_j| \cdot |\mathbb{G}_{j+1}| \cdots |\mathbb{G}_{s+k-1}| \cdot \log |\mathbb{G}_j|)$ field operations if the multiplications are done via FFTs. By our

[12] If $j > s$, each value is now a product of $(j-s-1)$ many Lagrange polynomial evaluations, which naively takes $O((j-s) \cdot |G_j|) = O(k \cdot |G_j|)$ field operations. However, this can be optimized to $O(|G_j|)$ field operations because we iterate over the polynomials in a lexicographic order, so we can store the product of these $(j - s - 1)$ terms, and update this product in amortized $O(1)$ field operations as we iterate over different values of $g_{s+1}, \ldots, g_{j-1}$. We omit further details for brevity.

construction, we have $\log|\mathbb{G}_j| \le |\mathbb{G}_{j-1}|$ if $j \ge 2$ (and $\log|\mathbb{G}_j| = 1$ if $j = 1$). Therefore, the total number of field operations required to perform these FFTs is $O(|\mathbb{G}_{j-1}| \cdot |\mathbb{G}_j| \cdot |\mathbb{G}_{j+1}| \cdots |\mathbb{G}_{s+k-1}|) = O(N)$. Thus, each round polynomial only requires $O(N)$ field operations to compute[13], so the total number of field operations is $O((s+k)N) = O(N(\log^* N + k))$ as desired.

7.4 Soundness of the mixed sumcheck protocol

We recall the notion of round-by-round soundness, and show that Figure 4 satisfies this notion.

State function. Let $(\mathcal{P}, \mathcal{V})$ be a PIOP for a relation $R = \{(\mathbb{x}, \mathbb{y}, \mathbb{w})\}$. A *state function* for $(\mathcal{P}, \mathcal{V})$ is a (possibly inefficient) function State that receives as inputs $\mathbb{x}$, $\mathbb{y}$, and a transcript tr and outputs a bit, and has the following properties:

- Empty transcript: if $\mathsf{tr} = \emptyset$ is the empty transcript, then $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 1$ if and only $(\mathbb{x}, \mathbb{y}) \in L(R)$.
- The prover moves: if tr is a transcript where the prover is about to move, and $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$ then, for every prover message Π, $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel \Pi) = 0$.
- Full transcript: if tr is a full transcript and $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$, then $\mathcal{V}$ rejects given this interaction transcript.

Round-by-round soundness. A k-round PIOP $(\mathcal{P}, \mathcal{V})$ for a relation $R = \{(\mathbb{x}, \mathbb{y}, \mathbb{w})\}$ has *round-by-round soundness* with errors $(\varepsilon_1, \ldots, \varepsilon_{\mathsf{k}})$ if there exists a state function State with the following property: for every $\mathbb{x}$, $\mathbb{y}$ and transcript $\mathsf{tr} = (\Pi_1, \alpha_1, \ldots, \Pi_{i-1}, \alpha_{i-1}, \Pi_i)$,

$$\Pr_{\alpha_i}\left[\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel \alpha_i) = 1 \mid \mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0\right] \le \varepsilon_i(\mathbb{x}, \mathbb{y}) \ .$$

Lemma 7.5. *Figure 4 is a PIOP for* $R = \{(\Sigma, (\llbracket\tilde{p}\rrbracket, \llbracket\tilde{q}\rrbracket), \perp) \ : \ \sigma = \sum_{\boldsymbol{x}\in G} \tilde{p}(\boldsymbol{x}) \cdot \tilde{q}(\boldsymbol{x})\}$ *with round-by-round soundness errors* $\left(\left(\frac{2\cdot|\mathbb{G}_j|-1}{|\mathbb{F}|}, \frac{2\cdot|\mathbb{G}_j|-1}{|\mathbb{F}|}\right)_{j\in[s+k-1]}\right)$.

Proof. Across the proof we let $\mathbb{x} = \sigma$ and $\mathbb{y} = (\llbracket\tilde{p}\rrbracket, \llbracket\tilde{q}\rrbracket)$. We set $\sigma_1 = \sigma$ and $\sigma_j = t_{j-1}(r_{j-1})$ whenever defined.

1. **Initial state**: The transcript $\mathsf{tr} = \emptyset$. We set $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 1$ if and only if

$$\sigma = \sum_{\boldsymbol{x}\in G} \tilde{p}(\boldsymbol{x}) \cdot \tilde{q}(\boldsymbol{x}) \ .$$

2. **Univariate sumcheck rounds**:

   - At this stage the transcript has the form $\mathsf{tr} = ((\llbracket t_i\rrbracket, \llbracket u_i\rrbracket, \llbracket v_i\rrbracket, z_i, r_i)_{i<j}, \llbracket t_j\rrbracket, \llbracket u_j\rrbracket, \llbracket v_j\rrbracket)$. The verifier chooses $z_j \xleftarrow{\$} \mathbb{F}$. We set $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel z_j) = 1$ if and only if $\forall i \in [j]$: $t_i(z_i) = (z_i \cdot u_i(z_i) + \frac{\sigma_j}{|\mathbb{G}_j|}) + v_i(z_i) \cdot \mathsf{V}_i(z_i)$ and

$$t_j(X) = \sum_{\substack{x_{j+1}\in\mathbb{G}_{j+1} \\ \vdots \\ x_{s+k-1}\in\mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1}) \ .$$

[13] Furthermore, if the oracles to these round polynomials are instantiated using univariate polynomial commitments [KZG10; BBHR18; ACFY24], then $\mathcal{P}$ still only requires linear time assuming these commitments require $O(z \log z)$ time where z is the degree of the polynomial.

- We show that if $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$ then

$$\Pr_{z_j}[\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel z_j) = 1] \leq \frac{2|\mathbb{G}_j| - 1}{|\mathbb{F}|} \ .$$

First, since $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$ it must be that either:
– $\exists i \in [j-1]$: $t_i(z_i) = (z_i \cdot u_i(z_i) + \frac{\sigma_i}{|\mathbb{G}_i|}) + v_i(z_i) \cdot \mathsf{V}_i(z_i)$
– $\sigma_j \neq \sum_{(x_j,\ldots,x_{s+k-1}) \in (\mathbb{G}_j \times \mathbb{G}_{s+k-1})} \tilde{p}(r_1, \ldots, r_{j-1}, x_j, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, x_j, \ldots, x_{s+k-1})$.
If the first condition holds, $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel z_j) = 0$, so we assume that this is not the case, and so the second condition holds. Now suppose that the following condition holds (as it must, or else again $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel z_j) = 0$):

$$t_j(X) = \sum_{\substack{x_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ x_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1}) \ .$$

Then,

$$\sum_{x_j \in \mathbb{G}_j} t_j(x_j) = \sum_{\substack{x_j \in \mathbb{G}_j \\ \vdots \\ x_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, x_j, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, x_j, \ldots, x_{s+k-1}) \neq \sigma_j \ .$$

By the soundness of the univariate sumcheck protocol then, unless with probability at most $\frac{2 \cdot |\mathbb{G}_j|}{|\mathbb{F}|}$, the check $t_j(z_j) \neq (z_j \cdot u_j(z_j) + \frac{\sigma_j}{|\mathbb{G}_j|}) + v_j(z_j) \cdot \mathsf{V}_j(z_j)$, which concludes the proof.

3. **Multilinear sumcheck rounds**:
   - At this stage the transcript has the form $\mathsf{tr} = ((\llbracket t_i \rrbracket, \llbracket u_i \rrbracket, \llbracket v_i \rrbracket, z_i, r_i)_{i<j}, \llbracket t_j \rrbracket, \llbracket u_j \rrbracket, \llbracket v_j \rrbracket, z_j)$. The verifier chooses $r_j \xleftarrow{\$} \mathbb{F}$. We set $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel r_j) = 1$ if and only if $\forall i \in [j]$: $t_i(z_i) = (z_i \cdot u_i(z_i) + \frac{\sigma_j}{|\mathbb{G}_j|}) + v_i(z_i) \cdot \mathsf{V}_i(z_i)$ and

$$t_j(r_j) = \sum_{\substack{x_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ x_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, r_j, x_{j+1}, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, r_j, x_{j+1}, \ldots, x_{s+k-1}) \ .$$

4. We show that if $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$ then

$$\Pr_{r_j}[\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel r_j) = 1] \leq \frac{2|\mathbb{G}_j| - 1}{|\mathbb{F}|} \ .$$

First, since $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$ it must be that either:

- $\exists i \in [j]$: $t_i(z_i) = (z_i \cdot u_i(z_i) + \frac{\sigma_i}{|\mathbb{G}_i|}) + v_i(z_i) \cdot \mathsf{V}_i(z_i)$
- $t_j(X) \neq \sum_{\substack{x_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ x_{s+k-1} \in \mathbb{G}_{s+k-1}}} \tilde{p}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1}) \cdot \tilde{q}(r_1, \ldots, r_{j-1}, X, x_{j+1}, \ldots, x_{s+k-1})$.

If the first case hold, then $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel r_j) = 0$. So we assume that the latter condition holds. Then, by the polynomial identity lemma, the probability that the left and right hand side agree is bounded by $\frac{2|\mathbb{G}_j|-1}{|\mathbb{F}|}$.

□

7.5 Linear-time prover for the mixed sumcheck protocol in $O(\log^* N)$ rounds

If the sumcheck prover is not space-constrained, then we can implement the prover of the PIOP described in Fig. 4 using a folding-style sumcheck similar to [CTY11; Tha13]. This leads to a sumcheck protocol that has a linear-time prover with $O(\log^* N)$ rounds. We consider the PIOP in Fig. 4 with $k = 2$, and therefore $\ell = n/2, s = \log^*(n/2)$. At the end of the j-th round of the protocol, $\mathcal{P}$ computes and stores the folded polynomials $\tilde{p}_j, \tilde{q}_j : \mathbb{F}^{s-j+1} \to \mathbb{F}$, which are defined as follows:

$$\tilde{p}_j(\boldsymbol{X}) = \sum_{\boldsymbol{g} \in G_j} \left(\prod_{i=1}^{j} \mathsf{L}_{i,g_i}(r_i) \right) \cdot \tilde{p}(\boldsymbol{g}, \boldsymbol{X}) \ ,$$

and $\tilde{q}_j(\boldsymbol{X})$ is defined similarly. By computing and storing these folded polynomials, $\mathcal{P}$ can compute the round polynomial t_j with only $O(N/|G_{j-1}|)$ field operations, which implies that all round polynomials can be computed with a total $O(N)$ field operations. We describe the prover implementation in detail in Fig. 6.

$\mathsf{LinearTimeMixedSC}^{\tilde{p},\tilde{q}}$:

1. Define $\ell := n/2$ and $s := \log^* \ell$.
2. Define subgroups $\mathbb{G}_1, \mathbb{G}_2, \ldots, \mathbb{G}_{s+1} \subseteq \mathbb{F}^\times$ as defined in Section 7.1.
3. Define initial folded polynomials $\tilde{p}_0(\boldsymbol{X}) := \tilde{p}(\boldsymbol{X})$ and $\tilde{q}_0(\boldsymbol{X}) := \tilde{q}(\boldsymbol{X})$.
4. For each $j \in [s+1]$:
   (a) Define the $(2|\mathbb{G}_j| - 2)$-degree polynomial t_j:

$$t_j(X) := \sum_{\substack{g_{j+1} \in \mathbb{G}_{j+1} \\ \vdots \\ g_{s+1} \in \mathbb{G}_{s+1}}} \tilde{p}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_{s+1}) \cdot \tilde{q}(\boldsymbol{r}_{j-1}, X, g_{j+1}, \ldots, g_{s+1})$$

   (b) Compute t_j as defined above:
      i. Initialize $f_j(X) \leftarrow 0$.
      ii. For each $\boldsymbol{g} = (g_{j+1}, \ldots, g_{s+1}) \in \mathbb{G}_{j+1} \times \ldots \times \mathbb{G}_{s+1}$:
      iii. Compute $\tilde{p}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g})$ and $\tilde{q}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g})$ from $\tilde{p}(X, \boldsymbol{g})$ and $\tilde{q}(X, \boldsymbol{g})$ respectively.
      iv. Compute $\mathsf{ProductPoly}(X) := \tilde{p}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g}) \cdot \tilde{q}(\boldsymbol{r}_{j-1}, X, \boldsymbol{g})$ via FFT.
      v. $f_j(X) \leftarrow f_j(X) + \mathsf{ProductPoly}(X)$.
      vi. $t_j(X) \leftarrow f_j(X)$.
   (c) Send $[\![t_j]\!]$ to $\mathcal{V}$.
   (d) Prove to $\mathcal{V}$ that $\sum_{g_j \in \mathbb{G}_j} t_j(g_j) = \sigma_j$, where $\sigma_j = \sigma$ if $j = 1$ and $\sigma_j = t_{j-1}(r_{j-1})$ otherwise, using a univariate sumcheck protocol:
      i. Compute the unique polynomials $u_j, v_j \in \mathbb{F}^{<|\mathbb{G}_j|}[X]$ such that $t_j(X) = u_j(X) + \mathsf{V}_j(X) \cdot v_j(X)$, where $\deg(u_j), \deg(v_j) < |\mathbb{G}_j|$.
      ii. Send $[\![u_j]\!], [\![v_j]\!]$ to $\mathcal{V}$.
   (e) Receive random challenge $r_j \xleftarrow{\$} \mathbb{F}$ from $\mathcal{V}$.
   (f) Compute folded polynomial $\tilde{p}_j(X_{j+1}, \ldots, X_{s+1})$ from $\tilde{p}_{j-1}(X_j, \ldots, X_{s+1})$ and r_j as follows:
      i. For each $\boldsymbol{g} = (g_{j+1}, \ldots, g_{s+1}) \in \mathbb{G}_{j+1} \times \ldots \times \mathbb{G}_{s+1}$:

ii. Compute

$$\tilde{p}_j(\boldsymbol{g}) = \sum_{g_j \in \mathbb{G}_j} \tilde{p}_{j-1}(g_j, g_{j+1}, \ldots, g_{s+1}) \cdot \mathsf{L}_{j,g_j}(r_j) \ .$$

Figure 6: Prover algorithm using $O(N)$ time and $O(\log^* N)$ rounds for the mixed polynomial sumcheck protocol.

Efficiency parameters. Since the PIOP is unchanged from Fig. 4, the efficiency parameters of the protocol are the same as described in Section 7.2 (other than the prover time complexity). We now analyze the efficiency of the prover $\mathcal{P}$ in Fig. 6.

- Round polynomial computation: In round j, $\mathcal{P}$ computes t_j by performing $\prod_{i=j+1}^{s+1} |\mathbb{G}_i|$ many FFTs of size $|\mathbb{G}_j|$, which takes $O\left(\prod_{i=j+1}^{s+1} |\mathbb{G}_i| \cdot |\mathbb{G}_j| \cdot \log |\mathbb{G}_j|\right) \leq O\left(\prod_{i=j-1}^{s+1} |\mathbb{G}_i|\right)$ field operations, where the inequality follows form the construction of the subgroups $\mathbb{G}_j$ since $\log |\mathbb{G}_j| \leq |\mathbb{G}_{j-1}|$ if $j \geq 2$ (and $\log |\mathbb{G}_j| = 1$ if $j = 1$). Adding the cost over all rounds $j \in [s+1]$, this is only $O(N)$ field operations.
- Univariate sumcheck protocols: For each $j \in [s+1]$, we have $|\mathbb{G}_j| \leq O(\sqrt{N})$, and therefore the polynomial division by $\mathsf{V}_j(r_j)$ can be done in $O(\sqrt{N})$ field operations. Adding the cost over all rounds $j \in [s+1]$, this is only $O(N)$ field operations.
- Folded polynomial computation: In round j, $\mathcal{P}$ first computes the table $\mathsf{LagProd}_j$ which requires $O(|\mathbb{G}_j|)$ field operations, and then computes $\tilde{p}_j, \tilde{q}_j$ from the table $\mathsf{LagProd}_j$ and the previous folded polynomials $\tilde{p}_{j-1}, \tilde{q}_{j-1}$, which requires $O\left(\prod_{i=j}^{s+1} |\mathbb{G}_i|\right)$ field operations. Adding these costs over all rounds $j \in [s+1]$, the total number of field operations is

$$O\left(\sum_{j=1}^{s+1} |\mathbb{G}_j|\right) + O\left(\sum_{j=1}^{s+1} \prod_{i=j}^{s+1} |\mathbb{G}_i|\right) \leq O(N) \ .$$

8 Implementation and Evaluation

We evaluate the performance of ProductSC compared to LinearTimeSC of [CTY11] and LogSpaceSC of [CMT12]. We focus on two metrics: (i) prover time; and (ii) prover memory.

8.1 Implementation

We implemented the three prover algorithms in Rust, utilizing the `arkworks` ecosystem for zkSNARKs development [ark]. Our implementation is open sourced at `compsec-epfl/space-efficient-sumcheck` and we plan to upstream it to `arkworks`.

Organization. We expose a common interface for a generic prover algorithm for a classical prover of $\mathsf{SC}^{(\mathbb{F},n,2)}$, which is then implemented by the three prover algorithms. The prover interface receives as an input streams of $\boldsymbol{p}$ and $\boldsymbol{q}$ of the polynomials, which allows us to accurately measure memory consumption. These streams consist of the evaluations of the input polynomials p and q over $\{0,1\}^n$ in a lexicographic order (most-significant bit first).

Primitives. We use `arkworks` for the underlying finite field arithmetic (provided by the `ark-ff` crate).

Optimizations. While our implementation has been optimized on a best-effort basis, it should be considered a reference implementation, rather than an optimized one. Our implementation of ProductSC closely follows the description in Fig. 2, with the following optimizations:

- We switch to LinearTimeSC after $n - n/k$ rounds of the protocol. This preserves the asymptotic space complexity of ProductSC while allowing us to benefit from the slightly faster LinearTimeSC for the last few rounds.
- We use LogSpaceSC for the first 2 rounds of the protocol.

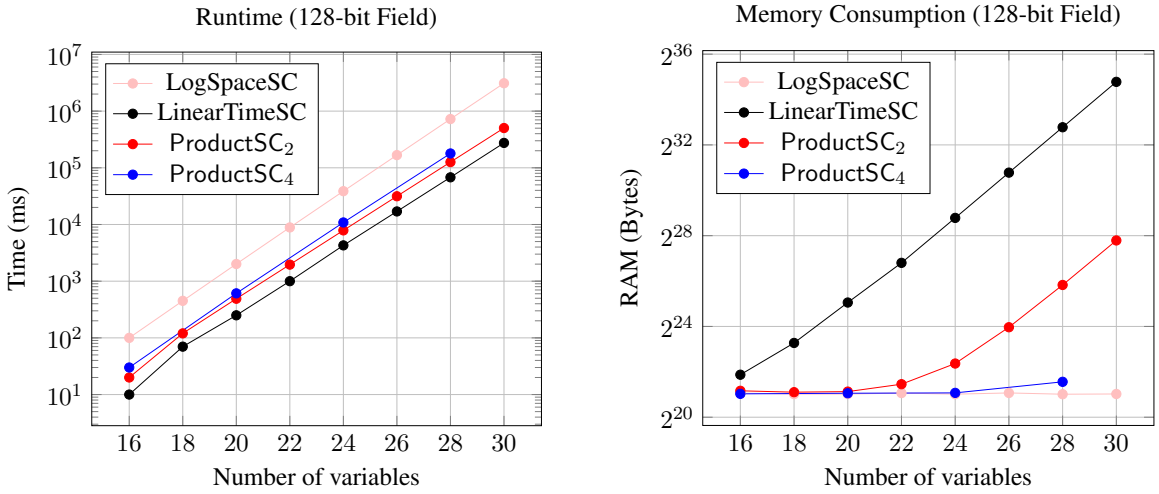
Benchmarks. We run our experiments on an AWS-hosted machine with instance type *m5.8xlarge* fitted with 32 vCPU and 128GiB of memory (Intel Xeon Platinum 8259CL CPU @ 2.50GHz). We measure (i) wall time; and (ii) maximum resident set size, using the *GNU-time* facility. We chose maximum resident set size as a proxy measure to estimate the space complexity of the algorithms we benchmark. Our methodology is as follows: we select an instance size by choosing the number of variables $n \in \{16, \ldots, 30\}$. Recall that then the instance size is $N = 2^n$. For each n, we collect both the wall time and the peak memory consumption from a single process that instantiates one prover of the chosen type. Since we observe that a Rust (1.74.1) binary requests a baseline amount of memory that is approximately 2 MiB, our results are then offset by this amount.

8.2 Results

In Figure 7, we compare running time and memory consumption across our implementations of prover algorithms. We also provide the raw data in Table 1.

- The asymptotic improvement in space of ProductSC translates in significantly lower memory consumption than LinearTimeSC across all instances that we tested. For $n = 24$, LinearTimeSC consumes 0.4 GiB of RAM and ProductSC 3.1 MiB. For $n = 28$, LinearTimeSC consumes 7.0 GiB of RAM and ProductSC 0.2 GiB.

- LinearTimeSC and ProductSC have similar running times, and are order of magnitudes faster than LogSpaceSC. For $n = 24$, LinearTimeSC runs in 4.3s and ProductSC 7.9s. For $n = 28$, LinearTimeSC runs in 67.6s and ProductSC 126.7s.



(a) Running time of the prover.

(b) Memory consumption of the prover.

Figure 7: Comparison of running time and memory of the prover algorithms considered in this paper. Number of variables ranges from 16 to 30 variables. They y-axis is log scaled.

	16	18	20	22	24	26	28	30
Runtime (Seconds)								
LogSpaceSC	0.1	0.5	2.0	8.9	38.7	167.7	724.9	3105.6
LinearTimeSC	0.0	0.1	0.3	1.1	4.3	16.9	67.6	271.1
ProductSC$_2$	0.0	0.1	0.5	2.0	7.9	31.5	126.1	503.3
ProductSC$_4$	0.0	-	0.6	-	10.9	-	179.1	-
Memory Consumption (MiB)								
LogSpaceSC	0.1	0.0	0.0	0.1	0.0	0.1	0.0	0.1
LinearTimeSC	1.7	7.7	31.2	109.3	437.4	1749.1	7001.9	27996.6
ProductSC$_2$	0.2	0.1	0.2	0.7	3.1	13.6	54.6	218.6
ProductSC$_4$	0.0	-	0.1	-	0.1	-	0.9	-

Table 1: Comparison of runtime and memory consumption of prover algorithms using a 128-bit field for input sizes ranging from 16 to 30 variables.

Acknowledgments

We thank Justin Thaler for pointing the connection of this work to [STW24, Appendix G]. We thank Ron Rothblum for discussions related to [Rot24] and [ARR25].

Alessandro Chiesa, Giacomo Fenzi, and Pratyush Mishra are supported in part by the Ethereum Foundation and the Sui Foundation. Pratyush Mishra and Tushar Mopuri are supported in part by a Sony Academic Research Award.

References

[ACFY24] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. "STIR: Reed-Solomon proximity testing with fewer queries". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 380–413.

[ark] arkworks contributors. *arkworks zkSNARK ecosystem*. 2022. URL: https://arkworks.rs.

[ARR25] N. Athamnah, N. Ron-Zewi, and R. Rothblum. *Linear Prover IOPs in Log Star Rounds*. ECCC TR25-090. 2025.

[BBHR18] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. "Fast Reed-Solomon Interactive Oracle Proofs of Proximity". In: *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming*. Vol. 107. ICALP '18. 2018, 14:1–14:17.

[BCHO22] J. Bootle, A. Chiesa, Y. Hu, and M. Orrù. "Gemini: Elastic SNARKs for Diverse Environments". In: *Proceedings of the 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '22. 2022.

[BCRSVW19] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward. "Aurora: Transparent Succinct Arguments for R1CS". In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '19. 2019, pp. 103–128. URL: https://eprint.iacr.org/2018/828.

[BFL91] L. Babai, L. Fortnow, and C. Lund. "Non-Deterministic Exponential Time has Two-Prover Interactive Protocols". In: *Computational Complexity* 1 (1991). Preliminary version appeared in FOCS '90., pp. 3–40.

[BFLS91] L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy. "Checking computations in polylogarithmic time". In: *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing*. STOC '91. 1991, pp. 21–32.

[BFS20] B. Bünz, B. Fisch, and A. Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020, pp. 677–706.

[BHRRS20] A. R. Block, J. Holmgren, A. Rosen, R. D. Rothblum, and P. Soni. "Public-Coin Zero-Knowledge Arguments with (almost) Minimal Time and Space Overheads". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020.

[BHRRS21] A. R. Block, J. Holmgren, A. Rosen, R. D. Rothblum, and P. Soni. "Time- and Space-Efficient Arguments from Groups of Unknown Order". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 123–152.

[BMMNW24] A. Baweja, P. Mishra, T. Mopuri, K. Newatia, and S. Wang. *Scribe: Low-memory SNARKs via Read-Write Streaming*. Cryptology ePrint Archive Report 2024/1970. 2024. URL: https://eprint.iacr.org/2024/1970.

[CBBZ23] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '23. 2023, pp. 499–530.

[CFFZ24] A. Chiesa, E. Fedele, G. Fenzi, and A. Zitek-Estrada. *A Time-Space Tradeoff for the Sumcheck Prover*. Cryptology ePrint Archive Paper 2024/524. 2024. URL: https://eprint.iacr.org/2024/524.

[CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020.

[CMT12] G. Cormode, M. Mitzenmacher, and J. Thaler. "Practical Verified Computation with Streaming Interactive Proofs". In: *Proceedings of the 4th Symposium on Innovations in Theoretical Computer Science*. ITCS '12. 2012, pp. 90–112.

[Coo73] S. A. Cook. "An observation on time-storage trade off". In: *Proceedings of the 5th annual ACM symposium on Theory of computing*. STOC '73. 1973, pp. 29–33.

[CTY11] G. Cormode, J. Thaler, and K. Yi. "Verifying computations with streaming interactive proofs". In: *Proceedings of the VLDB Endowment* 5.1 (2011), pp. 25–36.

[DTT10] A. De, L. Trevisan, and M. Tulsiani. "Time space tradeoffs for attacks against one-way functions and PRGs". In: *Proceedings of the 30th Annual International Cryptology Conference*. CRYPTO '10. 2010, pp. 649–665.

[GKR15] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum. "Delegating Computation: Interactive Proofs for Muggles". In: *Journal of the ACM* 62.4 (2015), 27:1–27:64.

[JKRS09] A. Juma, V. Kabanets, C. Rackoff, and A. Shpilka. "The Black-Box Query Complexity of Polynomial Summation". In: *Computational Complexity* 18.1 (2009), pp. 59–79.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '10. 2010, pp. 177–194.

[LFKN92] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. "Algebraic Methods for Interactive Proof Systems". In: *Journal of the ACM* 39.4 (1992), pp. 859–868.

[LMN25] C. Levrat, T. Medevielle, and J. Nardi. "A divide-and-conquer sumcheck protocol". In: *IACR Commun. Cryptol.* 2.1 (2025), p. 15.

[NTZ25] V. Nair, J. Thaler, and M. Zhu. *Proving CPU Executions in Small Space*. Cryptology ePrint Archive, Paper 2025/611. 2025. URL: https://eprint.iacr.org/2025/611.

[PP24] C. Pappas and D. Papadopoulos. "Sparrow: Space-Efficient zkSNARK for Data-Parallel Circuits and Applications to Zero-Knowledge Decision Trees". In: *Proceedings of the ACM Conference on Computer and Communications Security*. CCS '24. 2024, pp. 3110–3124.

[PP25] C. Pappas and D. Papadopoulos. *Hobbit: Space-Efficient zkSNARK with Optimal Prover Time*. Cryptology ePrint Archive, Paper 2025/1214. 2025. URL: https://eprint.iacr.org/2025/1214.

[PST13] C. Papamanthou, E. Shi, and R. Tamassia. "Signatures of Correct Computation". In: *Proceedings of the 10th Theory of Cryptography Conference*. TCC '13. 2013, pp. 222–242.

[Raz02] R. Raz. “On the complexity of matrix product”. In: *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*. STOC ’02. 2002, pp. 144–151.

[Rot24] R. Rothblum. “A Note on Efficient Computation of the Multilinear Extension”. In: (2024). URL: https://eprint.iacr.org/2024/1103.

[Set20] S. Setty. “Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. 2020, pp. 704–737.

[Sha92] A. Shamir. “IP = PSPACE”. In: *Journal of the ACM* 39.4 (1992), pp. 869–877.

[SP25] N. Singh and S. Patranabis. *SubLogarithmic Linear Time SNARKs from Compressed Sum-Check*. Cryptology ePrint Archive, Paper 2025/908. 2025. URL: https://eprint.iacr.org/2025/908.

[STW24] S. T. V. Setty, J. Thaler, and R. S. Wahby. “Unlocking the Lookup Singularity with Lasso”. In: *Proceedings of the 43rd Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’24. 2024, pp. 180–209.

[SY10] A. Shpilka and A. Yehudayoff. “Arithmetic Circuits: A survey of recent results and open questions”. In: *Foundations and Trends in Theoretical Computer Science* 5.3-4 (2010), pp. 207–388.

[Tha13] J. Thaler. “Time-Optimal Interactive Proofs for Circuit Evaluation”. In: *Proceedings of the 33rd Annual International Cryptology Conference*. CRYPTO ’13. 2013, pp. 71–89.

[VSBW13] V. Vu, S. Setty, A. J. Blumberg, and M. Walfish. “A hybrid architecture for interactive verifiable computation”. In: *Proceedings of the 34th IEEE Symposium on Security and Privacy*. S&P ’13. 2013, pp. 223–237.

[ZCLKZ24] S. Zhang, D. Cai, Y. Li, H. Kan, and L. Zhang. *Epistle: Elastic Succinct Arguments for Plonk Constraint System*. IACR ePrint Report 2024/872. 2024. URL: https://eprint.iacr.org/2024/872.

A Space-efficient prover for sumcheck over multilinear polynomials

We propose a family of prover algorithms for the multilinear-sumcheck protocol: $\{\mathsf{MultilinearSC}_k\}_{k\in[n]}$. The value k regulates the tradeoff between time and space efficiency. Increasing k reduces space consumption while increasing running time.

Outline. We partition the n rounds of the sumcheck protocol in k stages of length $\ell := \frac{n}{k}$.[14] At the start of each stage, the prover performs a precomputation that is then used for the rounds belonging to said stage.

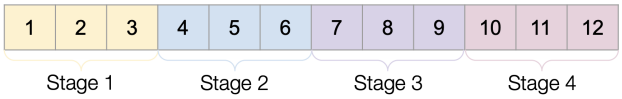


Figure 8: Example of the division of rounds into stages for $n = 12$ and $k = 4$.

Notation. We denote the empty string by ε. Given $\boldsymbol{v} \in \{0,1\}^\ell$ and $a, b \in [\ell]$ with $a \le b$, we define $\boldsymbol{v}[a : b] := (v_a, \ldots, v_b)$ and $\boldsymbol{v}[: b] := \boldsymbol{v}[1 : b]$. Given $\boldsymbol{b} \in \{0,1\}^n$ and a stage $s \in [k]$ we parse $\boldsymbol{b}$ as $\boldsymbol{b} = (\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3)$ where $\boldsymbol{b}_1 \in \{0,1\}^{(s-1)\ell}$, $\boldsymbol{b}_2 \in \{0,1\}^\ell$, and $\boldsymbol{b}_3 \in \{0,1\}^{(k-s)\ell}$. Intuitively, $\boldsymbol{b}_1$ contains the bits related to previous stages, $\boldsymbol{b}_2$ contains the bits related to the current stage, and $\boldsymbol{b}_3$ contains the bits related to future stages. We also divide the verifier randomness $\boldsymbol{r}$ in the same way so that $\boldsymbol{r} = (\boldsymbol{r}_1, \boldsymbol{r}_2, \boldsymbol{r}_3)$.

A.1 Precomputation

The algorithm has k stages and, at the beginning of stage $s \in [k]$, the algorithm precomputes an array $\mathsf{PS}_{(s)}$ of size $2^\ell = N^{1/k}$ that is derived as the partial sums of an auxiliary array $\mathsf{AUX}_{(s)}$. Later, in Appendix A.2, we show how this precomputation allows computing the evaluations of the sumcheck polynomials in stage s. Here we describe how to compute $\mathsf{AUX}_{(s)}$ and then how to derive $\mathsf{PS}_{(s)} := \mathsf{partialSum}(\mathsf{AUX}_{(s)})$.

Computing $\mathsf{AUX}_{(s)}$. If $k = 1$, $\mathsf{AUX}_{(s)}[\boldsymbol{b}] := f(\boldsymbol{b})$. Otherwise, $\mathsf{AUX}_{(s)}$ is defined as follows:

$$\forall\, \boldsymbol{b}_2 \in \{0,1\}^\ell,\ \mathsf{AUX}_{(s)}[\boldsymbol{b}_2] := \sum_{\boldsymbol{b}_1 \in \{0,1\}^{(s-1)\ell}} \chi_{\boldsymbol{b}_1}(\boldsymbol{r}_1) \sum_{\boldsymbol{b}_3 \in \{0,1\}^{(k-s)l}} f(\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3)\ .$$

We efficiently compute this array with the following procedure.

$\mathsf{Aux}^f(\boldsymbol{r}_1)$:
1. If $k = 1$:
   (a) For every $\boldsymbol{b} \in \{0,1\}^n$, set $\mathsf{AUX}_{(s)}[\boldsymbol{b}] := f(\boldsymbol{b})$.
   (b) Return $\mathsf{AUX}_{(s)}$.
2. For every $\boldsymbol{b}_2 \in \{0,1\}^\ell$, initialize $\mathsf{AUX}_{(s)}[\boldsymbol{b}_2] := 0$.
3. Initialize $\mathsf{st} := \mathsf{LagInit}((s-1)\ell, \boldsymbol{r}_1)$.
4. For every $\boldsymbol{b}_1 \in \{0,1\}^{(s-1)\ell}$:
   (a) Compute $(\mathsf{LagPoly}, \mathsf{st}) := \mathsf{LagNext}(\mathsf{st})$.

[14] If n does not divide k, we instead partition n into k stages of length $\ell := \lfloor \frac{n}{k} \rfloor$, and a final stage of length $n - k \cdot \ell$. In this note, we focus on the case where k divides n, but our implementation also supports the case where k does not divide n.

(b) For every $\boldsymbol{b}_2 \in \{0,1\}^\ell$, update $\mathsf{AUX}_{(s)}[\boldsymbol{b}_2]$:

$$\mathsf{AUX}_{(s)}[\boldsymbol{b}_2] := \mathsf{AUX}_{(s)}[\boldsymbol{b}_2] + \mathsf{LagPoly} \cdot \sum_{\boldsymbol{b}_3 \in \{0,1\}^{(k-s)\ell}} f(\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3)$$

5. Return $\mathsf{AUX}_{(s)}$.

The terms $\mathsf{LagPoly}$ are computed efficiently using the method in Section 3.1.

Partial sums. Given an arbitrary array $\boldsymbol{x}$, we write $\boldsymbol{S} := \mathsf{partialSum}(\boldsymbol{x})$ for the array $\boldsymbol{S}$ with $\boldsymbol{S}[-1] = 0$ and $\boldsymbol{S}[i] = \sum_{i=0}^{|\boldsymbol{x}|-1} x_i$. Note that, given $\boldsymbol{S}$, we can express the sum of elements of $\boldsymbol{x}$ between two indices i and j as:

$$x_i + \cdots + x_j = \boldsymbol{S}[j] - \boldsymbol{S}[i-1]$$

Note that computing $\boldsymbol{S}$ from $\boldsymbol{x}$ can be done in a linear pass over $\boldsymbol{x}$ and $O(|\boldsymbol{x}|)$ additions. Once this precomputation is done, each sum of consecutive elements only requires a single subtraction.

A.2 Round computation

Recall that, in round j, the prover aims to compute the evaluations of the polynomial $p_j(\mathsf{X})$ on $\{0,1\}$. We show how, given $\mathsf{PS}_{(s)}$ precomputed at the beginning of the corresponding stage s as in Appendix A.1, these evaluations can be computed efficiently *without making additional passes over the input stream.*

Let $j' := j - (s-1)\ell$ (thus, $j' \in [\ell]$ is the index of the j-th round in stage s). Write $\boldsymbol{b}_2 = (\boldsymbol{b}_2^{(\mathsf{s})}, \boldsymbol{b}_2^{(\mathsf{e})})$ with $\boldsymbol{b}_2^{(\mathsf{s})} \in \{0,1\}^{j'}$, $\boldsymbol{b}_2^{(\mathsf{e})} \in \{0,1\}^{\ell - j'}$. Accordingly, write $\boldsymbol{r}_2^{(\mathsf{s})} \in \mathbb{F}^{j'-1}$ for the current randomness. Then,

$$p_j(\mathsf{X}) = \sum_{\boldsymbol{b}_2 \in \{0,1\}^l} \chi_{\boldsymbol{b}_2^{(\mathsf{s})}}\left(\boldsymbol{r}_2^{(\mathsf{s})}, \mathsf{X}\right) \cdot \mathsf{AUX}_{(s)}[\boldsymbol{b}_2] \ .$$

Which can be rewritten as

$$p_j(\mathsf{X}) = \sum_{\boldsymbol{b}_2^{(\mathsf{s})} \in \{0,1\}^{j'}} \chi_{\boldsymbol{b}_2^{(\mathsf{s})}}\left(\boldsymbol{r}_2^{(\mathsf{s})}, \mathsf{X}\right) \underbrace{\sum_{\boldsymbol{b}_2^{(\mathsf{e})} \in \{0,1\}^{\ell - j'}} \mathsf{AUX}_{(s)}[\boldsymbol{b}_2^{(\mathsf{s})}, \boldsymbol{b}_2^{(\mathsf{e})}]}_{\mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(\mathsf{s})}, \mathbf{1}] - \mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(\mathsf{s})}, \mathbf{0}]} \ , \tag{16}$$

where $\mathbf{1} := 1^{\ell - j'}$ and $\mathbf{0} := 0^{\ell - j'}$. The inner sum can be computed in constant time from $\mathsf{PS}_{(s)}$. Moreover, to efficiently compute the terms $\chi_{\boldsymbol{b}_2^{(\mathsf{s})}}\left(\boldsymbol{r}_2^{(\mathsf{s})}\right)$ at each round the prover stores in memory the tree containing all the Lagrange polynomials relative to the $\boldsymbol{r}_2^{(\mathsf{s})}$, updating its leaves round after round after having received new randomness from the verifier. This uses space $O(N^{1/k})$ and can be efficiently updated at round $j' \in [\ell]$ in $O(2^{j'})$ time. Note that this operation is distinct from that described in Section 3.1, as the size of the tree is small enough that we can afford to completely materialize it into memory. Alternatively, one can also use those same techniques, trading a slightly higher number of field operations for space savings.

$\mathsf{MultilinearSC}_k^f$:
1. Set $\ell := n/k$.
2. For every round $j \in [n]$:

(a) If $(j-1) \bmod \ell = 0$:
  i. Set $s := 1 + (j-1)/\ell$.
  ii. Compute $\mathsf{AUX}_{(s)} := \mathsf{Aux}^f(\boldsymbol{r}_1)$.
  iii. Set $\mathsf{PS}_{(s)} := \mathsf{partialSum}(\mathsf{AUX}_{(s)})$.
  iv. Initialize $\mathsf{LagVector}^{(1)}[\varepsilon] = 1$.

(b) Let $j' := j - (s-1)\ell$ and $\boldsymbol{r}_2^{(s)} := \boldsymbol{r}_2[0 : j'-1]$.

(c) Compute $p_j(0)$ and $p_j(1)$ as

$$p_j(0) := \sum_{\boldsymbol{b}_2^{(s)} \in \{0,1\}^{j'-1}} \mathsf{LagVector}^{(j')}[\boldsymbol{b}_2^{(s)}] \left(\mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(s)}, 0, \mathbf{1}] - \mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(s)}, 0, \mathbf{0}]\right)$$

$$p_j(1) := \sum_{\boldsymbol{b}_2^{(s)} \in \{0,1\}^{j'-1}} \mathsf{LagVector}^{(j')}[\boldsymbol{b}_2^{(s)}] \left(\mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(s)}, 1, \mathbf{1}] - \mathsf{PS}_{(s)}[\boldsymbol{b}_2^{(s)}, 1, \mathbf{0}]\right)$$

(d) Send $p_j(0)$ and $p_j(1)$ to $\mathcal{V}$.

(e) Receive r_j from $\mathcal{V}$.

(f) Update the tree of Lagrange polynomials. For each $\boldsymbol{b} \in \{0,1\}^{j'-1}$:

$$\mathsf{LagVector}^{(j'+1)}[\boldsymbol{b}, 0] := \mathsf{LagVector}^{(j')}[\boldsymbol{b}] \cdot (1 - r_j)$$
$$\mathsf{LagVector}^{(j'+1)}[\boldsymbol{b}, 1] := \mathsf{LagVector}^{(j')}[\boldsymbol{b}] \cdot r_j$$

Figure 9: Prover algorithm for the multilinear-sumcheck protocol. MultilinearSC makes a pass of the input stream f in Item 2(a)ii, and thus makes k input passes in total.

A.3 Asymptotic efficiency

We analyze the time and space complexity of $\mathsf{MultilinearSC}_k$. First, we discuss the complexity of computing $\mathsf{PS}_{(s)}$, then that of computing $p_j(\mathsf{X})$ (given $\mathsf{PS}_{(s)}$), and finally the overall complexity.

Computation of $\mathsf{PS}_{(s)}$. At the start of stage s, the prover computes the evaluations of the Lagrange polynomials in $(s-1)\ell$ variables using the method in Section 3.1, which requires time $O(2^{(s-1)\ell})$. Additionally, the prover populates the table $\mathsf{AUX}_{(s)}$, which requires additional time $O(N)$.

As this operation is repeated k times (once at the start of each stage), the total time complexity is

$$T(N) = \sum_{s=1}^{k} \Big(O(2^{(s-1)\ell}) + O(N)\Big) = k \cdot O(N) \ .$$

Turning to space complexity, at each step the algorithm stores the tables $\mathsf{AUX}_{(s)}$ (which can be deleted after the computation of $\mathsf{PS}_{(s)}$), and the partial sum table $\mathsf{PS}_{(s)}$ both of size $O(2^l)$, the current value of st, $\mathsf{LagPoly}$, and the randomness $\boldsymbol{r}_1$. The space complexity for this is

$$S(N) = O(2^\ell) = O(N^{1/k}) \ .$$

Computation of $p_j(\mathsf{X})$. Write $j' := j - (s-1) \cdot \ell$ for the index of the round j in stage s. Note first that the table $\mathsf{LagVector}$ can be updated in time $O(2^{j'})$. Assuming that the table has been computed, p_j can be

computed as in Equation (16) in time $O(2^{j'})$, and thus the time complexity of the whole algorithm (excluding the precomputation steps) is:

$$T(N) = \sum_{s=1}^{k} \sum_{j'=1}^{\ell} O(2^{j'}) = k \cdot O(N^{1/k}) \ .$$

The only additional memory used in this portion of the computation is that used to store LagVector, of size $O(2^{\ell})$, thus

$$S(N) = O(2^{\ell}) = O(N^{1/k}) \ .$$

Overall complexity. The overall time complexity is

$$T(N) = k \cdot O(N) + k \cdot O(N^{1/k}) = k \cdot O(N) \ ,$$

and the overall space complexity is

$$S(N) = O(N^{1/k}) + O(N^{1/k}) = O(N^{1/k}) \ .$$

Further, the algorithm makes k passes over the input.

B Polynomial commitment scheme for mixed polynomials

If Fig. 4 is used to construct succinct arguments, the argument prover must initialize the polynomial oracles $[\![\tilde{p}]\!], [\![\tilde{q}]\!]$ via a polynomial commitment scheme (PCS), which enables the prover to prove an evaluation claim of $\tilde{p}$ and $\tilde{q}$ at a random point. Prior work on polynomial commitment schemes focuses on univariate polynomials or multilinear polynomials. Since we cannot use these directly, we briefly sketch how to construct a PCS for mixed polynomials that only requires streaming access to the evaluations of the mixed polynomial over G in a lexicographic order and $O(\log N)$ working space.

KZG polynomial commitment scheme. Recall the KZG [KZG10] PCS for univariate polynomials. Let $(\mathbb{H}_1, \mathbb{H}_2, \mathbb{H}_T, H_1, H_2, e) \xleftarrow{\$} \mathsf{SampleGrp}(1^\lambda)$ be a bilinear group where H_1, H_2 are generators of the groups $\mathbb{H}_1, \mathbb{H}_2$ respectively, and $e : \mathbb{H}_1 \times \mathbb{H}_2 \to \mathbb{H}_T$ is a *pairing map* that satisfies the following $e(a \cdot H_1, b \cdot H_2) = e(H_1, H_2)^{a \cdot b}$ for any scalars $a, b \in \mathbb{F}$. Let $\langle \omega \rangle$ be a subgroup of $\mathbb{F}^\times$ of order N. In the setup phase, a structured reference string (SRS) is generated, which consists of the following elements:

$$[H_1 \cdot \mathsf{L}_{\omega^0}(s), H_1 \cdot \mathsf{L}_{\omega^1}(s), \ldots, H_1 \cdot \mathsf{L}_{\omega^{N-1}}(s), H_2 \cdot s]$$

where s is a secret scalar that is disposed after the setup phase.

$\mathcal{P}$ commits to a univariate polynomial $f_0(X) = \sum_{i=0}^{N-1} f_i \cdot \mathsf{L}_{\omega^i}(X)$ of degree $N-1$ by sending the commitment $\mathsf{cm}(f_0) = H_1 \cdot f_0(s) = \sum_{i=0}^{N-1} (H_1 \cdot \mathsf{L}_{\omega^i}(s)) \cdot f_0(\omega^i)$ using the SRS values.

If $f_0(z) = y$ for some $z, y \in \mathbb{F}$, it must be the case that $(X - z)$ divides $f_0(X) - y$. Therefore, $\mathcal{P}$ computes $f_1(X) = \frac{f_0(X) - y}{X - z}$ by computing $f_1(\omega^i) = \frac{f_0(\omega^i) - y}{\omega^i - z}$ for each $i \in \{0, \ldots, N-1\}$. For proving the evaluation claim that $f_0(z) = y$, $\mathcal{P}$ sends the proof $\Pi = H_1 \cdot f_1(s) = \sum_{i=0}^{N-1} (H_1 \cdot \mathsf{L}_{\omega^i}(s)) \cdot f_1(\omega^i)$, which can be computed from the SRS. $\mathcal{V}$ then wants to check that $f_0(s) - y = (s - y) \cdot f_1(s)$. Assuming that the commitments to f and g are binding and the strong Diffie-Hellman assumption (see [KZG10] for details), this reduces to checking $e(\mathsf{cm}(f_0) - H_1 \cdot y, H_2) = e(\Pi, H_2 \cdot (s - y))$, which can be done by $\mathcal{V}$ efficiently using the SRS and the pairing map e.

Extending to mixed polynomials. Let $t = s + k - 1$ and let $f_0(\boldsymbol{X}) : \mathbb{F}^t \to \mathbb{F}$ be a mixed polynomial as defined in Eq. (12). Recall that $\mathcal{P}$ has streaming access to the evaluations of f_0 over $G = \mathbb{G}_1 \times \ldots \times \mathbb{G}_t$ in a lexicographic order, i.e., $[f_0(\boldsymbol{g})]_{\boldsymbol{g} \in G}$. If $f_0(\boldsymbol{z}) = y$ where $\boldsymbol{z} \in \mathbb{F}^t$ and $y \in \mathbb{F}$, then there exist polynomials f_j, h_j such that $h_1(\boldsymbol{X}) = f_0(\boldsymbol{X}) - y$ and for all $j \in [t]$,

$$h_j(X_j, \ldots, X_t) = (X_j - z_j) \cdot f_j(X_j, \ldots, X_t) + h_{j+1}(X_{j+1}, \ldots, X_t) \ . \tag{17}$$

The PCS for mixed polynomials is defined as follows:

- **Setup:** Sample $\boldsymbol{s} = (s_1, \ldots, s_t) \xleftarrow{\$} \mathbb{F}^t$ and initialize the following SRS:

$$\left[\left[H_1 \cdot \prod_{j=i}^{t} \mathsf{L}_{j, g_j}(s_j) \right]_{\boldsymbol{g} \in G, i \in [t]}, [H_2 \cdot s_j]_{j \in [t]} \right] \ .$$

  $\boldsymbol{s}$ are secret scalars that are disposed after the setup phase.
- **Commitment:** Similar to KZG, the commitment to f_0 is simply

$$\mathsf{cm}(f_0) = H_1 \cdot f_0(s) = \sum_{\boldsymbol{g} \in G} (H_1 \cdot \prod_{j=1}^{t} \mathsf{L}_{j, g_j}(s_j)) \cdot f_0(\boldsymbol{g}) \ ,$$

  which can be clearly computed from the SRS in a streaming manner with only $O(\log N)$ working space.

- **Evaluation proof:** $\mathcal{P}$ proves to $\mathcal{V}$ of the evaluation claim $f_0(\boldsymbol{z}) = y$ by proving Eq. (17) at the secret point $\boldsymbol{s} = (s_1, \ldots, s_t)$. To do so, $\mathcal{P}$ must first compute f_j, h_j for all $j \in [t]$, and then send the proof

$$\boldsymbol{\Pi} = [H_1 \cdot f_j(s_j, \ldots, s_t)]_{j \in [t]} \parallel [H_1 \cdot h_j(s_j, \ldots, s_t)]_{j \in \{2,\ldots,t\}} \ .$$

  Clearly, using $\boldsymbol{\Pi}$ and $\mathsf{cm}(f_0)$, $\mathcal{V}$ can check Eq. (17) by checking the following

$$\begin{aligned} e(\mathsf{cm}(f_0) - H_1 \cdot y, H_2) &= e(\Pi_1, H_2 \cdot (s_1 - z_1)) \text{ and} \\ \forall j \in \{2, \ldots, t\} : e(\Pi_{j+t-1} - \Pi_{j+t}, H_2) &= e(\Pi_j, H_2 \cdot (s_j - z_j)) \ . \end{aligned}$$

  We omit discussion of the security of this PCS, since the argument is similar to the security of the PST [PST13] PCS for multilinear polynomials.

Efficiency of the Evaluation Proof. Firstly, if $\mathcal{P}$ can compute streams of the evaluations of f_j and h_j over $\mathbb{G}_j \times \ldots \times \mathbb{G}_t$ in a lexicographic order, then $\mathcal{P}$ can compute the corresponding commitments and proofs with only $O(\log N)$ working space. Now, given streaming access to $h_j(X_j, \ldots, X_t)$, which is divisible by $(X_j - z_j)$, $\mathcal{P}$ computes $h_{j+1}(X_{j+1}, \ldots, X_t)$ by obtaining the partial evaluation of h_j at $X_j = z_j$:

$$\begin{aligned} h_{j+1}(g_{j+1}, \ldots, g_t) &= h_j(z_j, g_{j+1}, \ldots, g_t) \\ &= \sum_{g_j \in \mathbb{G}_j} \mathsf{L}_{j,g_j}(z_j) \cdot h_j(g_j, g_{j+1}, \ldots, g_t) \ , \end{aligned}$$

for all $g_{j+1} \in \mathbb{G}_{j+1}, \ldots, g_t \in \mathbb{G}_t$. Finally, using streaming access to both $h_j(X_j, \ldots, X_t)$ and $h_{j+1}(X_{j+1}, \ldots, X_t)$, $\mathcal{P}$ can compute $f_j(X_j, \ldots, X_t)$ as follows:

$$f_j(g_j, \ldots, g_t) = \frac{h_j(g_j, \ldots, g_t) - h_{j+1}(g_{j+1}, \ldots, g_t)}{g_j - z_j} \ ,$$

for all $g_j \in \mathbb{G}_j, g_{j+1} \in \mathbb{G}_{j+1}, \ldots, g_t \in \mathbb{G}_t$. Each f_j, h_j pair requires $O(|\mathbb{G}_j| \cdots |\mathbb{G}_t|)$ time to compute, and therefore the total time to compute all pairs is $\sum_{j=1}^{t} O(|\mathbb{G}_j| \cdots |\mathbb{G}_t|) = O(N)$.